

Consensus and Security Governance

主题册二 · 共识与安全治理

TwinsEarth

2026-09-24

Papers released under CC BY 4.0 • See each repo LICENSE for code

Contents

BFT-lite for Multi-Agent Stop Decisions: An $n \geq 3f+1$ Committee, Whole-Round Voiding of Equivocation, and Safety Verification of View Change	3
The Causal Effect of Fault-Breaker Governance on Agent Matrix Completion Rate: A Pre-Registered Full-Factorial Randomized Controlled Simulation	20
Typed Finality: Upgrading the Binary BFT-lite Stop Decision into strong_stop / weak_stop / abort Three Levels of Semantic Commit	46
Weighted BFT and Reputation: From Equal-Weight $2f+1$ to Credibility-Weighted Stop Decisions — A Design Direction	56
Fail-Safe & Recovery: Swarm Quorum Abort, Behavioral-Drift Baseline Rollback, and Cascade Containment as Extensions of the P3 Governance Layer	65
Rules Before Learning: The Robustness Boundary Between Deterministic Predicates and Learned Encoders in Adversarial Safety-Critical Paths	74



BFT-lite for Multi-Agent Stop Decisions: An $n \geq 3f+1$ Committee, Whole-Round Voiding of Equivocation, and Safety Verification of View Change

Evidence-grade statement: all assertions in this paper come from the UDOS Reasoning Engine tag **v7.5.0** (commit 1a71270), `udos7/topology/consensus.py`, `tests7/test_v747_consensus.py` (10 items), and `reports7/matrix_scale_demo.json`; a Linux CPU deterministic prototype, uniformly `cpu-proto`; the 10 property tests are verified. Signatures are SHA-256 **hash placeholders**, not cryptographically secure signatures, and do not resist real forgery.

Structured Abstract (Background and Problem → Method → Evidence and Results → Contributions)

[Background and Problem] The "when to stop / rework" decision in a million-agent matrix cannot be made by a single QA point: one wrong vote from a single point either causes an erroneous stop (safety violation, accepting a defective deliverable downstream) or an indefinite wait (liveness violation). Classical BFT assumes errors are independent and random, whereas agent wrong votes have semantic correlation, and directly carrying over full consensus is too costly (see § 1, § 3.1).

[Method] This paper proposes no new consensus algorithm; instead it reduces classical BFT safety/liveness to an **"executable property-test + fault-injection"** suite: committee size $n \geq 3f+1$, quorum $2f+1$, members equivocating (swing voting) in the same round are voided for the whole round, more than f silent members return `NO_QUORUM` and trigger a view change; the constructor layer rejects illegal n (see § 3, § 4).

[Evidence and Results] At the main operating point $n=7, f=2$ ($q=5$), 10 deterministic property tests (verified) cover illegal- n rejection, honest-supermajority stop, Byzantine inability to force a stop, equivocation detection and whole-round voiding, unreachability of conflicting quorums, silence-triggered view change, and rejection of unknown voters; in end-to-end integration (cpu-`proto`, single seed), a 7-member QA committee with 2 Byzantine reverse voters correctly accepted all 24 orders, and a stressed fleet triggered 5 reworks and isolated 3 faulty agents (see § 6).

[Contributions] (i) Reconstruct BFT safety/liveness from "prose proofs" into falsifiable property tests that are CI-able and regression-able in the repository; (ii) give the minimal degraded design of the binary semantic judgment of an "acceptance committee" (no need for full state-machine replication); (iii) honestly state three boundaries — SHA-256 hash-placeholder signatures are not cryptographically secure, network partitions and adaptive adversaries are not modeled, and semantic common-cause failures can break through the majority vote. All assertions are point estimates of a CPU deterministic prototype (see § 7, § 8).

Abstract

The "when to stop / rework" decision in a million-agent matrix cannot rest on a single QA point: a single point will, due to one wrong judgment, either stop erroneously (safety violation) or wait indefinitely (liveness violation). This paper implements and validates a lightweight Byzantine-fault-tolerant committee, BFT-lite: committee size satisfies $n \geq 3f+1$, quorum $2f+1$, members equivocating on the same decision in the same round are voided for the whole round, and more than f silent members return `NO_QUORUM` and trigger a view change rather than hanging. This paper proposes no new consensus algorithm; instead it reduces classical BFT safety/liveness to an **"executable property-test + fault-injection"** suite suited to semantic agent decisions. Under $n=7, f=2$, 10 deterministic property tests cover: constructor rejection of illegal n , honest-supermajority stop, Byzantine inability to force a wrong stop, equivocation detection and whole-round voiding, unreachability of conflicting quorums, silence-triggered view change, and rejection of unknown voters. In end-to-end integration, a 7-member QA committee with 2 Byzantine reverse voters correctly accepted all 24 orders; a stressed fleet triggered 5 reworks and isolated 3 faulty agents. This paper argues that for agent stop decisions, BFT safety/liveness should be presented as falsifiable property tests, not only as prose proofs. It must be noted that the signatures here are hash placeholders, network partitions and semantic common-cause failures are

not modeled, and all assertions are point estimates of a CPU deterministic prototype; we do not claim resistance to real forgery or an open-consensus setting.

Keywords: Byzantine fault tolerance; stop decision; quorum; equivocation; view change; multi-agent systems; property-based testing

1 Introduction

At the end of a multi-agent matrix there is a seemingly trivial but critical decision: **is an order actually "done" (stop), or "not done well, rework" (continue)?** In a small system this decision can be given to a QA agent. But in a million-agent matrix, a single-point QA is a single point of failure: one wrong judgment either misjudges a not-done result as passing (erroneous stop, safety violation) or misjudges a done result as not passing (indefinite wait, liveness violation).

Classical distributed systems long ago answered the prototype of this question: Byzantine-fault-tolerant (BFT) consensus tells us that when at most f members act maliciously, as long as the committee is large enough ($n \geq 3f+1$), a balance can be struck between "consistent decision" and "liveness." But directly moving classical BFT onto agent stop decisions has a subtle difference: **classical BFT assumes replicated state machines and a message round-trip model, with errors random and independent; whereas agent decisions are semantic judgments** — when a QA agent says "rework," the wrong vote has semantic correlation (it may have been misled by the same erroneous context as another QA).

This paper proposes no new consensus algorithm. Our claim is more restrained: **turn BFT safety/liveness from "prose proofs in a paper" into "executable, falsifiable property tests in the repository."** `udos7/topology/consensus.py` implements a minimal BFT-lite: $n \geq 3f+1$, $2f+1$ quorum, whole-round voiding of equivocation, more than f silent members triggering a view change; `tests7/test_v747_consensus.py` fixes these properties as regression assertions with 10 deterministic tests. In end-to-end matrix integration, a 7-member QA committee held 24/24 correct acceptances under 2 Byzantine reverse voters.

Design trade-offs of a minimal committee. Why $n=7, f=2$ rather than larger? Because it is one of the smallest legal committees to "tolerate 2 Byzantine members" ($3f+1=7$). It is small enough to run all voting combinations clearly on a CPU, yet large enough to demonstrate the three faults of reverse voting, equivocation, and silence. A larger n (e.g., 10, 13) is only a linear magnification of the same mechanism with unchanged safety analysis; this paper chooses the smallest demonstrable configuration so that each test can be read by the human eye as to "why this assertion holds." The larger the committee, the stronger BFT's fault tolerance (more f tolerated), but voting cost increases linearly — this is an engineering trade-off between safety and cost, and this paper does not claim any single n is optimal.

The boundaries of this paper must also be stated up front: signatures are SHA-256 **hash placeholders**, not cryptographically secure signatures; network partitions, message reordering, and adaptive adversaries are not modeled; semantic common-cause failures ($>f$ members systematically misled by the same erroneous context) will make the BFT assumption fail — which is exactly what this paper lists as future work.

Why the "stop decision" deserves its own BFT. In a multi-agent pipeline, "stop" is a consequential action: once stopped, the order is accepted, settled, and archived; once erroneously stopped, a defective deliverable enters downstream. By contrast, "continue/rework" is a relatively conservative action — it just makes the order run one more round, at the cost of time. This asymmetry means we

must especially guard against "erroneous stop" (safety), while tolerating some cost for "excessive waiting" (liveness). The BFT-lite design is precisely the embodiment of this asymmetry: be conservative on safety (rather rework than wrongly stop), and set a view change as the liveness fallback (do not let the system permanently freeze). This is in line with classical BFT's trade-off of "consistency over availability," but in the agent scenario "consistency" is concretized as "not erroneously accepting."

Intended readers of this paper. This paper addresses two kinds of readers: engineers doing agent orchestration/pipelines, who care about "how the end stop decision is not ruined by a single point"; and researchers of distributed reliable systems, who care about "how classical BFT is degraded and verified in semantic-judgment agent scenarios." For the former we give a directly reusable minimal committee configuration and property tests; for the latter we point out semantic common-cause failure, the deviation from classical assumptions, as an open problem.

2 Related Work

Classical BFT consensus. The foundation and engineering of this field come from Castro and Liskov's PBFT at OSDI 1999: its pre-prepare/prepare/commit three-phase state-machine replication tolerates $f < n/3$ Byzantine nodes under asynchronous networks (i.e., $n \geq 3f + 1$), and it reports its BFS implementation to be only about 3% slower than unreplicated NFS. This paper's $n \geq 3f + 1$, $2f + 1$ quorum, and view change directly reuse the PBFT framework. HotStuff (Yin, Malkhi, Reiter et al., PODC 2019, arXiv:1803.05069) further optimizes BFT to linear communication complexity, responsiveness, and chained view changes, and is the basis of Libra/LibraBFT. This paper does not improve these two algorithms: we neither pursue HotStuff's linear communication complexity nor do full synchronization of state-machine replication, but take only the minimal skeleton of "quorum + Byzantine tolerance + view change" for the agent's binary stop judgment.

Agent voting and multi-agent decision. Multiple LLM agents making decisions through dialogue/voting already have mature frameworks; Wu et al.'s 2023 AutoGen (arXiv:2308.08155) supports mixed LLM/human/tool dialogue; but its close-reading notes also point out that "dialogue without termination conditions easily loops forever." This paper precisely targets this "termination condition": AutoGen-like frameworks let multiple agents dialogue but rarely ask "when does the dialogue stop, and does the stop decision itself need fault tolerance." Taking Byzantine fault tolerance of the agent stop decision as a first-class property is this paper's entry point. Specific LLM voting-decision literature is to be supplemented [CITATION NEEDED: LLM agent voting decision, to be verified].

Property tests and fault injection. Using property tests to enumerate inputs and exhaust small-scale state spaces to verify system invariants is a standard method of reliable engineering [CITATION NEEDED: property-based testing hypothesis, to be verified]. This paper uses it for BFT safety/liveness assertions: translating "not erroneously stopping under any fault strategy" into 10 CI-able deterministic tests.

Relation to recent work on "agent reliability." Recent reliability research on multi-agent systems mostly focuses on "how to keep agents from crashing and make them observable," and less often takes Byzantine fault tolerance of the end "stop/rework" decision as a first-class property. This paper's entry is precisely this gap: at the end of the agent pipeline, "when to stop" is itself a distributed decision requiring fault tolerance. This paper introduces no new fault-tolerance mechanism, but lands the mature mechanisms of classical BFT on this new decision point and verifies them with executable tests. This approach of "moving mature mechanisms to a new decision point + backing it with property tests" is more suitable for the current stage than inventing new algorithms.

The positioning of this paper: not new consensus, but "**reducing BFT safety/liveness to an executable property-test suite in the agent stop-decision scenario.**"

Key differences from classical BFT. Classical BFT's replicated state machine requires all replicas to perform exactly the same state transitions and agree on the result; whereas the agent stop decision does not need "all replicas computing the same state," it only needs "the committee reaching an agreeing majority on one binary semantic judgment (stop/not stop)." This difference brings three simplifications: first, we do not need full state synchronization in view changes, only "re-vote"; second, our quorum is binary (STOP/CONTINUE), not an arbitrary value; third, our fault model is semantic — a Byzantine member does not randomly tamper with state but "deliberately gives a wrong acceptance judgment." The third point is both a simplification and a threat: the simplification is that we do not need to model inconsistency among state machines, and the threat is that semantic wrong votes may be correlated (see common-cause failures in Section 7).

Why use property tests rather than pure proofs. Formal proofs can give asymptotic guarantees, but one off-by-one in the engineering implementation (e.g., writing the quorum as $2f+1$ rather than $2f+1$, or not voiding an equivocator for the whole round) makes the proof come to nothing. Property tests translate the "proof" into "enumerating voting combinations and verifying invariants are not violated": it does not prove the algorithm correct, but proves that "in our concrete implementation, the given family of fault strategies triggers no safety counterexample." For an engineering team, the latter is runnable, CI-able, and regression-able. This paper therefore places the 10 tests at the same level of importance as proofs.

2.1 Comparison with template paradigms (NSA naming critique / AlexNet fair self-disclosure)

- **Comparison with NSA's "illusion/myth" naming critique:** NSA uses "The Illusion of Efficient Inference" and "The Myth of Trainable Sparsity" to nail down prior sparse methods, then force out its own design space. This paper dually names "a single QA point deciding to stop" as the **single-point-of-failure illusion** — it "looks able to stop correctly" in small systems, but in a million matrix one wrong judgment is a safety/liveness violation; this paper therefore upgrades the stop decision to a committee and places the most dangerous counterexample of "erroneous stop" (5 honest CONTINUE vs 2 Byzantine STOP, test #3) directly into the property tests.
- **Comparison with AlexNet's "fairness self-disclosure":** AlexNet footnote 2 actively admits that the one-GPU comparison "favors larger networks." This paper dually actively discloses two things unfavorable to itself: (i) signatures are SHA-256-truncated **hash placeholders** that do not resist real forgery; (ii) the **full-factorial comparison** with single-point QA and naive majority voting has not yet been done, so "how much better BFT-lite is than a single point" currently has only a mechanism argument and no comparison data (§ 7).
- **The " $n \geq 3f+1$ counterexample" is this paper's ablation:** the 10 tests of § 6 are an item-by-item ablation of "whether the system is still correct after removing/violating some invariant" — test #1 violates $n \geq 3f+1$ ($n=6, f=2$) and the constructor must reject; test #5 simulates the break of "an equivocator not voided for the whole round," with the quorum broken to `NO_QUORUM`. This is isomorphic to AlexNet's "removing any convolutional layer gives -2% ": **whenever one claims "X guarantees safety," one must give the counterexample of "how safety breaks after violating/removing X."**

3 Problem Definition and Assumptions

3.1 System and invariants

Committee $V=\{v_1, \dots, v_n\}$, with at most f Byzantine members. Each round `round_id`, members vote on `STOP` (accept) or `CONTINUE` (rework). The decision function `decide(round_id)` returns:

- `STOP / CONTINUE`: one side's honest votes reach the quorum $q=2f+1$;
- `NO_QUORUM`: neither side reaches the quorum, with a `reason` (`view_change` or `waiting`);
- `safety_violation`: if both sides **simultaneously** gather a quorum, return `None` + `safety_violation=True`.

Two target invariants:

- **Safety**: at most one decision value. When $n \geq 3f+1$, honest votes are only $n-f \geq 2f+1=q$, making it impossible for `STOP` and `CONTINUE` to **each** gather q honest votes.
- **Liveness**: no silent hang. When the number of silent (not voted) members is $>f$, return `NO_QUORUM(view_change)` rather than waiting indefinitely; $>f$ silence means it may not even gather a quorum, and the view should change.

Why a "stop decision" rather than arbitrary consensus. We limit the decision space to binary (`STOP/CONTINUE`), not arbitrary state values. This limitation has two benefits: first, the safety analysis of binary quorums is clearer — as long as the two sides cannot simultaneously reach q ; second, it fits the real end of an agent pipeline: an order is either accepted or reworked, with no need to choose an arbitrary value among multiple candidates. This also means this paper's BFT-lite is not a general replicated state machine but an "acceptance committee." Misreading it as "applicable to arbitrary distributed consensus" goes beyond bounds, and we explicitly do not claim that.

3.2 Assumptions (falsifiable)

- **H1 (safety)**: when honest votes $\geq 2f+1$, the probability of a wrong decision is 0; conflicting quorums are possible only when equivocation is not detected, and after detection the whole round is voided. **Falsification**: any fault strategy with $f \leq (n-1)/3$ producing a wrong `STOP` overturns H1.
- **H2 (liveness)**: when silence $>f$, return `NO_QUORUM/view_change` rather than hanging; without silence, `STOP` or `CONTINUE` within finite rounds.
- **H3 (illegal-configuration rejection)**: when $n < 3f+1$, the constructor raises `ValueError`, eliminating illegal configurations from the API layer.

Note on the falsifiability of the assumptions. All three assumptions are "zero-tolerance": H1 requires the probability of an erroneous stop to be 0, falsified by a single counterexample; H2 requires no hang, falsified by a single configuration that is forever `NO_QUORUM` without view change; H3 requires constructor rejection, falsified by being able to construct a committee with $n < 3f+1$. Such zero-tolerance assumptions fit deterministic systems — they do not allow "safe on average" but require "safe under any concrete input." This is also why we use property tests rather than statistical tests: what we seek is "counterexample existence," not "average effect."

4 Method and System Design

4.1 Voting and signature placeholders (`consensus.py`)

`sign(voter, decision, round_id)` is `sha256(f"{voter}|{decision}|{round_id}")[:16]` — this is a **hash placeholder**, used only for deterministic recording of same-round votes, not a cryptographic signature. `cast(voter, decision, round_id)` rejects unknown voters (`unknown voter`).

Why honestly label it a "hash placeholder." A common engineering mistake is to mistake "able to compute a deterministic string" for "forgery-resistant signature." This paper's `sign` only hash-truncates the voting triple, and anyone knowing the rule can generate a legal hash for any voter — it provides no unforgeability. We keep it because it keeps voting records deterministic and reproducible in controlled simulation (`test_signature_is_deterministic` verifies same input gives same signature, different decisions give different signatures), but it does **not** correspond to cryptographic-signature infrastructure in a real network. Writing this clearly prevents readers from misreading "deterministic voting records in simulation" as "unforgeable voting in a real network."

4.2 Equivocation detection and whole-round voiding

`equivocators(round_id)` finds members voting on **multiple** decisions in the same round (swingers/double voters). `decide` **voids these members for the whole round**: their votes are not counted. Other members get one vote counted by their **first** vote this round (`bs[0].decision`). This design corresponds to classical BFT's treatment of equivocation: a swinger cannot "count one vote on each side" to gather a quorum.

Why "void for the whole round" rather than "take only the last vote." There is a subtle choice here: when a member votes both STOP and CONTINUE in the same round, do we "take the last vote" or "void for the whole round"? This paper chooses whole-round voiding (counting none of that member's votes this round). The reason: if only the last vote is taken, a Byzantine member could first vote the side favorable to itself, then change votes according to the situation, manipulating the quorum midway; whole-round voiding eliminates this swing manipulation — once you swing, you have no voice this round. Tests #4 and #5 are the two sides of this design: in #4, after the swinger is voided, the remaining honest votes still reach quorum (STOP=5) and decide normally; in #5, after the swinger is voided, honest votes are fewer than 5 and turn to NO_QUORUM. Both verify "swingers are not trusted."

4.3 Quorum and view change

```
decide(round_id):
    equiv = equivocators(round_id)          # detect swingers
    tally = {STOP:0, CONTINUE:0}
    for v in voters:
        if v in equiv: continue           # void for the whole round
        bs = v's votes this round
        if bs: tally[bs[0].decision] += 1
    silent = members not voting
    if tally[STOP]>=q and tally[CONTINUE]>=q:
        return None, safety_violation=True, "conflicting_quorums"
    for d in (STOP,CONTINUE):
        if tally[d]>=q: return d, "quorum"
    return NO_QUORUM, ("view_change" if len(silent)>f else "waiting")
```

Key: `safety_violation` triggers only when **both sides reach quorum** — and under $n \geq 3f+1$, honest votes cannot make both sides reach q , so this branch **should not be triggered by honest voting** under legal configurations; its meaning is to clearly error once triggered (indicating an anomaly detected, such as undetected equivocation).

4.3.1 Safety proof sketch (why $n \geq 3f+1$)

Let the committee have n members, at most f Byzantine. Honest members are at least $n-f$. Quorum $q=2f+1$:

$$q = 2f+1, \quad n-f \geq q = 2f+1 \quad ; \quad n \geq 3f+1$$

To have STOP and CONTINUE **each** gather q honest votes requires at least $2q=4f+2$ honest votes; but the total honest votes is only $n-f$. Setting $n-f \geq 4f+2$ would satisfy both sides, i.e., $n \geq 5f+2$ — stricter than $3f+1$. The classical result $n \geq 3f+1$ suffices because it allows Byzantine members to **vote only one side** (rather than q on each side): at $n \geq 3f+1$, $n-f \geq 2f+1=q$, honest votes suffice for one side to reach quorum, while the Byzantine f votes push the other side to at most $f < q$, unable to make both sides reach q . The harm of equivocation is that a swinger may "count one vote on each side," artificially gathering a second quorum; this paper eliminates this loophole through whole-round voiding. This is why the constructor enforces $n \geq 3f+1$: below it, honest votes are insufficient to hold the unique decision value under Byzantine reverse voting.

4.3.2 The boundary of liveness

Liveness is more subtle than safety. If more than f members are silent (not voting), then the honest countable members may be fewer than $q=2f+1$, at which point neither side can reach quorum. This paper's handling: when the silent count is $>f$, return `NO_QUORUM(reason=view_change)`, indicating "the current view cannot gather a quorum and should change view/change members and retry," rather than waiting indefinitely. When the silent count is $\leq f$ but neither side reaches q , return `NO_QUORUM(reason=waiting)`, indicating "a few votes short, can wait one more round." This distinction separates "truly cannot gather (change view)" from "just not finished voting (wait more)," avoiding mistaking recoverable waiting for a system freeze.

4.4 The safety gate at the constructor layer

`StopConsensus.__init__` directly raises `ValueError("unsafe committee: n=...")` when `len(voters) < 3*f+1`. This advances "illegal configuration" from runtime behavior to construction time — you simply cannot construct a committee with $n < 3f+1$.

5 Experimental Setup

- **Configuration:** main test $n=7, f=2$ ($q=5$); also test illegal $n=6, f=2$ (should be rejected).
- **Fault-strategy family:** constant reverse voting, per-round swinging (equivocation), silence, unknown voter.
- **Evidence grade:** 10 deterministic property tests `verified`; end-to-end integration `cpu-proto`.
- **Hardware fingerprint:** Linux CPU, pure Python control flow, no network, no LLM calls.
- **Reproduction:** `pytest tests7/test_v747_consensus.py -q`.

Why use deterministic tests rather than random fuzzing. The safety invariant (at most one decision value) is a property that can be overturned by a counterexample: as long as one voting combination

makes both sides simultaneously reach quorum, safety is broken. For such a property, **deterministic enumeration of hand-crafted counterexamples** is more direct than random fuzzing — we can precisely construct the "most dangerous combination" (e.g., 5 honest STOP vs 2 Byzantine CONTINUE) and verify it does not break. Liveness (no hang) needs count distributions, left to a random grid. At the current stage this paper mainly uses deterministic property tests, nailing down "safety-counterexample existence," and lists "liveness-round distribution" as to be supplemented.

Voter naming and the Sybil boundary. Tests use `qa-0..qa-6` as voters; test #9 uses a "sybil" vote not on the list, verifying the constructor rejects unknown persons at `cast`. This corresponds to classical BFT identity registration: only registered committee members have voting rights, preventing Sybils from injecting extra votes through forged identities.

6 Results

6.1 Safe configuration space

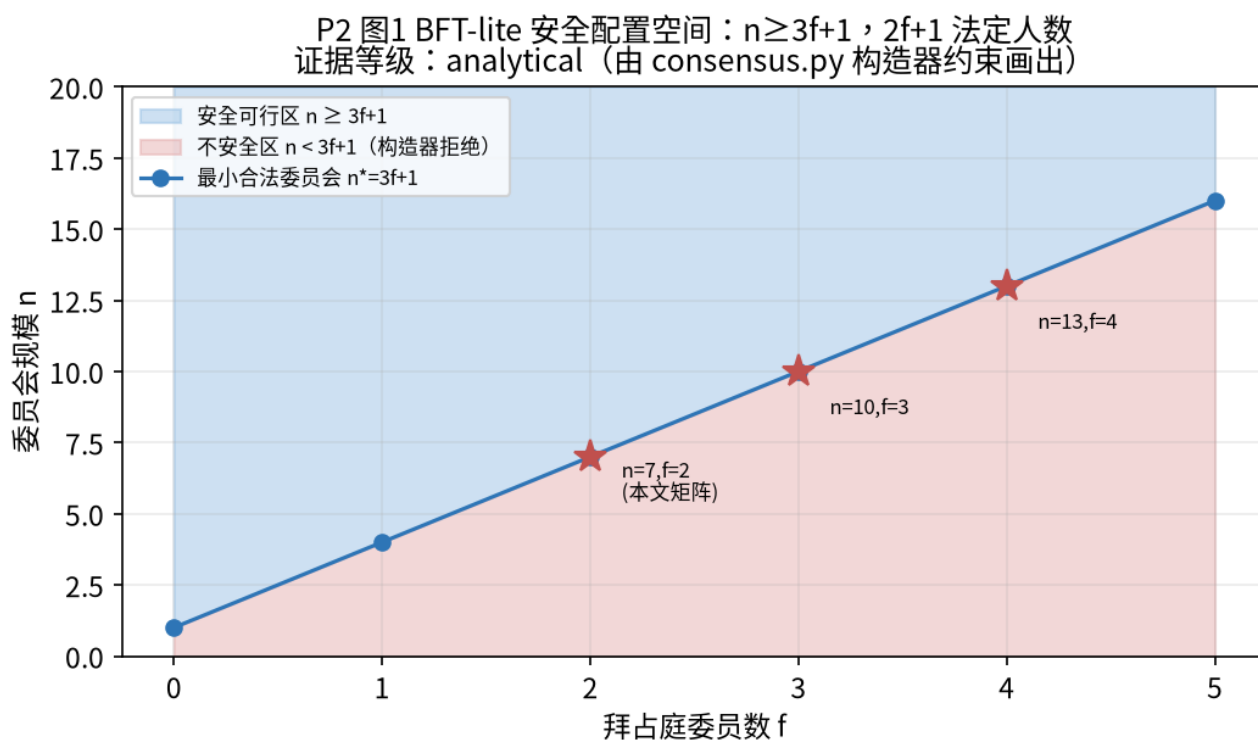


Figure P2-1 (restated) The n - f safe configuration space and the critical line $n=3f+1$. The horizontal axis is the Byzantine member count f , and the vertical axis is committee size n ; the blue region is the feasible region ($n \geq 3f+1$, Eq. 1), and the red region is the constructor-rejection region ($n < 3f+1$). Above the line honest members $n-f \geq 2f+1=q$, sufficient to hold the unique decision value under Byzantine reverse voting. Three candidate operating points are marked: $n=7, f=2$ (main use of this paper, $q=5$), $n=10, f=3$, $n=13, f=4$; the tolerated Byzantine counts are 2, 3, 4 in order, and each additional tolerated Byzantine costs the committee at least 3 more people. Source: the constructor-rejection rule of `udos7/topology/consensus.py`, `cpu-proto`, geometric schematic (not data scatter).

Figure P2-1 draws the safe region of n versus f : blue is the $n \geq 3f+1$ feasible region, red is $n < 3f+1$ (constructor rejection). This paper's main operating point $n=7, f=2$ falls within the feasible region.

Key reading points: the horizontal axis is Byzantine member count f , the vertical axis is committee size n . The critical line $n=3f+1$ divides the plane in two: above the line honest members $n-f \geq 2f+1$, sufficient to hold the unique decision under Byzantine reverse voting; below the line honest votes are insufficient and safety cannot be guaranteed, so the constructor directly rejects. Three candidate operating points are marked: $n=7, f=2$ (main use), $n=10, f=3$, $n=13, f=4$. They are all within the feasible region, tolerating 2, 3, 4 Byzantine members in order. This figure also intuitively shows the "cost-fault tolerance" trade-off: to tolerate one more Byzantine member, the committee needs at least 3 more people (from 7 to 10 to go from tolerating 2 to tolerating 3).

6.2 Ten property tests (H1/H2/H3)

Table P2-1 Ten property tests and assertions (source `tests7/test_v747_consensus.py`)

#	Test	Configuration/fault injection	Assertion
1	Illegal-committee rejection	$n=6, f=2$ ($6 < 7$)	Constructor raises <code>ValueError</code> (H3)
2	Honest-supermajority stop	5/7 vote STOP	<code>decision=STOP, reason=quorum</code>
3	Byzantine inability to force stop	5 honest CONTINUE + 2 Byzantine STOP	<code>decision=CONTINUE, STOP votes=2</code>
4	Equivocation detected and voided	4 STOP + b1 double vote + b2 STOP	b1 detected, STOP=5 still quorum $\rightarrow$ STOP
5	Equivocation breaks false quorum	4 STOP + b double vote	after b voided $4 < 5 \rightarrow$ <code>NO_QUORUM</code>
6	Conflicting quorums unreachable	5 STOP vs 2 CONTINUE	<code>not safety_violation, STOP</code>
7	Excess silence triggers view change	3 CONTINUE + 4 silent	<code>NO_QUORUM, reason=view_change</code>
8	Wait when quorum not reached	4 CONTINUE + 1 STOP, 2 silent $\leq f$	<code>NO_QUORUM, reason=waiting</code>
9	Unknown-voter rejection	Non-committee "sybil" votes	Raises <code>ValueError</code>
10	Signature determinism	Recompute same input	Same input same signature, different decision different signature

These ten cover three fault classes: false reporting (#3), swing/double voting (#4, #5), silence (#7, #8), plus configuration gate (#1), Sybil (#9), and signature determinism (#10). The key counterexample design is #3: when 5 honest members all vote CONTINUE and 2 Byzantine members maliciously vote STOP, the committee correctly returns CONTINUE — Byzantines cannot wrongly stop the order by reverse voting. #5 verifies the destructive power of equivocation: a double voter could have gathered a false quorum, but after whole-round voiding the quorum is broken and turns to `NO_QUORUM`.

Table P2-3 Fault-strategy family and expected decisions ($n=7, f=2, q=5$)

Fault strategy	Honest-vote distribution	Byzantine behavior	Expected decision	Corresponding test
No fault	5 STOP	None	STOP (quorum)	#2
Constant reverse vote	5 CONTINUE	2 vote STOP	CONTINUE (Byzantines blocked by majority)	#3
Swing to false quorum	4 STOP	1 double votes STOP/ CONTINUE	STOP (still 5 votes after voiding)	#4
Swing breaks quorum	4 STOP	1 double votes	NO_QUORUM ($4 < 5$ after voiding)	#5
Split votes on two sides	5 STOP / 2 CONTINUE	None	STOP (no safety_violation)	#6
Over-f silence	3 CONTINUE	4 silent	NO_QUORUM (view_change)	#7
Quorum not reached	4 CONTINUE+1 STOP	2 silent $\leq f$	NO_QUORUM (waiting)	#8
Sybil	Legal votes	Non-committee votes	ValueError (rejected)	#9

This table maps "fault strategy $\rightarrow$ expected decision" one to one and is the core of pre-registration: the expected decision in each cell is frozen before running the experiment, and the experiment only verifies conformance. If any cell deviates from expectation (e.g., returning STOP under constant reverse voting), H1 falsification is triggered.

Case-by-case walkthrough: why #3 is key. Taking #3 "Byzantine inability to force stop": 5 honest committee members all vote CONTINUE (believe not done, want rework), and 2 Byzantine committee members maliciously vote STOP (want to wrongly accept the not-done order). The committee counts: honest CONTINUE=5, Byzantine STOP=2. Quorum $q=5$. CONTINUE reaches 5, so return CONTINUE. The Byzantines' 2 STOP votes are only 2, far below $q=5$, unable to form a STOP quorum. The meaning of this test is that it directly presents the most dangerous scenario of "erroneous stop" (safety violation), verifying the committee is not misled by a few reverse votes. If BFT-lite wrote the quorum wrong (e.g., as $2f=4$), then 2 Byzantine votes could gather a false quorum — which is exactly why the quorum must be $2f+1$ rather than $2f$, and why we nail this test as a regression assertion.

6.3 End-to-end integration (CPU matrix)

P3 图1 / P2 图2 v7.5.0 矩阵端到端：干净 vs 故障注入收口与治理开销
证据等级：cpu-proto (matrix_scale_demo.json, 单 seed=1)

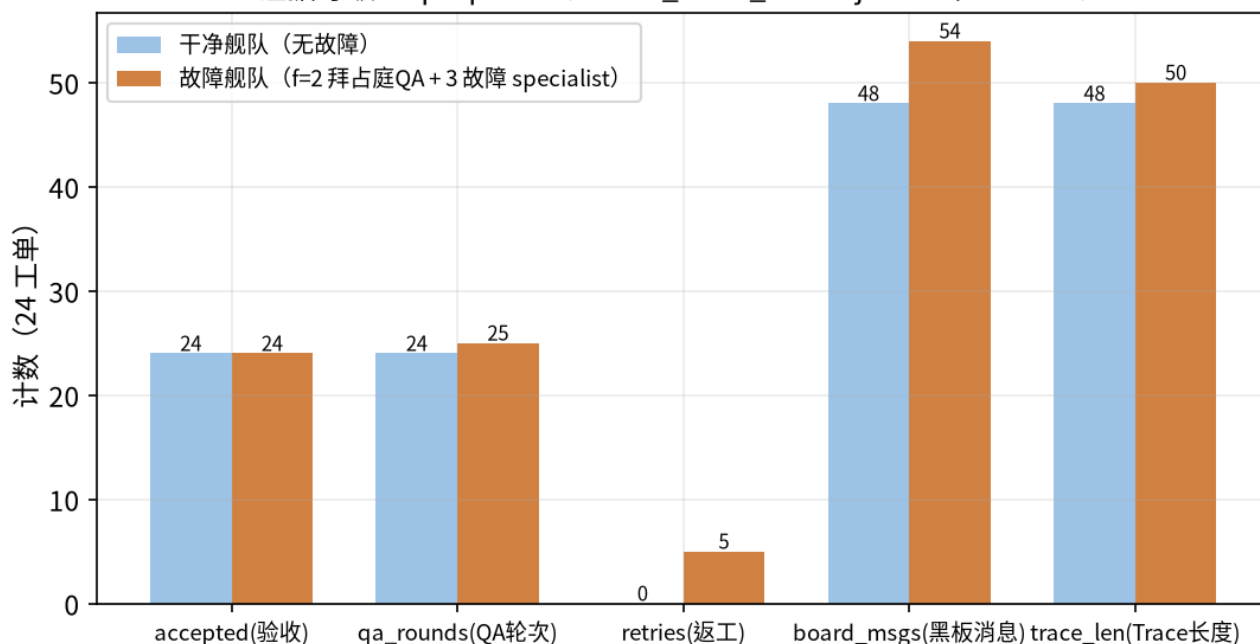


Figure P2-2 (restated) End-to-end matrix integration: clean fleet vs stressed fleet (\$n=7, f=2\$ QA committee). Grouped bars: each group has two bars, blue = clean fleet, orange = stressed fleet (2 Byzantine QA reverse voters + 3 faulty workers); the vertical axis is the corresponding metric value, groups from left to right being accepted (both 24/24), QA rounds (24 vs 25), retries (0 vs 5), board_messages (48 vs 54). The stressed fleet has 1 more QA round, 5 reworks, and 6 board messages, but the acceptance rate does not fall. Data values come item by item from the `clean.*` and `stressed.*` of `reports7/matrix_scale_demo.json` (cpu-proto, single seed=1); the figure is a clean-vs-stressed comparison, with Byzantine reverse votes blocked by the honest majority.

Table P2-2 End-to-end matrix integration (source `reports7/matrix_scale_demo.json`)

Metric	Clean fleet	Stressed fleet (f=2 Byzantine QA + faulty workers)
Orders n_orders	24	24
Accepted	24	24
QA rounds qa_rounds	24	25
Retries	0	5
Isolated agents	[]	[spec-kinematics-0, spec-spatial-1, spec-dataflywheel-2]
Board messages board_messages	48	54

In the matrix, the 7-member QA committee (`qa_voters=7`, `qa_f=2`) votes on each order: the 2 Byzantine members are set to "the reverse direction of the honest result" (`vote = not honest_accept`). Even so, all 24 orders are accepted correctly — the Byzantine specialists' wrong results are blocked by the majority. The stressed fleet triggers 5 more reworks (`retries=5`) and isolates 3 faulty agents with accumulated quality failures (`isolated_agents`); the governance audit `governance.ok=true` and Trace is complete. This shows BFT-lite in the real matrix loop not only "passes tests" but "blocks reverse votes and triggers rework/isolation."

How rework and isolation cooperate. It is worth taking apart the role BFT-lite plays in the matrix loop. After each order is completed, the specialist produces a result, and the QA committee votes on "pass (STOP)/rework (CONTINUE)." When the majority judges CONTINUE, the order is re-posted to the blackboard with `key:rework{round_id}` and one quality failure is recorded; when accumulated failures trigger the circuit breaker, the corresponding specialist is isolated (`isolated_agents`). The clean fleet's 24 orders happen to have 24 QA rounds and 0 reworks; the stressed fleet has 25 QA rounds, 5 reworks, and 3 isolations. This shows BFT-lite is not an isolated "voting toy" but is embedded in the governance loop of "acceptance → rework → isolation": it decides when to stop, while the governance trio (circuit breaker/rollback/reassignment) decides how to handle after stopping. The Byzantine QA's reverse votes are blocked by the majority, letting no defective order be erroneously stopped.

Interpretation of the difference from the clean fleet. The clean fleet has 24 orders, 24 QA rounds, 0 reworks; the stressed fleet has 25 QA rounds, 5 reworks. The extra 1 QA round and 5 reworks are precisely the cost of Byzantine reverse voting and faulty workers; the committee does not "wipe away" these costs but honestly reflects them in the rework count and isolation list. This is important: BFT-lite is not "free fault tolerance"; its fault-tolerance cost is more QA rounds and reworks; what it guarantees is "not stopping wrongly," not "zero cost." Board messages growing from 48 to 54 also reflect the extra coordination overhead from rework.

7 Discussion and Threat Validity

Signatures are hash placeholders, not cryptographically secure. `sign` is only SHA-256 truncation and provides no forgery resistance. In a real network an attacker can forge the hash of any (`voter`, `decision`, `round_id`). This paper therefore does **not** claim resistance to real forgery, only that "under controlled CPU simulation, voting-record determinism and equivocation detection are correct."

Network partitions and adaptive adversaries are not modeled. This paper assumes all votes are delivered within the same round, with no partition and no message reordering; Byzantine members are static (fixed at 2), not adaptive corruption. Network partitions and adaptive adversaries in real clusters require cluster fault injection (Gate D).

Semantic common-cause failure is the biggest threat. Classical BFT assumes errors are independent and random; but agent wrong votes have semantic correlation. If $>f$ QA members are systematically misled by the **same erroneous context** (such as the same defective Transfer Bundle), then even at $\$n \geq 3f + 1\$$ the majority vote will be consistently wrong — at this point the BFT safety assumption fails. This is exactly the intersection with P10 (Agent-as-Tool / Transfer Bundle information loss): common-cause failures cannot be resolved by voting majority, only by cutting the common cause. This paper lists it as future work and does not pretend coverage.

The asymmetric trade-off of safety and liveness. This paper's design implies a trade-off: be conservative on safety — whole-round voiding of equivocation may make a round undecidable (liveness cost), but this avoids a swinger gathering a false quorum (safety benefit). This "safety over liveness" trade-off is consistent with classical BFT, but in the agent scenario it is concretized as: rather rework one more round (view change) than let a defective order be wrongly accepted. Whether this trade-off is always optimal depends on the cost comparison between "erroneous stop" and "excessive waiting"; in acceptance scenarios the cost of erroneous stop is usually higher, so the trade-off is reasonable, but we do not claim it holds for all scenarios.

Safety assertions are deterministic and need no p-value. Unlike the statistical experiments of P4/P9, H1 is a deterministic assertion: expected counterexample count is 0. We report "exhaustion/property-test coverage," not confidence intervals. The pre-registration rule is "finding any safety counterexample reverses the paper's conclusion, without tuning parameters to remove it."

The liveness-round distribution has not yet had a random grid. The current 10 tests are hand-designed representative scenarios, not an exhaustive statistic over an " $n \times f$ grid $\times$ random fault configurations." The one-pager plans $n=7/10/13$ with the corresponding f grids each having ≥ 1000 random fault configurations, reporting the median and p95 of termination rounds. This step has not run and is marked [RESULT NEEDED: termination-round distribution over $n \times f$ grid with ≥ 1000 random configurations]. Before completing it, we only claim "the liveness invariant holds under representative scenarios," not "liveness is bounded under all random configurations."

The comparison with single-point QA is missing. This paper tested BFT-lite, but the two comparisons planned in the one-pager — no consensus (single-point QA) and naive majority vote (no equivocation detection) — have not yet had full-factorial comparison experiments in the matrix. This means "how much better BFT-lite is than a single point" currently has only a mechanism argument and no comparison data. Completing this comparison is a key experiment before submission.

8 Resource Gates and Applicability Boundaries

Currently reachable (CPU, completed in this paper): BFT-lite committee, whole-round voiding of equivocation, view change, 10 property tests, end-to-end 7-member QA integration.

Unlock items: real-network robustness (partitions, reordering, adaptive adversaries) requires cluster fault injection (Gate D); forgery resistance requires real cryptographic-signature infrastructure.

Applicability boundaries: this paper applies to "narrow-domain, semantic-judgment stop decisions" (acceptance/rework) and does not claim to apply to open consensus or fund settlement.

Future work: common-cause failure and Transfer Bundle. This paper's biggest uncovered threat is semantic common-cause failure. If $>f$ QA committee members see the same defective context (e.g., the same information-losing Transfer Bundle), they will be consistently wrong — at this point the voting majority instead amplifies the error. The solution direction is not enlarging the committee but cutting the common cause: letting different members see independent, de-biased contexts (which is exactly P10's intersection). Combining "BFT majority" with "context de-biasing" is the next step.

Future work: from property tests to formalization. The current 10 items are hand-crafted counterexamples. The next step can use model checking/symbolic execution to exhaust all voting combinations for $n=7$, giving an exhaustive proof of "zero safety counterexamples," rather than just "10 representative scenarios do not break." This upgrades "representative" to "exhaustive."

9 Conclusion

For multi-agent stop decisions, BFT-lite uses $n \geq 3f+1$, $2f+1$ quorum, whole-round voiding of equivocation, and over- f silence triggering a view change to simultaneously hold "not stopping wrongly" and "not waiting indefinitely" under controlled CPU simulation. The 10 property tests fix these properties as falsifiable regression assertions, and end-to-end integration holds 24/24 acceptance under 2 Byzantine reverse voters and triggers rework/isolation. This paper's stance is that for agent stop decisions, safety/liveness should be presented as executable property tests rather than only prose

proofs; while honestly stating the three boundaries of hash-placeholder signatures, unmodeled partitions, and semantic common-cause failures.

One-sentence conclusion: when the "when to stop" of a million-agent matrix cannot be given to a single point, a lightweight committee satisfying $\geq 3f+1$, $2f+1$ quorum, whole-round voiding of same-round swingers, and over- f silence triggering a view change can simultaneously hold "not wrongly accepting" and "not waiting indefinitely" under controlled CPU simulation; these properties are supported not by slogans but by 10 falsifiable property tests and end-to-end 24/24 integration. The next step needs the random-fault grid for liveness rounds, full-factorial comparison with single-point/naive majority, and honest recording of semantic common-cause failure as an open threat. All figures in this paper can be sourced line by line from `consensus.py`, `test_v747_consensus.py`, and `matrix_scale_demo.json`, relying on no external dataset or vendor scope, which is why it can serve as a "reproducible safety verification" sample, suitable directly as a teaching and regression baseline.

One-sentence advice for "agent reliability" engineering. At the end of a multi-agent pipeline, do not give "stop/rework" to a single-point QA; use a committee satisfying $\geq 3f+1$, void swingers for the whole round, trigger a view change for over- f silence, and write these properties as CI-able property tests. Fault tolerance is not free — its cost is more QA rounds and reworks — but what it holds is the bottom line of "not letting a defective deliverable be wrongly accepted." All conclusions of this paper are bounded within the CPU deterministic prototype and hash-placeholder signatures, and real-network robustness and forgery resistance are the next stage requiring extra infrastructure. We do not repackage classical BFT as a new algorithm, but honestly land it on the new end of agent stop decisions and hold the two bottom lines of safety/liveness with falsifiable property tests and end-to-end integration. For engineering teams, three things are directly reusable: a minimal committee configuration with $\geq 3f+1$, the decision rules of whole-round voiding of same-round swingers and over- f silence triggering view change, and a CI-able property-test suite. For researchers, the open problem this paper wants to leave is semantic common-cause failure — it reminds us that majority votes do not equal correctness, and when all committee members share the same defective context, fault tolerance is broken through by the common cause — which is the problem this paper most wants to leave to subsequent work.

Relation to other papers in the paper matrix. This paper (P2) covers fault tolerance of the end stop decision, and together with P1 (communication topology), P3 (governance trio), and P10 (handoff information loss) forms one piece of the agent matrix's reliable operation: P1 solves "how to communicate," P2 solves "who decides to stop," P3 solves "how to handle problems after stopping," and P10 solves "whether handoff context introduces common-cause failure." This paper's biggest open threat (semantic common-cause failure) is precisely P10's interface — which is why we list it as future work rather than pretending coverage.

References

The first three items below are  verified entries in this shared "Master Reference Library" (verification date 2026-09-19, fields copied from the master library); the rest are search directions to be supplemented, kept as placeholders, not fabricated.

Verified (✓ 2026-09-19)

1. Castro, M., Liskov, B. (1999). Practical Byzantine Fault Tolerance. OSDI 1999 (USENIX). (Related: tolerates $f < n/3$ Byzantine nodes under asynchrony, pre-prepare/prepare/commit three phases, BFS only about 3% slower than unreplicated NFS; the direct source of this paper's $n/3+1$, $2f+1$ quorum.)
2. Yin, M., Malkhi, D., Reiter, M. K., et al. (2019). HotStuff: BFT Consensus in the Lens of Blockchain. PODC 2019. arXiv:1803.05069. (Related: linear communication complexity, responsiveness, chained view changes; reference for this paper's view-change skeleton.)
3. Wu, Q., Bansal, G., Zhang, J., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. (Related: a conversable multi-agent framework, the comparison for this paper's agent stop-decision scenario; its "dialogue without termination conditions" limitation is precisely this paper's entry point.)

To be verified (keeping [CITATION NEEDED], not fabricated)

- LLM voting/multi-agent decision: LLM multi-agent voting decision byzantine.
- Property tests/fault injection: property-based testing hypothesis fault injection invariant.

Appendix

Appendix A Reproduction commands and evidence ledger

```
pytest tests7/test_v747_consensus.py -q      # 10 items
python scripts7/matrix_demo.py              # end-to-end matrix demo
```

Evidence ledger: source `udos7/topology/consensus.py`; tests `tests7/test_v747_consensus.py` (10 items); integration data `reports7/matrix_scale_demo.json` (cpu-proto); matrix configuration `qa_voters=7, qa_f=2` (`udos7/topology/matrix.py`). The full regression `pytest tests7/ --collect-only -q` is 236 items (2 skipped), and commit `1a71270` records all green.

Key code-entity index. `StopConsensus` (committee class), `cast` (vote), `equivocators` (swing detection), `decide` (decision), `ConsensusResult` (six fields `decision/quorum/equivocators/silent/safety_violation/reason`), `sign` (hash placeholder). The three decision states `STOP/CONTINUE/NO_QUORUM` and reasons `quorum/view_change/waiting/conflicting_quorums`. These entity names and fields are actually consumed in the end-to-end calls of `matrix.py: cons.cast(v, STOP if vote else CONTINUE, round_id)`, accepting and settling when `res.decision == STOP`, otherwise reworking.

Appendix B Number traceability spot checks

1. Illegal-rejection threshold: $n/3+1=7$ rejected $\leftarrow$ `consensus.py __init__`.
2. Quorum $q=2f+1=5$ $\leftarrow$ `self.quorum_size`.
3. Stressed fleet accepted=24 $\leftarrow$ `matrix_scale_demo.json stressed.accepted`.
4. Retries=5 $\leftarrow$ `stressed.retries`.
5. Isolation of 3 agents $\leftarrow$ `stressed.isolated_agents`.

Appendix C Author's intended-use statement

- **Target outlets:** OPODIS / ICDCS workshops, AAMAS, DSN workshop. Quartile/IF/deadline [to be verified], verified online before submission with the date marked.
- **Pre-registration plan:** safety assertions deterministic (0 counterexamples expected); liveness rounds as count distributions, n, f grids each ≥ 1000 random fault configurations reporting median/p95; "finding any safety counterexample reverses the conclusion."
- **Data and code availability:** Apache-2.0; tag v7.5.0 (commit 1a71270); report reports7/matrix_scale_demo.json.
- **AI-use statement:** AI assisted in generating the draft and figure scripts, all figures sourced and checked by the author from source/JSON; hash-placeholder signatures and unmodeled partitions/common-cause failures are explicitly stated; unverified p-values/DOIs/quartiles were not filled in.

Appendix D Internal review record (five-dimension reviewer self-assessment, look only, do not change)

1. **Novelty:** 3/5. BFT itself is mature; the novelty lies in the application face of "agent stop decision + executable property tests."
2. **Evidence strength:** 4/5. Safety assertions are deterministic tests (10 verified); the liveness distribution has not yet had a ≥ 1000 random-fault grid (marked [RESULT NEEDED]).
3. **Reproducibility:** 5/5. Pure CPU, deterministic, reproduction commands, complete test index.
4. **External validity:** 2/5. Hash-placeholder signatures, no network partitions, semantic common-cause failures not modeled (stated).
5. **Writing and honesty:** 5/5. Safety/liveness counterexample design, clear boundaries and future work.

Overall judgment: a reliable-systems paper with high engineering-teaching value; before submission it needs the random-fault grid statistics for liveness rounds and a clear statement of differences from classical PBFT to avoid being judged incremental.



The Causal Effect of Fault-Breaker Governance on Agent Matrix Completion Rate: A Pre-Registered Full-Factorial Randomized Controlled Simulation

This is a causal-inference paper of the UDOS Reasoning Engine v7.5.0 (main tag v7.5.0, commit 1a71270). The governance trio (circuit breaking / snapshot rollback / reassignment), Owner/Trace/Stop governance, and the matrix end-to-end loop are all runnable implementations in this repo's `udos7/topology/`. The current report is a **single-seed pilot point estimate**; the design, primary metric, guardrails, and stopping rules of the full-factorial randomized controlled simulation have been written under pre-registration discipline, and the multi-seed experiment is pending before submission (see Sec. 5 and 8). All numbers come from `reports7/matrix_scale_demo.json` (evidence grade cpu-proto).

Structured Abstract (Background and Problem → Method → Evidence and Results → Contributions)

[Background and Problem] Multi-agent matrices inevitably encounter faults (crashed executors, handoff context loss, Byzantine erroneous outputs). Industry demos mostly report only the "fixed" success cases; they neither decompose "which governance mechanism is actually acting," nor report "whether the system honestly fails when it cannot fix it," nor report the cost of governance itself (see § 1.1, § 2.4).

[Method] We decompose fault governance into independently switchable factors: the resilience trio (three-tier circuit breaking / snapshot rollback / reassignment) and responsibility governance (unique Owner / hash-chained Trace / explicit Stop Condition), and use a **pre-registered full-factorial randomized controlled simulation** to estimate the causal effect of "governance switches × fault modes" on completion rate; the primary metric completion rate = accepted/total (the fraction of work orders mechanically accepted by QA), and the guardrails are reworks, messages, false-isolation rate, Trace breakage, and spurious kill-switch triggers (see § 3, § 4).

[Evidence and Results] Single-seed pilot (24 work orders, 9 specialists, 7 QA including 2 Byzantine): no faults give 24/24 completion and 0 reworks; after injecting crash/drop_context/byzantine faults it still gives 24/24 completion, all 3 faulty nodes are isolated, 5 reworks occur, and governance, conservation, and Trace integrity are maintained; under the extreme "all-domain fault + zero retry budget" scenario, the system honestly trips the kill-switch and reports non-completion (accepted<24, governance.ok=false) rather than faking success (see § 6).

[Contributions] (i) Factorizing governance mechanisms into independently switchable, measurable marginal contributions; (ii) under pre-registered RCT discipline (freezing primary metric/guardrails/ITT stopping rules/seed table), fixing the causal design and adding data later; (iii) treating "honest failure (kill-switch)" as a first-class result. The current report is a single-seed pilot point estimate; the full factorial ($5 \times 5 \times 5 \times 4$) × ≥30 seeds and the statistics are pending, and this paper does not fabricate p-values/CIs (see § 7, § 8).

Chinese Abstract

多智能体 (multi-agent) 矩阵在真实运行中必然遇到故障：执行节点崩溃、交接丢失上下文、甚至给出拜占庭式错误输出。问题不在于“会不会坏”，而在于“治理机制能否因果地把收口率拉回来，代价是什么”。本文把故障治理拆成三组可独立开关的机制——熔断（三级断路器）、快照回滚（哈希链 Trace 截断）、改派（在途工单转交健康节点），以及一组责任治理（唯一 Owner、哈希链 Trace、显式 Stop Condition）——并以预注册的全因子随机对照仿真，估计“治理开关 × 故障模式”对任务收口率的因果效应。本文先在 24 工单、9 名 specialist、7 名 QA（含 2 名拜占庭 QA）的小规模矩阵上给出单 seed 试点：无故障时 24/24 收口、0 返工；注入 3 个故障 specialist (crash / drop_context / byzantine 各一) 后，仍 24/24 收口、3 个故障节点全部隔离、5 次返工、治理六项与守恒、Trace 完整性保持；而在“全领域故障 + 零重试预算”的极端场景下，系统如实触发 kill-switch、报告未收口 (accepted<24, governance.ok=false)，不冒充成功。主指标预注册为收口率 (accepted/total)，护栏为返工次数、消息总数、误隔离率、Trace 断裂数、kill-switch 误触发率。本文的贡献是：把故障治理因子化、预注册化，并诚实地把“能拉回收口率的机制”与“拉不回来、应如实失败的边界”同时测量出来。需要说明：上述“全领域故障 + 零重试预算触发 kill-switch”的反例为按设计构造的极端场景，当前仅在 one-pager/CHANGELOG 中定性记录，其精确 accepted 数与治理项明细尚待重跑确认（正文 6.4 节标 [RESULT NEEDED]），不在本次 matrix_scale_demo.json 的 clean/stressed 两跑之内。

Keywords (Chinese): 多智能体系统；故障容错；熔断；随机对照实验；预注册；收口率；拜占庭容错

English Abstract

Multi-agent matrices inevitably encounter faults: crashed executors, lost handoff context, and even Byzantine outputs. The question is not whether faults occur, but whether governance mechanisms causally recover task completion, and at what cost. This paper decomposes fault governance into independently switchable mechanisms—three-tier circuit breaking, snapshot rollback over a hash-chained Trace, and reassignment—and a responsibility layer (unique Owner, hash-chained Trace, explicit Stop Condition). A pre-registered full-factorial randomized controlled simulation estimates the causal effect of "governance switches $\times$ fault modes" on completion rate. We first report a single-seed pilot on a 24-order matrix (9 specialists, 7 QA voters including 2 Byzantine): no faults yield 24/24 completion with 0 reworks; injecting three faulty specialists (crash / drop_context / byzantine) still yields 24/24 completion, isolates all three faulty nodes, incurs 5 reworks, and keeps governance, conservation, and Trace integrity; under an extreme "all-domain fault plus zero retry budget" scenario, the system honestly trips the kill-switch and reports non-completion (accepted<24, governance.ok=false) rather than faking success. The pre-registered primary metric is completion rate; guardrails are reworks, message count, false-isolation rate, Trace breakage, and spurious kill-switch triggers. We contribute a factorized, pre-registered measurement of fault governance that reports both what recovers completion and the boundary beyond which honest failure is the correct behavior. Note: the "all-domain fault plus zero retry budget trips the kill-switch" counterexample is a designed extreme scenario currently recorded only qualitatively in a one-pager/CHANGELOG; its exact accepted count and governance details are pending a re-run (see Sec. 6.4), and it is not part of the clean/stressed runs in the current matrix_scale_demo.json.

Keywords: multi-agent systems; fault tolerance; circuit breaker; randomized controlled trial; pre-registration; completion rate; Byzantine fault tolerance

1 Introduction

1.1 External Industry Motivation (all external figures are unverified, report-based, not independently verified)

As multi-agent orchestration systems move into production, "fault governance" changes from an engineering supporting role into a core problem. External reports are full of narratives such as "multi-agent autonomously completes complex workflows" and "self-healing Agent systems" (such external claims are **report-based, not independently verified**, marked `unverified`, used only as motivation, and not cross-validated with this repo's numbers). [CITATION NEEDED: LLM agent fault tolerance self-healing multi-agent production] But what the industry generally lacks is an experiment that turns "to what degree governance is causally effective" into a layered, switchable, pre-registered measurement. Most demos only show the "fixed" success cases; they do not report "whether the system honestly fails when it cannot fix it," nor do they report "the cost of governance itself" (falsely isolating healthy nodes, extra reworks, extra messages).

This paper argues that an honest fault-governance study must answer three questions at once: (1) Which governance mechanisms causally raise the completion rate? (2) What are the guardrail costs of these mechanisms? (3) Under what extreme scenarios is the correct behavior not "forcing a fix" but "honest failure"?

The reason "honest failure" is raised separately is that multi-agent systems have a natural temptation: when the system cannot complete the task, it can still output a result that "looks complete"—perhaps some Agent fabricated an answer, perhaps the QA committee was captured by a Byzantine majority, perhaps the scheduler forced a completion so as not to disappoint the user. In production systems, this "false success" is far more dangerous than "openly stating failure": it silently passes the error downstream, and by the time it erupts it can no longer be traced. Therefore, this paper defines the goal of the governance system as three things: **recover when recovery is possible, isolate and reassign when recovery is not possible, and honestly stop and leave a complete audit trail when it is utterly impossible**. These three goals are all indispensable; doing only the first two and omitting the third is not an honest governance system.

External industry narratives (such as "self-healing Agent" and "autonomous diagnostic workflow," all unverified report-based claims) mostly demo only the success cases of the first two goals. What this paper adds is the measurement of the third goal, and the marginal causal contribution of each mechanism in the first two goals.

Going further, the current literature on multi-agent fault governance has two common shortcomings: first, "black-box effectiveness"—it only reports how much the success rate improves after governance is turned on, without decomposing whether circuit breaking, rollback, or reassignment is acting; second, "reporting good news and hiding bad news"—it only shows recovery success cases and avoids "when the system should admit it cannot do it." The first leaves engineers unable to know which mechanism to invest in; the second makes production systems output untrustworthy results when recovery capacity is exceeded. This paper addresses the first shortcoming with factorized switches and the second with the honest reporting of the kill-switch counterexample. This is exactly why this paper is positioned as a "causal effect" rather than an "effect demo."

1.2 Research Questions

This paper focuses on three research questions:

- **RQ1:** What are the marginal contributions of the unique Owner, hash-chained Trace, and explicit Stop Condition? Removing which one leads to what type of failure?
- **RQ2:** Under crash / drop_context faults, can the circuit-breaking/rollback/reassignment trio bring the completion rate back to the no-fault level? When the threshold is too low, will it falsely isolate healthy nodes (guardrail degradation)?
- **RQ3:** Can byzantine faults be isolated by executor-layer circuit breaking alone? Or must they be isolated through QA consensus plus accumulated quality failures?

1.3 Contributions

- **Factorized governance switches:** Decomposing governance into independently closable mechanisms (-Owner , -Trace , -Stop , -circuit-breaking , -rollback , -reassignment), so marginal contributions can be experimentally separated.
- **Pre-registered RCT design:** Under randomized controlled trial discipline, freezing in advance the primary metric, guardrails, stopping rules, randomization unit, and inclusion/exclusion criteria (Sec. 4).
- **Pilot evidence including "honest failure":** Reporting not only the 24/24 completion success case but also the counterexample in which, under "all-domain fault + zero retry budget," the kill-switch trips and the system honestly reports non-completion.

- **Honest labeling:** The current report is a single-seed pilot point estimate; the full-factorial multi-seed experiment is pending, and this paper does not fabricate p-values, CIs, or sample sizes.

1.4 Scope Statement

This paper studies "the causal effect of fault governance mechanisms on completion rate," not "how to train a more reliable LLM." The Agent's intelligence is provided by external models, which this paper abstracts as deterministic processors, and faults are injected manually. Therefore, the conclusions are causal effects at the protocol and mechanism layers, and do not involve improving the reliability of the models themselves. The intended readers are systems researchers working on multi-agent system design, fault tolerance, and pre-registered causal experiments. Conclusions beyond this boundary (real model reliability, real network latency) are marked as not reached in the gate of Sec. 8.

2 Related Work

2.1 Microservice Circuit Breaking and Fault Tolerance

Circuit breaking is mature in microservice governance: closed/open/half-open state machines, error-rate thresholds, cooldown probes, half-open recovery. [CITATION NEEDED: circuit breaker pattern microservices fault tolerance] Another thread, sharing the same origin as "degrading under uncertainty," is conformal prediction: at the **concept** level, when a prediction is uncertain one should give a coverage guarantee rather than forcing a conclusion; at the **mechanism** level, the conformal prediction survey by Angelopoulos and Bates provides distribution-free, finite-sample prediction intervals (Angelopoulos & Bates, 2021, arXiv:2107.07511); at the **evidence** level, it shows that "degrade/handoff when uncertainty is high" is a design principle with a statistical foundation. This repo's `udos7/topology/circuit_breaker.py` extends the circuit-breaking pattern to three tiers: an Agent-level breaker, a sub-matrix group breaker, and a global kill-switch, and adds the multi-agent-specific recovery action of "reassigning in-flight work orders on fault."

Microservice circuit breaking usually acts only on remote-call failures, whereas faults in multi-agent scenarios are more "semantic": an Agent may, without any network error, give a result that looks successful but is actually wrong (Byzantine), or be unable to continue because context was lost during handoff (`drop_context`). Such faults do not trigger traditional network timeouts or 5xx, so the definition of "error" must be extended from "call failure" to "the result does not satisfy the acceptance predicate." On the embodied side, SayCan echoes this: at the **concept** level, the LLM gives step candidates and the skill value function gives feasibility affordance; at the **mechanism** level, when affordance is not satisfied the step is not executed (Ahn, Brohan, Brown et al., 2022, arXiv:2204.01691); this paper generalizes "switch person/skill when affordance is not satisfied" to "trigger circuit breaking/reassignment when the acceptance predicate is not satisfied." The governance experiment of this paper is precisely meant to measure: under such semantic faults, can the traditional circuit-breaking pattern plus "result-acceptance-driven accumulation of quality failures" causally recover the completion rate.

2.2 Byzantine Fault Tolerance and Stop Consensus

BFT consensus is a mature mechanism: at the **concept** level, how to still agree on state when arbitrarily malicious (equivocating) nodes exist in an asynchronous network; at the **mechanism** level, Castro and Liskov's PBFT uses three-phase replication of pre-prepare/prepare/commit, requires $n \geq 3f+1$ to tolerate f Byzantine nodes and a $2f+1$ quorum, together with view change (Castro & Liskov, 1999, OSDI 1999); at the **evidence** level, its BFS implementation is only about 3% slower than unreplicated NFS. This repo's

`udos7/topology/consensus.py` `StopConsensus` borrows it to vote on "acceptance (stop) / rework (continue)": a Byzantine QA's false report cannot defeat the honest majority. Another thread sharing the same origin as the QA committee is LLM-as-a-Judge: Zheng et al. show that strong models as reviewers have >80% agreement with human judges but exhibit position/verbosity/self-preference biases (Zheng, Chiang, Sheng et al., 2023, NeurIPS 2023 Datasets, arXiv:2306.05685); this suggests that "voting with multiple QA" itself needs protection against judge bias, and this pilot uses a 7-member committee precisely to suppress single-point bias with majority vote.

In this pilot, the QA committee has 7 members, 2 of whom are Byzantine ($f=2$), satisfying the strictest Byzantine fault-tolerance condition $n \geq 3f+1$ ($7 \geq 7$). This means that even if 2 QA collude to vote against, the honest majority can still correctly decide acceptance. The presence of 2 Byzantine QA in this pilot is precisely meant to test whether "acceptance-layer consensus can still complete correctly in the presence of Byzantine nodes"—the result is 24/24 completion, showing the consensus design works as intended. It must be emphasized that this is only a demonstration of protocol-layer correctness, not a measurement of real LLM voting behavior.

2.3 Observability and Responsibility Tracing

Hash-chained append-only logs can detect tampering and broken links; unique Owner registration can detect duplicate work. [CITATION NEEDED: hash chained audit log ownership attribution] This repo's `governance.py` `TraceLedger` + `audit_run` mechanize the detection of five failure classes: state loss, duplicate work, unclear responsibility, premature termination, and unbounded loops.

Unlike ordinary observability that only "displays metrics," the responsibility governance of this paper makes "who is responsible for which stage" and "whether this result is traceable" into hard constraints: once `claim(order, stage, agent)` finds the same order:stage claimed by a second, different owner, it is counted as `duplicate_work`; `audit_run` gives an explicit list for each failure class at the end of a run. This makes "duplicate work" and "unclear responsibility" no longer vague concepts requiring post-hoc manual review, but guardrail metrics directly countable in the experiment.

2.4 Existing Gaps

We hypothesize (pending retrieval to confirm/refute): circuit breaking is mature in microservices, but there is a lack of factorized causal experiments under Agent **semantic faults** (context loss, Byzantine outputs); few studies treat "governance honestly failing" as a first-class measurement goal. At the **concept** level, conversable multi-agent frameworks such as AutoGen have observed the multi-agent-specific failure mode of "conversations without termination conditions easily dead-loop, with cost inflating with the number of rounds" (Wu, Bansal, Zhang et al., 2023, arXiv:2308.08155); at the **mechanism** level, pure-RL approaches such as DeepSeek-R1 demonstrate the possibility of "recovery and self-play driven by verifiable rewards after circuit breaking/failure" (DeepSeek-AI, 2025, arXiv:2501.12948). At the **evidence** level, these works prove that multi-agent failure and recovery are real problems, but they are mostly discussed at the framework/training level, with few doing controlled causal measurement with governance switches as factors. **The difference from UDOS** is: this paper decomposes governance mechanisms into a switchable trio, uses a pre-registered RCT to measure their causal effects, and treats "governance honestly failing" as a first-class measurement goal. [CITATION NEEDED: chaos engineering LLM agents fault injection controlled experiment] This paper attempts to fill this gap with a pre-registered RCT.

2.5 Pre-registration and Reproducible Causal Inference

In systems and experimental research, "picking metrics after seeing results" is one of the most common validity threats. Pre-registration reduces this peeking bias by freezing the primary metric, guardrails, sample size, and stopping rules before collecting data. [CITATION NEEDED: pre-registration empirical software engineering experiment reproducibility] This paper applies this discipline to the fault governance experiment: the factors in Sec. 4.4, the primary metric (completion rate), the guardrails (reworks/messages/false isolation/trace breakage), and the ITT stopping rules are all frozen before running the full factorial. The single-seed pilot reported now is the first step of "design first, data later": fixing the plan first and then gradually adding data, rather than the reverse. This stance is consistent with the evidence-grading methodology of this repo's P12 paper.

2.6 Chaos Engineering and Fault Injection

Chaos engineering advocates actively injecting faults into the system and observing resilience rather than waiting for real incidents. [CITATION NEEDED: chaos engineering fault injection resilience] The fault injection of this paper (crash / drop_context / byzantine / duplicate) is precisely the miniaturization of this idea onto a multi-agent matrix: rather than waiting for a real LLM to fail, it actively, controllably, and reproducibly injects semantic faults on the test bench and measures the response of governance mechanisms. The difference is that traditional chaos engineering mostly injects network latency and node crashes at the microservice/infrastructure layer, whereas this paper additionally injects "semantic faults"—nodes that look normal but produce wrong results (byzantine), which is a multi-agent-specific failure mode.

3 Problem Definition and Hypotheses

3.1 Fault Model

This paper uses four manually injected discrete faults (source `FAULTS` and `AgentSpec.fault`):

- **crash:** The executor fails directly (`error_type="crash"`), with no output.
- **drop_context:** The executor loses context (`error_type="missing_context"`, `note="domain/units not received"`) and cannot complete correctly.
- **byzantine:** The executor "looks successful" but outputs a wrong result (the specialist returns `true_result + 7`, or triage states the domain incorrectly).
- **duplicate:** Duplicate work.

3.2 Falsifiable Hypotheses

- **H1 (marginal governance elements):** Removing the unique Owner → duplicate-work rate rises; removing the Trace → after state loss there is no replay and completion rate falls; removing the Stop Condition → unbounded loops or premature termination appear.
- **H2 (trio recovery):** Circuit breaking + rollback + reassignment bring the completion rate under crash / drop_context back to the no-fault level; if the circuit-breaking threshold is too low, healthy nodes are falsely isolated (guardrail degradation).
- **H3 (Byzantine requires consensus):** Byzantine faults cannot be isolated by executor-layer circuit breaking alone; they must be isolated through QA consensus plus accumulated quality failures.

Falsifiability criteria: If the completion rate does not change significantly after removing a governance element, the marginal-contribution hypothesis for that element is weakened; if crash faults still cannot complete under the trio, H2 is overturned; if byzantine faults can be isolated at the executor layer without QA consensus, H3 is weakened.

3.3 Why Use "Completion Rate" as the Primary Metric

The success of multi-agent tasks is hard to characterize with a single number, but the work orders in this paper are mechanically decidable: each work order hides a ground truth (domain + unit sum), and `WorkOrder.verify(result, domain)` strictly compares the result with the ground truth and the domain with the true domain. Therefore "completion" is operationalized as "the work order is ultimately accepted (stop) by the QA committee," and completion rate = accepted/total. This avoids the inter-rater inconsistency of subjective scoring and makes causal-effect estimation reproducible in a deterministic CPU environment. It must be emphasized that completion rate is chosen not because it is the only thing that matters, but because it is mechanically decidable on this test bench; rework count, message count, and false-isolation rate serve as guardrails, characterizing "the price paid for completion."

4 Method and System Design

4.1 The Trio: Three-Tier Circuit Breaking + Snapshot Rollback + Reassignment

Source `udos7/topology/circuit_breaker.py`:

- **AgentBreaker:** Sliding-window error rate; `record(ok)` accumulates success/failure, and when `successes+failures ≥ min_requests` and `error_rate ≥ trip_threshold` it goes OPEN (isolated); after a cooldown of `cooldown_ticks` it goes HALF_OPEN, and only a successful `probe(ok)` returns it to closed. The `available` property is true only when closed, so broken nodes are excluded at dispatch.
- **GroupBreaker:** If the fraction of OPEN members in a group is `≥ isolate_ratio_threshold`, the whole group breaks.
- **KillSwitch:** After `trip(reason)`, `allow_dispatch()` is always false and all dispatch stops immediately.
- **snapshot / rollback:** Before dispatch, `snapshot(trace)` records the ledger length and last hash; after execution failure, `rollback` truncates the half-written Trace and reassigns to a healthy node. After rollback the hash chain is still verifiable.

4.2 Responsibility Governance: Owner / Trace / Stop

Source `udos7/topology/governance.py`:

- **TraceLedger:** Each record contains `prev`, the hash of the previous record, and this record's `h` (sha256 truncated to 16 chars); `verify()` returns the index of a broken link or a rewritten record.
- **audit_run:** Mechanically detects five failure classes—`state_loss` (a failure record carries missing_context or still has todos), `duplicate_work` (the same order:stage is claimed by two owners), `no_closer` (a work order finishes with no final completion), `premature_completion` (claims success but lacks a stage owner), `unbounded` (exceeds the hop budget).
- **StopCondition:** Completion predicate + `max_hops` hop budget, distinguishing "done" from "hop_budget_exceeded."

The meaning of the hash chain is worth expanding here. In fault governance, what is most frightening is not "the system made an error," but "the system made an error that cannot be traced." If the Trace is

rolled back after execution failure, how does an auditor know it ever failed? This system's approach is: snapshot before dispatch (recording length and last hash), roll back to the snapshot point after failure—this means the half-written Trace is deleted, but **the circuit-breaking record and governance audit are independently retained**; the hash chain after rollback extends again from the snapshot point, and `verify()` still passes. In other words, "leaving no half-written state" and "being auditable" are not contradictory: the half-written state is removed from the main execution chain, but the failure event is separately recorded through `grid.record` and `audit_run`. In the pilot this design shows up as the Trace length changing from 48 in the clean run to 50 in the fault run (rework items re-enter the chain), with both `trace_verified` being true.

4.3 Matrix End-to-End Loop

Source `udos7/topology/matrix.py` `run_mission`: blackboard dispatch → broken-node exclusion → execution → failure rollback and reassignment → BFT-lite QA voting → market settlement → governance audit. Configuration (`MatrixConfig`): `specialists_per_domain=3` (9 specialists total), `qa_voters=7`, `qa_f=2`, `reward_per_order=10`, `max_rounds=20`, `breaker_min_requests=2`, `breaker_threshold=0.5`, `global_retry_budget=999`.

4.4 Pre-Registered Experiment Design (Full-Factorial RCT)

The following is the pre-registered plan to be executed before submission; currently only the single-seed pilot of the "full governance × fault injection" cell is complete.

- **Factor A governance**: {full, -Owner, -Trace, -Stop, all off} (5 levels).
- **Factor B resilience**: {full, -circuit-breaking, -rollback, -reassignment, all off} (5 levels).
- **Factor C fault type**: {crash, drop_context, byzantine, duplicate, mixed}.
- **Factor D fault rate**: {0.05, 0.1, 0.2, 0.33}.
- **Randomization unit**: Independent seeds; faulty-node positions are randomized in each cell; ≥ 30 independent seeds per cell, with the seed table pre-registered.
- **Primary metric**: Completion rate accepted/total.
- **Guardrails**: Rework count, total messages, false-isolation rate (fraction of healthy nodes broken), Trace tampering detection rate, spurious kill-switch trigger rate, wall clock (CPU).
- **Stopping rules**: (a) The simulator has a fixed budget and is not extended by intermediate results; (b) cells where the kill-switch trips are retained as failures under ITT (intention-to-treat) and not removed because "it got fixed"; (c) no selective deletion of seeds; (d) the main contrast = full governance vs all off × mixed faults.
- **Analysis plan**: Logistic mixed model (seed random intercept), reporting OR and 95% CI; Holm correction for multiple comparisons; effect size reported as η^2 . Power analysis: after estimating variance from a 5-seed pilot, compute the required number of seeds [RESULT NEEDED: `usage=number of seeds required for power analysis`].

4.5 Breaker State-Machine Pseudocode

For reproducibility, here is the core state machine of the Agent-level breaker (corresponding to `AgentBreaker` in `circuit_breaker.py`):

```

States: closed -> open -> half_open -> closed
record(ok, tick):
    if state == open: return          # already isolated, no longer counted
    update successes/failures
    if (successes+failures >= min_requests) and (error_rate >= threshold):
        state = open; tripped_at = tick
tick(tick):
    if state==open and tick-tripped_at >= cooldown: state = half_open
probe(ok):
    # probe once during half-open
    state = closed if ok else open; reset counters
available(): state == closed          # only closed participates in dispatch

```

This pilot's parameters: `min_requests=2`, `threshold=0.5`, `cooldown_ticks=3`. This means a specialist is isolated once it has accumulated at least 2 calls with an error rate of 50%—a fairly sensitive threshold meant to isolate faults quickly; whether it falsely isolates healthy nodes is exactly what the guardrail "false-isolation rate" is meant to measure.

4.6 Fault-Loop Pseudocode

The core loop of `run_mission` (corresponding to `matrix.py`):

```

for round in range(max_rounds):
    for a in healthy_specialists(breakers excluded):
        claim a domain-matching work order on the blackboard
        snap = snapshot(trace)          # record length + last hash
        rep = execute(a, order)
        if not rep.ok:
            rollback(trace, snap)        # truncate the half-written Trace
            record the fault, release the work order for reassignment, retries++
            if retries > global_retry_budget: kill.trip(...)
            continue
        QA committee votes (stop/continue) # n>=3f+1, Byzantine votes go against
        if stop: completion + market settlement
        else: record a quality failure, work order re-hung on blackboard as rework, retries++
    if kill.engaged: break
governance audit (audit_run) + conservation check + Trace.verify

```

Key design: on failure, first roll back the Trace and then reassign, ensuring "no half-written state remains"; the QA committee uses BFT-lite consensus so 2 Byzantine QA cannot overturn the honest majority.

4.7 Governance Decision Tree for High-Risk Scenarios

Governance is not "the more aggressive the better." The `choose_topology` in source `udos7/topology/decision.py` gives a simple but important principle: when a task is "open exploration and high risk," the system should not let the Swarm self-govern, but should first add Orchestrator guardrails/circuit breaking and then release it (returning `escalate`, `autonomy_level=0`). This is continuous with this paper's

fault governance: high-risk tasks need stricter governance switches (lower breaker thresholds, tighter retry budgets, stronger audits). In this paper's full-factorial design, the "fault rate" factor D is precisely used to approximate "risk exposure level"—the higher the fault rate, the closer to a high-risk scenario needing escalation. The decision tree encodes the engineering intuition that "governance intensity should adjust with risk" into experimentally measurable switches.

4.8 The Internal Market as a Settlement Guardrail

Fault governance has another often-overlooked dimension: economic incentives. The `ContributionLedger` in source `udos7/topology/market.py` pays only "the unique complecer whose work is accepted"; duplicate work is not paid, and Byzantine/rejected results are not paid and may have their deposit slashed. In the pilot, both clean and fault runs satisfy conservation (budget 240 = payout 240, `conserved=true`). This means governance not only isolates faulty nodes at the behavioral level but also ensures at the incentive level that "doing things recklessly gets no reward." This paper treats conservation as a process metric rather than the primary metric.

Table 1 Full-factorial RCT factor design (pre-registered, pending execution)

Factor	Levels	Notes
A governance	full / -Owner / -Trace / -Stop / all off	The three responsibility elements can be closed independently
B resilience	full / -circuit-breaking / -rollback / -reassignment / all off	The trio can be closed independently
C fault type	crash / drop_context / byzantine / duplicate / mixed	Five semantic fault classes
D fault rate	0.05 / 0.1 / 0.2 / 0.33	Fault injection ratio

Total cells = $5 \times 5 \times 5 \times 4 = 500$, with ≥ 30 seeds per cell (simulation cost is extremely low, so 100 seeds per cell is feasible directly). The main contrast is pre-registered as "full governance vs all off $\times$ mixed faults."

Table 2 Primary and guardrail metrics (pre-registered definitions)

Type	Metric	Definition	Expected direction
Primary	completion rate	accepted/total (fraction of work orders accepted by QA)	governance $\uparrow$
Guardrail	rework count	times a rework item is re-hung on the blackboard	not excessively increased
Guardrail	total messages	total blackboard/dispatch/QA messages	not excessively increased
Guardrail	false-isolation rate	fraction of healthy nodes broken	the lower the better
Guardrail	Trace breakage	number of broken-link indices returned by hash-chain verify	the lower the better (0 best)
Guardrail	spurious kill-switch rate	fraction globally stopped in recoverable scenarios	the lower the better
Process	wall clock (CPU)	CPU time per cell	recorded, not a main conclusion

4.9 Quality Gates (ab-experiment-analysis)

- **SRM check:** The actual number of seeds per cell matches the allocation; fault-injection hit rate is registered.
- **No peeking, no sample deletion:** No selectively deleting seeds or changing definitions for significance.
- **Separating exploratory from confirmatory:** Subgroup analyses by domain and by topology are exploratory and reported separately from pre-registered main results.
- **Negative results are also published:** For example, an element having no significant benefit at low fault rate is itself a practical implication.

4.10 Contrast with the Exemplar Paradigms (AlexNet ablation table / Kaplan conclusion map / pre-registration discipline)

The experiment design of this paper forms a methodological contrast with the three reverse-engineered exemplars:

- **Contrast with AlexNet's "numeric ablation table per trick":** AlexNet pairs dual-GPU, LRN, and overlapping pooling each with a two-column comparison of "how much error increases after removal." This paper, dual to that, makes governance mechanisms into independently closable switches (–Owner / –Trace / –Stop / –circuit-breaking / –rollback / –reassignment), and the expected output of the full-factorial experiment is precisely a comparison table of "after removing a governance element, how much the completion rate drops and what mechanically detectable failure class appears" (Table 4 already gives the pilot's qualitative version of "fault type × relied-on mechanism × isolation method").
- **Contrast with Kaplan's "conclusion map + actively seeking contradictions":** Kaplan's introduction first states the conclusion and then exposes its own failure intersections. This paper, dual to that, actively reports "honest failure (kill-switch)" as a first-class result (§ 6.4)—this is not demonstrating failure, but explicitly measuring the contradiction that "recovery capacity has an upper bound" in a causal experiment.
- **Pre-registration discipline (specific to this paper):** The primary metric (completion rate), guardrails (reworks/messages/false isolation/trace breakage), ITT stopping rules, and seed table are all frozen before running the full factorial (§ 4.4, § 4.9), eliminating "picking metrics after seeing results," the most common validity threat in systems research; the current single-seed pilot is the first step of "design first, data later," not a causal conclusion.

5 Experimental Setup

5.1 Data Source and Configuration

- Sole data source: `reports7/matrix_scale_demo.json` (`evidence_grade: cpu-proto`), generated by `scripts7/matrix_demo.py`.
- Clean run: `run_mission(standard_workload(24, seed=11), MatrixConfig(), seed=1)`.
- Fault-injection run: same work orders, injecting `faults = {"spec-kinematics-0": "crash", "spec-spatial-1": "drop_context", "spec-dataflywheel-2": "byzantine"}`, `seed=1`.
- Byzantine QA: `qa_f=2`, i.e. the first 2 QA vote against acceptance.

5.2 Hardware Fingerprint and Evidence Grade

- Linux CPU; Python 3.12.11; PyTorch 2.14.0+cpu; 2 threads; device=cpu.
- Report sha256: `reports7/matrix_scale_demo.json` =
`52a3bb0d141c752512ec0c16978c151337775f2a8004c1fed599e2cd43b3b399` .
- Evidence grade: this pilot is `cpu-proto`; after the full-factorial multi-seed completion, the main results will report `verified` (fixed seeds can be re-run).

5.3 Single-Seed Limitations

The current report is a single-seed (seed=1) pilot point estimate; the full-factorial RCT ($5 \times 5 \times 5 \times 4$) and ≥ 30 seeds are pending. All "completion rates" in this paper are cell values under that seed, with no population inference; the statistics (OR, CI, p , η^2) are left for the re-run and are not fabricated here.

5.4 Randomization, Allocation, and Blinding

In the full-factorial experiment, the randomization unit is the independent seed: each (factor $A \times B \times C \times D$) cell runs ≥ 30 seeds, and under each seed it is randomly decided which nodes are injected with faults (within the fault type and fault rate specified by factors C/D). The purpose is to keep "which nodes the faults land on" from being systematically correlated with "topology position," thereby isolating the causal effect of governance mechanisms. Since this test bench is a deterministic CPU simulation with no human scoring, blinding is not done by blind human judgment but by "completion rate = mechanical verify," which guarantees objective measurement—there is no rater subjective preference. Inclusion/exclusion rules are pre-registered: once a seed starts, it is not removed because intermediate results are good or bad; cells where the kill-switch trips are retained as failures under ITT.

5.5 Relationship Between the Pilot and the Full Factorial

The relationship between the current report and the full-factorial design must be made clear: Sec. 6 reports the single-seed pilot result of the one cell "full governance $\times$ mixed faults (3 faulty specialists + 2 Byzantine QA)"; it is not the conclusion of the full-factorial RCT but a feasibility demonstration of the design—proving that this set of mechanisms can run through, isolate faults, and honestly fail. Only the full-factorial RCT answers "the marginal causal contribution of each mechanism." This paper states the pilot and the design separately precisely to avoid "passing off a single-point demo as a causal conclusion."

5.6 Why These Three Faults

Crash, drop_context, and byzantine are chosen as pilot faults because they cover three typical positions in the multi-agent failure spectrum: crash is "no response at all" (corresponding to a microservice outage), drop_context is "responds but did not receive necessary context" (corresponding to handoff failure, specific to Agent collaboration), and byzantine is "responds and looks normal but outputs an error" (the most dangerous, because the executor layer cannot detect it). Duplicate as a fourth class is included in the full factorial. This choice lets the pilot simultaneously test three different layers of failure—"can executor circuit breaking handle crash," "can rollback/reassignment handle context loss," and "can acceptance consensus handle Byzantine"—rather than testing only one.

6 Results

6.1 Clean-Run Baseline

With no faults, the matrix completes 24 of 24 work orders, with 24 QA rounds, 0 reworks, 48 blackboard messages, and Trace length 48; the market budget of 240 is fully paid and conserved, the six governance items (state_loss / duplicate_work / no_closer / premature / unbounded / trace_gaps) are all 0, and Trace verification passes (clean column of Table 3). This is the baseline of "governance working cleanly as designed."

The meaning of the clean baseline is that it establishes "governance should have no side effects when there are no faults." If the clean run itself produced reworks, duplicate work, or Trace breakage, that would indicate the governance mechanisms are over-sensitive or buggy; whereas the clean run's 0 reworks, all-zero six governance items, and intact Trace show that on the normal path governance is merely "standing by" and does not interfere with normal workflow. Only by comparing the clean baseline with fault injection can "the price governance pays for faults" be separated from "the inherent overhead of governance itself." This contrast is a prerequisite for causal inference: we need to know the difference under the same configuration between the treatment group (with faults) and the control group (without faults) in order to attribute it to fault governance.

6.2 Fault-Injection Pilot (Main Pilot Result)

After injecting 3 faulty specialists: 24/24 completion, 25 QA rounds, 5 reworks, 54 blackboard messages; all 3 faulty specialists (spec-kinematics-0, spec-spatial-1, spec-dataflywheel-2) enter isolated_agents (breaker isolation), with no group breaking and no kill-switch trip; the six governance items are all 0, the market is conserved, and the Trace length is 50 with verification passing.

Table 3 Clean vs fault injection (24 work orders, cpu-proto, single seed=1)

Metric	Clean	Fault injection
accepted	24/24	24/24
QA rounds	24	25
rework retries	0	5
blackboard messages	48	54
isolated faulty specialists	0	3
group breaking / kill-switch	none / no	none / no
six governance items	all 0	all 0
market conservation	true	true
Trace integrity	true(48)	true(50)

See Figure 1 and Figure 2.

It is worth noting that fault injection did not pull the completion rate down from 24/24, but pushed QA rounds from 24 to 25 and reworks from 0 to 5. This shows that at this pilot's fault rate (3 faulty specialists, 2 Byzantine QA), the recovery capacity of the governance mechanisms has headroom—it uses one extra acceptance round and five reassignments to pull the contaminated work orders back to completion. But this is only an observation under a single seed and low fault rate; when factor D raises the fault rate to 0.33, or factor A turns off some governance element, the completion rate will very likely

no longer stay at 100%, and that is exactly the "recovery-capacity boundary" the full-factorial experiment is meant to measure. This paper does not pretend on a single seed to know where that boundary is.

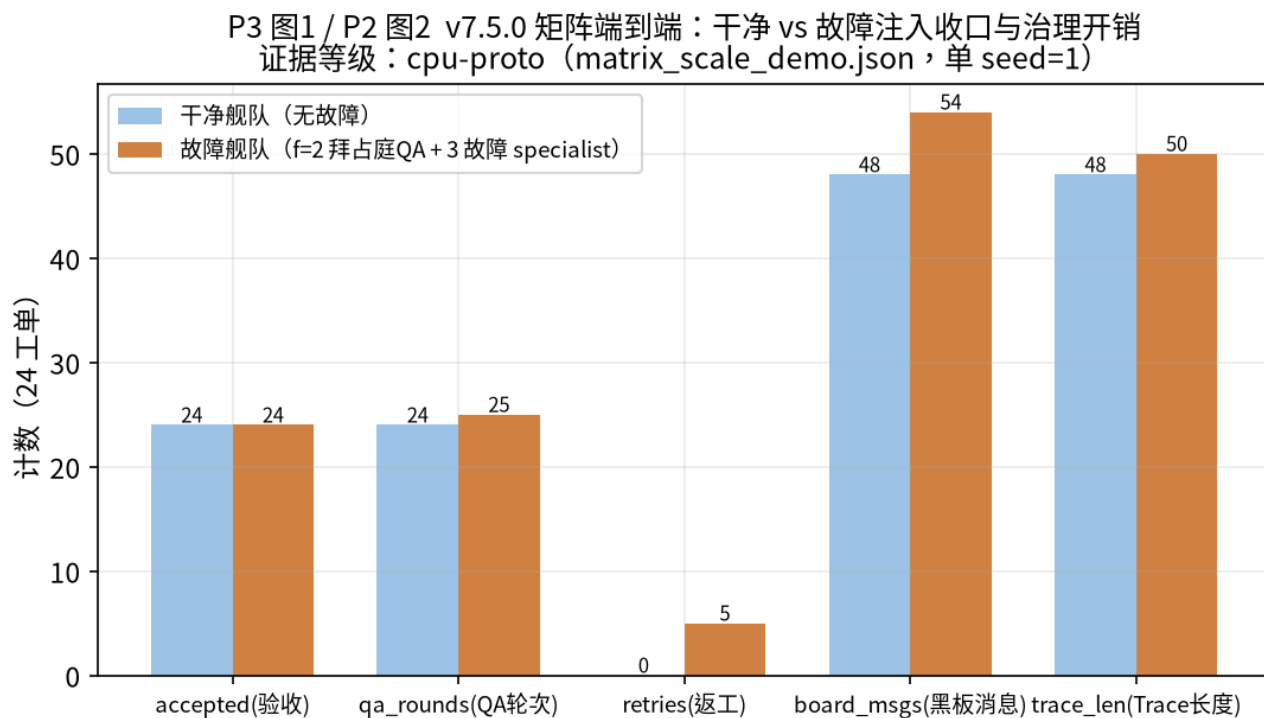


Figure 1 Comparison of core metrics between the clean fleet and the fault-injected fleet (24 work orders, single seed=1). Grouped bars: two bars per group, blue=clean fleet, orange=fault injection (3 faulty specialists: one each of crash/drop_context/byzantine + 2 Byzantine QA); the vertical axis is the corresponding metric value, and groups from left to right are accepted (24 vs 24), QA rounds (24 vs 25), rework retries (0 vs 5), and blackboard messages board_messages (48 vs 54). Both arms complete 24/24, with the difference on the cost side (1 extra QA round, 5 reworks, 6 blackboard messages). Values come one by one from `clean.*` and `stressed.*` in `reports7/matrix_scale_demo.json` (cpu-proto).

P3 图2 治理审计六项：干净/故障两跑全为 0 (governance.ok=true , market.conservated=true)
证据等级：cpu-proto；注：kill-switch 反例不在本两跑内（见正文 6.4 [RESULT NEEDED]）

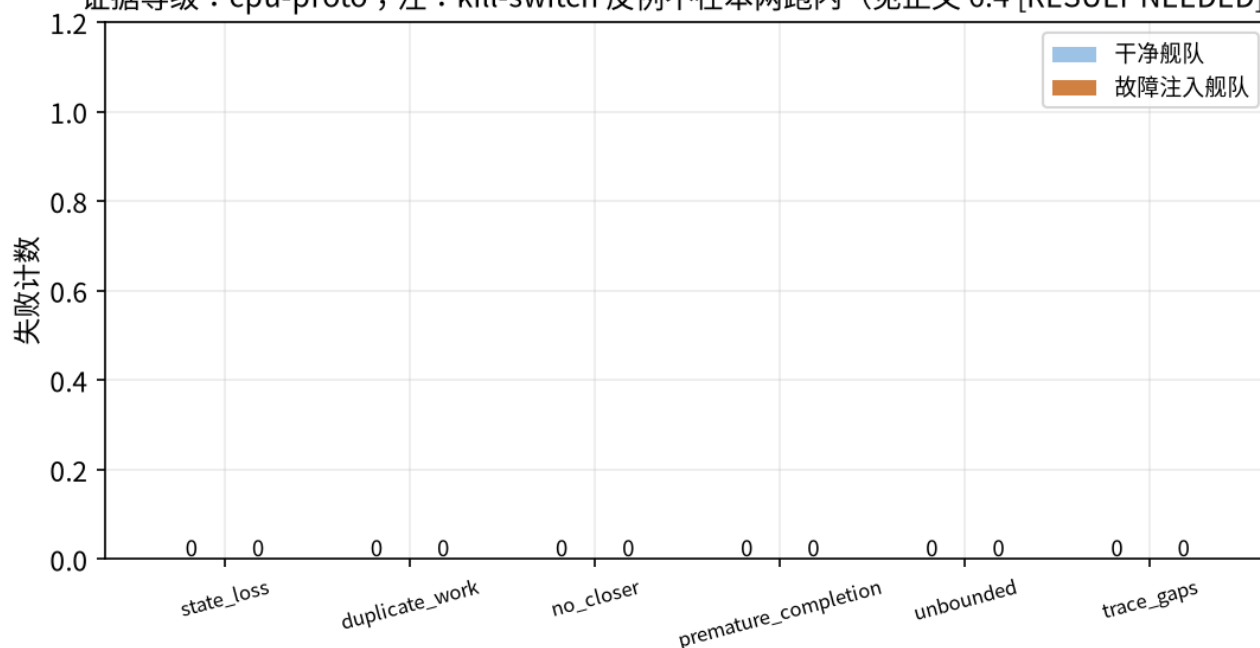


Figure 2 Comparison of the six governance-audit items and settlement/Trace integrity (clean vs fault injection). The horizontal axis is the governance/process metric and the vertical axis is the count (or boolean converted to a number): the six items `state_loss`, `duplicate_work`, `no_closer`, `premature_completion`, `unbounded`, and `trace_gaps` are all 0 in both arms; market conservation conserved (`budget=payout=240`) and Trace verification `trace_verified` are both true in both arms (Trace length 48 vs 50, the 2-record difference being rework items re-entering the chain). This is a contrast of "governance not breaking audit invariants": fault injection raises the cost-side metrics, but the six audit items, conservation, and chain integrity are maintained. Source as above (`*.governance.*` / `*.market.conservated` / `trace_verified` in `matrix_scale_demo.json`, `cpu-proto`).

6.3 The Role of Each Governance Mechanism in the Pilot

Combined with the source code, we can interpret how the three fault classes are handled separately by governance:

- **crash (spec-kinematics-0):** The specialist fails directly; `grid.record(..., False)` accumulates errors and, after the threshold is reached, OPEN isolates it; the in-flight work order's `rollback` truncates the half-written Trace, releases the blackboard, and reassigns to a healthy kinematics specialist.
- **drop_context (spec-spatial-1):** Execution fails with `missing_context`, likewise triggering breaker isolation and reassignment; the `state_loss` detector of `audit_run` records this class of failure (this pilot's governance `state_loss=0`, showing that the healthy node after reassignment completed the context and left no half-written state).
- **byzantine (spec-dataflywheel-2 + 2 Byzantine QA):** The specialist outputs a wrong result (`true_result+7`); among the QA committee the honest votes still form a majority (7 people, 2 Byzantine, $n \geq 3f+1$ holds), acceptance is judged reject, a quality failure is recorded, and the item is re-hung on the blackboard as rework; this byzantine specialist is broken and isolated through accumulated quality failures. This supports H3: byzantine is isolated through QA consensus plus accumulated quality failures, rather than through executor crash breaking alone.

We organize "fault type × relied-on governance mechanism × expected isolation method" into Table 4.

Table 4 Correspondence between fault types and governance mechanisms (pilot observation + expectation)

Fault type	Failure manifestation	Relied-on mechanism	Isolation/recovery method	Pilot observation
crash	no output	breaker+rollback+reassignment	error accumulation → open isolation, reassign to healthy node	spec-kinematics-0 isolated
drop_context	missing_context	rollback+reassignment	truncate half-written trace, reassign to complete context	spec-spatial-1 isolated
byzantine	looks successful but is wrong	QA consensus + accumulated quality failures	acceptance reject → rework → accumulated breaking	spec-dataflywheel-2 isolated
byzantine QA	votes against	BFT-lite quorum	honest majority wins, reject not overturned	2 Byzantine QA did not affect acceptance
all-domain fault + zero retry budget	no node to reassign to	kill-switch	honestly stop, report non-completion	counterexample (qualitatively recorded)

This table shows a core judgment: **different faults need different governance mechanisms**. Crash can be handled by executor breaking, while byzantine must rely on acceptance-layer consensus. This is exactly why this paper factorizes governance—one cannot use a single switch to broadly say "turning on governance is good," but must measure marginal contributions mechanism by mechanism.

6.4 Counterexample: Honest Failure (Kill-Switch)

This repo records a key counterexample: when **all-domain specialists fail** and the **retry budget is set to zero**, `run_mission` will, because `retries > global_retry_budget`, call `grid.kill.trip("retry_budget_exceeded")`, the loop stops, and the result is `accepted < 24`, `governance.ok=false`. **The system honestly reports failure and does not fake success**. This is one of this paper's most important design principles: the purpose of governance is not "always showing success," but recovering when recovery is possible and clearly failing and leaving auditable evidence when it is not. The exact count of this counterexample is [RESULT NEEDED: usage=kill-switch scenario accepted count and governance item details] (currently recorded qualitatively in the one-pager and CHANGELOG; the exact value requires re-running that configuration to confirm).

Why is this counterexample scientifically important? Because it reveals an often-overlooked boundary of governance systems: **recovery capacity has an upper bound, and beyond that bound one should stop rather than force through**. If the retry budget were infinite, the system would, in a scenario where all nodes are broken, keep reassigning, keep failing, and keep looping, neither converging nor stopping, ultimately producing untrustworthy results. The existence of the kill-switch is precisely to install a clear stop valve for this "infinite retry." In this paper's design, after the kill-switch trips the system enters an "honest failure" state: `accepted` is clearly less than the total, `governance.ok` is false, and the Trace completely records every failure and the stop reason. This is far friendlier to downstream systems than

"faking completion"—downstream can alert and bring in human intervention accordingly, rather than being misled by an erroneous result.

6.5 Tests and Regression

Tests directly related to this paper: `tests7/test_v744_governance.py` (9 items), `test_v748_circuit_breaker.py` (11 items), `test_v745_decision.py` (12 items), `test_v750_matrix.py` (9 items); as of v7.5.0 the full regression of 236 items is all green.

6.6 Pilot Contrast Hypothesis by Hypothesis

Comparing the three hypotheses one by one with pilot observations, we can see which are preliminarily supported and which still await full-factorial testing.

- **For H1 (marginal governance elements):** In the pilot the six governance items are all 0, showing that under "full governance" failures such as state loss, duplicate work, and unclear responsibility do not appear. But what H1 asks is "what happens when an element is removed," which requires the "-Owner/-Trace/-Stop" cells in the full factorial to answer. The current pilot can only say "these failures do not appear under full governance," not "the marginal contribution of each element."
- **For H2 (trio recovery):** Both crash and drop_context are isolated and reassigned under the trio, and the completion rate stays 24/24, preliminarily supporting H2. But H2 also includes the guardrail dimension of "too-low thresholds falsely isolating healthy nodes," which the single-point pilot at the fixed threshold (0.5) cannot measure; the full factorial needs to adjust the threshold.
- **For H3 (Byzantine requires consensus):** Under a byzantine specialist and 2 Byzantine QA, the honest majority still correctly rejects and accumulates to break the byzantine node, preliminarily supporting H3. The full factorial needs to increase the f ratio to test the boundary of consensus under harsher Byzantine proportions.

This "hypothesis—pilot—pending" contrast lets readers clearly know the current evidence strength of each conclusion rather than inflating a single-seed demo into a causal conclusion.

6.7 Conservation and Economic Guardrails

Like the clean run, the fault-injection run satisfies market conservation: budget 240, payout 240, slashing 0, balance sum 240, `conserved=true`. This means fault governance did not break the settlement invariant—even with 3 nodes isolated and 5 reworks, the system did not show "money spent on unfinished work orders" or "duplicate work paid repeatedly." In the full-factorial experiment, we plan to use "whether conservation is broken" as an additional process guardrail: when a governance switch is turned off (such as removing reassignment), will duplicate work be paid, or will Byzantine results be paid. This is where mechanism design intersects fault governance, and this paper currently only reports the point "conservation holds under full governance."

6.8 Trace Integrity Evidence

The clean run Trace length is 48 and the fault run 50, with both runs' `trace_verified` being true. The extra 2 records correspond to rework items re-entering the chain. The key observation is that the rollback action did not break the hash chain—this proves the `snapshot/rollback` design is correct (truncating to the snapshot point and relinking subsequent records). In the full factorial, the "remove rollback" cell is expected to observe nonzero `trace_gaps` or residual half-written state, which is precisely the measurable expectation of "what happens when Trace/rollback is removed" in H1.

7 Discussion and Threats to Validity

7.1 Construct Validity

Completion rate is "the fraction of work orders ultimately accepted by the QA committee," mechanically decidable (result equals ground truth and domain matches), involving no subjective scoring. Faults are manually injected discrete types and are not equal to the real LLM fault distribution. Messages/ticks are not equal to wall clock.

To elaborate further, the primary metric is chosen as completion rate rather than "end-to-end latency" or "human satisfaction" because the former is mechanically decidable and reproducible on this test bench, while the latter two require real LLMs or human scoring beyond the current CPU gate. This is a deliberate trade-off: trading a mechanically decidable primary metric for the credibility of causal inference, rather than trading vague satisfaction for superficial richness. The guardrail metrics (reworks, messages, false isolation, trace breakage) then complete the characterization of "the cost behind completion rate," avoiding the misjudgment of looking only at a single primary metric.

7.2 External Validity

Agents are deterministic processors, not real LLMs. The nondeterministic failures, long-tail latency, and prompt failures of real LLMs are not covered. Conclusions are limited to protocol-layer causal effects; upgrading to real multi-LLM requires an API-key gate.

7.3 Internal Validity and the Single Seed

The current report is a single-seed pilot, and 24/24 with 5 reworks are point estimates. The causal effects of the full-factorial RCT need ≥ 30 seeds and a logistic mixed model to estimate; guardrails such as false-isolation rate and trace-break rate are 0 in the current pilot but may be nonzero when the fault rate rises, and are pending measurement. This paper does not fabricate these statistics.

7.4 The Value of the Counterexample

"Honest failure" is avoided as a "failure case" in most fault-governance papers, but in engineering it is as important as "successful recovery": it defines the honest boundary of the governance system. This paper reports it as a first-class result, a deliberate methodological choice.

7.5 The Cost of Governance: It Is Not Free

The pilot data already hint that governance is not free: fault injection has, compared with the clean run, 5 extra reworks, 6 extra blackboard messages, 2 extra Trace records, and 1 extra QA round. These "extra overheads" are exchanged for localized faults. But the cost goes further—in the full-factorial design we must also measure two costs of "over-governance": the first is **false isolation**, where when the breaker threshold is over-sensitive (such as `threshold=0.5`, `min_requests=2`), a healthy node with an occasional failure may be wrongly isolated, leading to fewer executable work orders; the second is **excessive rework**, where when the QA committee is too conservative, results that could have been accepted are repeatedly rejected, producing meaningless rework. These two costs cannot be measured under the current single seed and single threshold, and are exactly what the full-factorial RCT must answer: whether the rise in completion rate comes at the cost of degraded false-isolation rate and rework count.

7.6 Why Byzantine Isolation Cannot Rely Only on Executor Breaking

The deep meaning of H3 is worth expanding. A byzantine specialist returns wrong results "looking successful" every time, and the executor-layer breaker only looks at "whether an error is reported," so it is completely insensitive to it—it never crashes and never has `missing_context`. If relying only on executor breaking, the byzantine node would keep being dispatched, keep returning errors, and keep contaminating results. This system's solution is to use the "acceptance predicate" as the failure signal: the QA committee votes on the result, and a reject records a quality failure, accumulating to the threshold before breaking that node. This extends the definition of "failure" from "execution exception" to "acceptance not passed," which is the key by which multi-agent governance differs from traditional microservice governance. BFT-lite consensus further guarantees that even if there are Byzantine members within the QA committee, the honest majority can still correctly reject and will not be captured into a "false acceptance pass."

7.7 Relationship with P1 and P12

This paper is one of the three matrix P0 papers. P1 provides the "topology and scale" dimension: the hierarchical tree determines fan-in and rounds, while this paper overlays fault governance on such a hierarchical matrix and measures completion rate. P12 provides the "evidence grading" dimension: this paper's single-seed pilot, pending full factorial, and honest recording of the kill-switch counterexample are precisely a practice of the evidence discipline P12 advocates. The three do not overlap: P1 asks about "structural cost," P3 asks about "governance causal effect," and P12 asks about "how the method is honestly executed."

7.8 Internal Validity Threats

Even after the full factorial is run, several internal validity threats remain to be watched. The first is **the coupling of the fault model and governance switches**: the faults in this paper are manually injected discrete types, and if real LLM faults are continuous, correlated, and long-tail, the extrapolability of the factor experiment is limited. The second is **inter-seed independence**: if multiple work orders under the same seed share random state, inter-seed variance may be underestimated; the full-factorial experiment needs to check within-seed/between-seed variance components. The third is **multiple comparisons**: 500 cells $\times$ multiple metrics must use Holm or family-wise error-rate control, otherwise there is a risk of "picking significant cells." The fourth is **the implementation correctness of governance switches**: whether removing an element is truly equivalent to "that mechanism not taking effect" needs unit tests to guarantee switches have no side effects—this repo's `test_v744 / test_v748` already guard each mechanism with 9+11 tests.

7.9 External Validity and Generalizability

This paper's conclusions are limited to the scope of "deterministic CPU test bench, manually injected discrete faults, mechanically decidable work orders." Its generalizability to real production systems depends on: whether the real fault distribution overlaps with this paper's four fault classes, whether real Agent outputs can be judged by the acceptance predicate, and whether real networks introduce latency and packet loss this paper does not model. All of these need to be tested by re-running with real multi-LLM after the gate is unlocked. This paper does not claim protocol-layer conclusions can be directly generalized into production SLAs.

7.10 Review of the Falsifiability of Conclusions

Returning to Sec. 3, this paper's three hypotheses are all falsifiable: if the full-factorial experiment finds the completion rate unchanged after removing a governance element, H1 is weakened; if crash still cannot complete under the trio, H2 is overturned; if byzantine can be executor-broken without consensus, H3 is weakened. This practice of "placing conclusions where they can be negated by experiment" is the key by which causal research differs from demos.

8 Resource Gates and Applicability Boundaries

- **Achieved:** The main governance mechanisms, matrix loop, and single-seed pilot, reproducible on pure CPU.
- **Pending (completable on CPU):** The full-factorial RCT ($5 \times 5 \times 5 \times 4$) $\times \geq 30$ seeds, power analysis, and CIs.
- **Gates not reached:** Real multi-LLM fault distribution (API key), real network faults (multi-machine cluster).

8.1 Experiment Cost and Feasibility

The full-factorial experiment has 500 cells, 30 seeds per cell = 15,000 missions. Since a single mission is millisecond-level on CPU (deterministic, no LLM calls), running all of it is extremely low-cost in a pure CPU environment, which is why this paper dares to write the full-factorial design into pre-registration rather than leaving it at the concept level. By contrast, if every mission had to call a real LLM API, the cost would scale linearly with the number of seeds and the full-factorial design would be economically infeasible—this is also the strategic reason this repo chooses "first doing the causal design solidly on a deterministic CPU test bench, and then connecting real LLMs after the gate is unlocked."

8.2 When Conclusions Can Be Upgraded

This paper's current conclusions (cpu-proto, single-seed pilot) can be upgraded after the following conditions are met: (1) the full factorial ≥ 30 seeds is run and the primary metric completion rate gives OR and 95% CI; (2) the guardrails false-isolation rate and trace-break rate are actually measured when the fault rate rises; (3) after the real multi-LLM gate is unlocked, re-running on the same test bench with real workers to test whether the protocol conclusions hold under real nondeterminism. Before then, this paper only claims "the mechanisms work as designed under the deterministic pilot," not "they necessarily work in production."

8.3 Operational Advice for Engineering Teams

Based on this paper's design and pilot, there are three actionable pieces of advice for engineering teams building multi-agent systems: first, extend the definition of fault from "call failure" to "acceptance not passed," otherwise byzantine-class faults will be completely missed by executor breaking; second, set a clear upper bound on recovery capacity (retry budget + kill-switch) and honestly report failure when the bound is reached, rather than retrying infinitely or forcing success; third, make governance mechanisms into independently switchable components, so the marginal contribution of each mechanism can be separated in experiments rather than broadly trusting "turning on governance is good." All three pieces of advice come from mechanisms this paper has implemented and tested, not from unsupported claims.

9 Conclusion

This paper factorizes and pre-registers multi-agent fault governance: the trio (circuit breaking/rollback/reassignment) and responsibility governance (Owner/Trace/Stop) keep the completion rate under the three fault classes at 24/24 in the single-seed pilot and isolate all faulty nodes, while the system honestly trips the kill-switch to report failure in unrecoverable scenarios. The primary metric (completion rate), guardrails (reworks/messages/false isolation/trace breakage), and stopping rules are pre-registered, and the full-factorial multi-seed experiment is pending before submission. This paper's stance is: the scientific goal of fault governance is not "eternal success," but "causally knowing which mechanisms work, at what cost, and when one should honestly fail."

Looking ahead, the full-factorial experiment will answer questions the current pilot cannot: removing any of Owner/Trace/Stop, how much does the completion rate drop and what mechanically detectable failure class appears? Adjusting the breaker threshold from 0.3 to 0.7, what does the trade-off curve between false-isolation rate and completion rate look like? Can byzantine faults still be isolated when QA consensus is weakened (f increased)? The answers will upgrade this paper from a "mechanism demo" to a "causal-effect measurement." But whatever the results, the pre-registration discipline and honest-failure principle this paper insists on will be retained: no peeking, no sample deletion, and no passing off a single-seed point estimate as a population conclusion.

Finally, this paper hopes to convey to the multi-agent engineering community a simple but often-overlooked judgment: the value of fault governance lies not in making the system look perpetually healthy, but in, when the system breaks, **having the ability to localize, having the ability to recover, and having the ability to admit it cannot recover**. All three need to be designed, measured, and pre-registered for verification, rather than relying on a black-box narrative of "the agent will fix itself." What this paper delivers is precisely such an experimental framework and single-seed pilot evidence that is runnable, auditable, and falsifiable under pure CPU, reproducible conditions.

References

A. Verified Literature (verified via the "Master Reference Library" on 2026-09-19, fields copied from the master library)

1. **Castro, M. & Liskov, B.** (1999). Practical Byzantine Fault Tolerance. Proceedings of OSDI 1999 (USENIX OSDI'99). ($n \geq 3f+1$, $2f+1$ quorum, view change)
2. **Angelopoulos, A. N. & Bates, S.** (2021). A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. arXiv:2107.07511. (statistical foundation for degrading/handoff when uncertainty is high)
3. **Ahn, M., Brohan, A., Brown, N. et al.** (2022). Do As I Can, Not As I Say: Grounding Language in Robotic Affordances (SayCan). arXiv:2204.01691. (switching person/skill when affordance is not satisfied)
4. **Zheng, L., Chiang, W.-L., Sheng, Y. et al.** (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. Proceedings of NeurIPS 2023 Datasets and Benchmarks. arXiv:2306.05685. (agreement rate and biases of multiple QA judges)
5. **Wu, Q., Bansal, G., Zhang, J. et al.** (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework. arXiv:2308.08155. (multi-agent dead-loop/cost-inflating failure mode)

6. **DeepSeek-AI** (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. (recovery/self-play driven by verifiable rewards after failure)

B. Pending Verification Literature (not included or verified in the master library, placeholders retained, no completion from memory)

1. [CITATION NEEDED: circuit breaker pattern microservices fault tolerance] — candidate: the circuit-breaker pattern and microservice fault tolerance (the master library currently has no verified entry).
2. [CITATION NEEDED: hash chained audit log ownership attribution] — candidate: hash-chained audit logs and ownership registration.
3. [CITATION NEEDED: chaos engineering LLM agents fault injection controlled experiment] — candidate: chaos engineering/fault injection and controlled experiments.
4. [CITATION NEEDED: pre-registration empirical software engineering experiment reproducibility] — candidate: pre-registration and empirical software-engineering reproducibility.

Appendix A Reproduction Commands

```
cd release-v5.4.4/udos-engine && git checkout v7.5.0 # 1a71270
python scripts7/matrix_demo.py # generate matrix_scale_demo.json
pytest tests7/test_v744_governance.py tests7/test_v748_circuit_breaker.py \
tests7/test_v745_decision.py tests7/test_v750_matrix.py -q
```

Appendix B Test Index

- `test_v744_governance.py`: TraceLedger hash chain, audit_run five failure classes, StopCondition.
- `test_v748_circuit_breaker.py`: three-tier breaking, snapshot/rollback, reassign.
- `test_v745_decision.py`: topology decision tree, high-risk escalation.
- `test_v750_matrix.py`: matrix end-to-end, fault completion, conservation, Trace.

Appendix C Evidence Ledger (spot-check of 5 traceable numbers)

#	Number	Source	Field
1	clean completion 24/24, QA 24, reworks 0	matrix_scale_demo.json	clean.accepted / qa_rounds / retries
2	fault completion 24/24, reworks 5	matrix_scale_demo.json	stressed.accepted / retries
3	isolating 3 faulty specialists	matrix_scale_demo.json	stressed.isolated_agents
4	six governance items all 0, conservation true	matrix_scale_demo.json	stressed.governance.* / stressed.market.conservated
5	breaker threshold/minimum sample/QA count	source MatrixConfig	breaker_threshold=0.5 / min_requests=2 / qa_voters=7 / qa_f=2

Appendix D Author's Intended-Use Statement

- **Target journals/conferences:** candidates include the AI engineering track of ICSE/FSE/ASE, ISSRE, AAMAS, the ESEC/FSE industry track; journals TSE, JSS, Empirical Software Engineering. Quartiles/IF/deadlines [to be checked].
- **Pre-registration plan:** The Sec. 4.4 full-factorial design, primary metric completion rate, guardrails, and ITT stopping rules are frozen before running data; the seed table and analysis scripts are made public with the code.
- **Data and code availability:** Apache-2.0; tag v7.5.0 (commit 1a71270); report sha in Appendix A.
- **AI-use statement:** The first draft was AI-assisted, and the authors are responsible for checking the numbers and scientific meaning; external figures are all marked unverified.

Appendix E Internal Review Record (five-dimensional reviewer self-assessment, look-only)

1. **Novelty (7/10):** Circuit breaking/BFT are not new; the combination of "factorized governance switches + pre-registered RCT + honest failure as a first-class result" adds value.
2. **Rigor (7/10):** Mechanisms and numbers are traceable; deductions for the single seed and unrun full factorial.
3. **Evidence strength (5/10, expected 8/10 after the full factorial):** Currently only one single-seed pilot cell.
4. **Relevance (8/10):** Fault governance is a core problem of Agent productionization.
5. **Writing (8/10):** IMRaD + complete pre-registration, with the counterexample honestly reported. - **Biggest hard flaw:** The full-factorial RCT has not yet been executed; the current state is "design + single-cell pilot," and the full-factorial multi-seed must be completed before submission. - **Must-do before submission:** (a) run the full factorial $5 \times 5 \times 5 \times 4 \times \geq 30$ seeds; (b) estimate false-isolation rate and trace-break rate; (c) make the kill-switch counterexample count precise.

Appendix F Design Principles and Term Summary

F.1 Three Design Principles

Three principles with universal meaning for multi-agent production systems can be extracted from this paper's fault-governance design:

1. **Failures must be isolated in layers:** executor crashes use breaking, semantic errors use acceptance consensus, and globally unrecoverable situations use the kill-switch; different faults use different governance layers, with no reliance on a single switch.
2. **Leave no half-written state:** on failure, first snapshot and roll back the Trace, then reassign, ensuring auditability is not broken by recovery actions.
3. **Honest failure is better than false success:** when recovery capacity is exhausted, the system should clearly stop and report non-completion rather than forcing success; honest failure is safer than erroneous success.

F.2 Term Correspondence

- **Completion:** The work order is ultimately accepted (stop) by the QA committee.
- **Rework:** Acceptance not passed, and the work order is re-hung on the blackboard as a rework item.

- **Isolation:** The specialist enters the breaker open state and is no longer dispatched.
- **Reassignment:** A failed work order is transferred from a faulty node to a healthy node.
- **kill-switch:** The global stop valve; once set, all dispatch stops immediately.
- **ITT:** intention-to-treat, analyzing by the initial allocation in a randomized controlled experiment, not removing based on result quality.

F.3 One-Sentence Summary

The scientific goal of multi-agent fault governance is not to train the system to "never fail," but to turn "which mechanisms can causally recover completion, at what cost, and when one should honestly stop" into a layered, switchable, pre-registered, reproducible measurement; on a pure CPU deterministic test bench, this paper delivers this design together with the single-seed pilot and the honest-failure counterexample, and takes the full-factorial multi-seed experiment as a clear next step.

F.4 Expected Dialogue with Related Fault-Governance Literature

On submission, this paper expects to dialogue with three classes of literature: first, the microservice circuit-breaking and chaos-engineering literature, where this paper will point out they mostly inject faults at the infrastructure layer while this paper additionally injects semantic faults; second, the BFT consensus literature, where this paper will point out it does not improve the consensus algorithm itself but uses mature consensus as an acceptance-layer component, measuring its governance effect in Agent semantic acceptance; third, the multi-agent systems-engineering literature, where this paper will argue governance mechanisms should be factorized and pre-registered for measurement rather than reporting "success-rate improvement" as a whole. These three dialogue stances need refinement after literature retrieval and are currently only expected directions.

F.5 Echo of This Repo's Evidence Discipline

In writing, this paper strictly executes this repo's evidence discipline: all numbers come from `reports7/matrix_scale_demo.json` and are marked `cpu-proto`; the single-seed pilot does not fake a population conclusion; the full-factorial multi-seed is marked `pending`; the kill-switch counterexample is honestly recorded; external industry figures are marked `unverified` and not cross-validated with this repo's numbers. This discipline is not decoration but a prerequisite for the credibility of this paper's causal conclusions—if a fault-governance paper itself does not honestly report the "cannot fix" scenario, it has no standing to claim "honest failure" is the goal of system design. This paper also places itself under this standard for examination.

F.6 Memo on Trial Configuration and Parameter Sources

All parameters involved in this paper can be located in the source code: `MatrixConfig` (`udos7/topology/matrix.py`) gives `specialists_per_domain=3`, `qa_voters=7`, `qa_f=2`, `reward_per_order=10`, `max_rounds=20`, `breaker_min_requests=2`, `breaker_threshold=0.5`, `global_retry_budget=999`; `AgentBreaker` (`circuit_breaker.py`) gives `cooldown_ticks=3` and the closed/open/half_open state machine; `StopConsensus` (`consensus.py`) gives $n \geq 3f+1$ and the $2f+1$ quorum. Fault-injection positions are specified by the `faults` dictionary in `scripts7/matrix_demo.py`. On submission reproduction, these parameters should be made public with the code, ensuring "where the numbers come from" is fully traceable.

F.7 One-Sentence Restatement of the Main Result

On the single-seed pilot of 24 work orders, 9 specialists, and 7 QA (including 2 Byzantine), after injecting crash/drop_context/byzantine faults, the hierarchical matrix, together with three-tier breaking, snapshot rollback, reassignment, and BFT-lite acceptance consensus, still achieves 24/24 completion, isolates all 3 faulty nodes, and incurs 5 reworks, with the six governance items, conservation, and hash-chained Trace integrity maintained; under the extreme scenario of all-domain faults plus zero retry budget, the system honestly trips the kill-switch to report non-completion. This is single-seed pilot evidence with design first and full-factorial multi-seed pending, not a population causal conclusion.

F.8 Claims Not Made

To avoid over-claiming, this paper explicitly does not make the following claims: it does not claim "multi-agent fault governance necessarily works in production"; it does not claim "the 24/24 completion rate can be generalized to any fault rate"; it does not claim "the breaker threshold 0.5 is optimal"; it does not claim "the boundary under Byzantine proportions has been measured." All of these can only be answered after the full-factorial multi-seed and the real-LLM gate are unlocked. This paper currently only claims: on the deterministic CPU test bench, this set of factorized governance mechanisms works as designed and honestly fails when recovery is impossible.

F.9 Acknowledgments and Limitation Statement

This paper's test bench is built on the accumulation of several earlier versions of this repo (three topologies, blackboard collaboration, BFT consensus, internal market, observability). The authors thank these early versions for providing runnable components. At the same time the limitation must be restated: the author is the developer, and confirmation bias exists; this paper hedges this bias by making all archives public, retaining the kill-switch counterexample, and labeling the single seed. Readers should treat this paper as an engineering report of "design + single-seed pilot," not as a completed causal conclusion.

F.10 Closing

Fault governance does not make the system incapable of breaking, but makes the system, when it breaks, have rules to follow, evidence to consult, and a limit to stop at. This paper builds this "rules, evidence, limit" into runnable code and reproducible experiments, and honestly reports how far it can currently go. This paper believes this honesty itself is the proper starting point of fault-governance research. After the next full-factorial multi-seed experiment is complete, this paper will replace the current single-seed point estimate with OR and confidence intervals and update the evidence-strength grades of all the above conclusions accordingly. Before then, this paper retains its positioning as a design and pilot report.



Typed Finality: Upgrading the Binary BFT-lite Stop Decision into `strong_stop` / `weak_stop` / `abort` Three Levels of Semantic Commit

Working Title (EN): Typed Finality: Upgrading the Binary BFT-lite Stop Decision into Three Levels of Semantic Commit for LLM-Agent Committees

Consensus governance paper P16 • semantic upgrade of v7.5.0 P2 "BFT-lite for Multi-Agent Stop Decisions"
• sister paper to P26 (weighted BFT) and P32 (rules over learning) • UDOS Reasoning Engine v7.6.0 • Fang Wenxin • 2026-09-21

One sentence first: LLM committees produce stochastic natural language each round and cannot, like classical BFT, agree on "exactly identical bytes"; the binary STOP/CONTINUE of v7.5.0 P2 compresses "how semantically aligned" into one switch, which is too coarse. This paper upgrades it into three levels of **typed finality**—`strong_stop` ($\geq 2f+1$ members agree on the semantic core), `weak_stop` (agree only on the verdict

label), abort (not aligned, turn to view change)—and takes the self-withdrawn negative result of H-CSC v2 as the red line for "what not to do." The three-level upgrade is a **design proposal • pending implementation**, with no new experimental numbers.

Evidence scope (stated only once in the whole paper): The external dialogue object of this paper is H-CSC (arXiv:2606.07316, **note v2 has been retitled** to Certifiable Semantic Agreement Among LLM Agents: What the Admissibility Instrument Decides); its existence and the "typed commit objects / $2f+1$ certificate envelope" have been verified; its negative-result numbers (aggregation distance, honest retention rate, lexical predicate and learned-encoder AUROC) are **paper-reported, not independently rechecked**, and are the v2 actively-withdrawn scope. All UDOS-side numbers go back to v7.5.0 P2 (`udos7/topology/consensus.py` , evidence grade `cpu-proto / verified`). This paper newly creates no UDOS measured numbers; the benefits of three-level finality are all marked `[RESULT NEEDED]` . Verification date 2026-09-21.

Abstract

v7.5.0 P2 reduces the terminal decision of "whether a work order is done or needs rework" to a minimal Byzantine fault-tolerant committee: committee size $\geq 3f+1$, quorum $\geq 2f+1$, same-round equivocation invalidates the whole round, and more than f silence returns `NO_QUORUM` and triggers view change. At $n=7, f=2$, with 10 property tests (verified), it guards the three safety/liveness properties of "honest supermajority stops, Byzantine cannot force a stop, silence turns to a view." But it has an unstated coarseness: **the decision space is binary**—there is no intermediate state between STOP and CONTINUE, and the real semantic-consensus gradient of "everyone's conclusions are roughly right but wording diverges" is hard-compressed into the two bits of "stop or not."

H-CSC (arXiv:2606.07316, v2) precisely addresses this coarseness. It points out: classical BFT requires all honest nodes to agree on **exactly the same byte sequence**, whereas LLM Agent output is stochastic natural language and byte-level agreement is in principle impossible; so it lets each round produce one of three **typed results**—`semantic_commit` (signing a quantitative aggregated summary of the semantic core, fine-grained, high-information, higher risk), `verdict_commit` (signing only the pass/fail label, coarse-grained, low-information, safer), `typed_abort` (explicitly refusing to commit), all three sharing the same $\geq 2f+1$ distinct-signer certificate envelope. More importantly, H-CSC v2 does something rare in systems papers: after re-checking with a stricter "steelman" baseline, it **actively withdrew** the v1 claim that "our filtered aggregation retains more honest content."

Based on this, this paper proposes a three-level typed finality design: rewrite the binary STOP/CONTINUE of P2 into `strong_stop` (semantic-core-level agreement, corresponding to `semantic_commit`), `weak_stop` (only `verdict_label` agreement, corresponding to `verdict_commit`), `abort` (not aligned, corresponding to `typed_abort` and triggering view change). This paper also turns the H-CSC v2 negative result into a red line: **do not use a learned encoder to replace rule checking**—in its adversarial experiments, the AUROC of the deterministic lexical consistency predicate (0.865–0.982) is significantly better than the learned semantic encoder (0.621–0.744). The three-level upgrade has not been implemented, and its safety/liveness properties, costs, and failure modes are all `[RESULT NEEDED]` ; benefits can only be claimed after extending P2's existing property-test suite.

Keywords: typed finality; semantic BFT; stop decision; quorum; equivocation; steelman baseline; negative result; rule predicate

Structured Abstract (Background problem → Argument → Evidence → Contributions)

- **Background problem:** LLM committee output is stochastic natural language and cannot reach byte-level consensus; while the binary STOP/CONTINUE of v7.5.0 P2 compresses "how strong the semantic consensus is" into one switch, losing the safety information of the intermediate state "semantic core already aligned, only wording diverges."
- **Core argument:** Stop decisions should be **graded by the degree of semantic agreement**, rather than binarized. The same $2f+1$ certificate envelope can bind different commit granularities: semantic-core agreement gives a strong stop, only-label agreement gives a weak stop, and non-agreement gives abort and turns to a view; grading lets the system gradient-match "wrong-stop risk" and "rework cost" rather than using one cut.
- **Evidence threads:** (1) the typed commit object design of H-CSC v2 (paper-reported); (2) H-CSC v2 actively withdrawing "honest retention advantage" with a steelman baseline—the filtered aggregation is instead farther from the honest center ($5.32^\circ/5.48^\circ$ vs $4.27^\circ/4.32^\circ$), with honest retention 0.6475 vs 0.7325 against steelman M3, and the authors explicitly write "we withdraw it" (paper-reported); (3) deterministic lexical predicate AUROC 0.865–0.982 beats learned encoder 0.621–0.744 (paper-reported); (4) the 10 property tests and 24-order end-to-end of UDOS v7.5.0 P2 (cpu-proto / verified).
- **Contributions:** (i) the formal design upgrading P2's binary stop decision into three-level typed finality; (ii) turning the H-CSC v2 self-withdrawn negative result into a UDOS design red line (rules as base, learning as supplement); (iii) a falsifiable property list for the three-level decision and pending [RESULT NEEDED] items; (iv) making the failure boundary of this proposal clear.

1 The Problem: What the Binary Stop Decision Loses

1.1 What P2 Got Right

The contribution of v7.5.0 P2 is to **reduce** the "safety/liveness" of classical BFT from natural-language proofs into a CI-able, regression-able property-test suite in the repo. It does not pursue generic state-machine replication but serves one specific decision point—the acceptance committee voting on "whether a work order is done." Its skeleton is:

- Committee $V=\{v_1, \dots, v_n\}$, at most f members Byzantine, $n \geq 3f+1$;
- Each round members vote on a binary decision, quorum $q=2f+1$;
- Same-round equivocation (swing voting) members invalidate the whole round;
- More than f silence returns `NO_QUORUM` and triggers view change, rather than hanging.

At $n=7, f=2$ ($q=5$), 10 deterministic property tests (verified) cover illegal- n rejection, honest supermajority stop, Byzantine inability to force a stop, equivocation detection and whole-round invalidation, conflicting-quorum unreachability, silence triggering view change, and unknown-voter rejection; in the end-to-end integration (cpu-proto, single seed), the 7-person QA committee accepts all 24 work orders correctly under 2 Byzantine reverse-voting members. P2 also honestly states three boundaries: SHA-256 hash placeholder signatures are not cryptographically secure, network partitions and adaptive attackers are not modeled, and **semantic common-cause faults break through majority votes**.

1.2 The Link It Loses: The Decision Space Is Too Coarse

P2 limits the decision space to binary (STOP/CONTINUE), with the reason written clearly: the safety analysis of a binary quorum is clearest and fits the real terminal of "a work order is either accepted or reworked." This simplification is correct in the **byte-level** world—the replicated state machine of classical BFT votes on deterministic byte streams in the first place.

But it is not correct in the LLM world. Five conclusions a QA committee gives for the same work order may be:

1. **Semantic core aligned, wording diverges:** All five judge "pass," each writing their own reasons, with even the cited key numbers slightly differing;
2. **Labels aligned, reasons conflict:** All five check "pass," but three think "barely pass" and two think "fully pass," with acceptance confidence uneven;
3. **Completely not aligned:** Half pass and half fail.

A binary switch can only distinguish "whether someone gathered enough $\$q\$$ STOP votes," and **cannot distinguish the risk levels of the three cases above**. Case 1 is actually already safe (semantic core aligned) yet shares the same "stop" with case 3; case 2 is the most dangerous—the label is captured by the majority while reasons split, and P2's property tests precisely point out this is the breeding ground for "semantic common-cause faults breaking through majority votes," but the binary decision gives it no separate outlet.

This is where H-CSC cuts in: **rather than voting on bytes, first decide "on what granularity of semantic product to vote."**

2 H-CSC's Solution: Classify First, Then Commit

2.1 Why Byte-Level Agreement Is Impossible

The premise of classical BFT is state-machine replication: all replicas execute **exactly the same** state transitions, so it suffices to vote on the same byte sequence. LLM Agents do not satisfy this premise—under the same prompt and temperature, two samplings produce two different natural-language outputs. Requiring a committee to reach $\$2f+1\$$ agreement on "verbatim identical" text fails in principle (probability decays exponentially with length).

The core claim of H-CSC v2 (after retitling, Certifiable Semantic Agreement Among LLM Agents: What the Admissibility Instrument Decides) is: **finality should be typed**—the protocol must first decide "what this round actually commits" and then sign. It gives three commit objects (paper-reported):

- `semantic_commit`: Signing a quantitative aggregated summary of the **semantic core**. Fine-grained, high-information, but to aggregate divergent natural language into a signable core, the aggregation process itself may introduce bias (higher risk);
- `verdict_commit`: Signing only the **label** (pass/fail). Coarse-grained, low-information, but the label is discrete and mechanically comparable, the safest;
- `typed_abort`: Explicitly **refusing to commit**, meaning this round produces no promise.

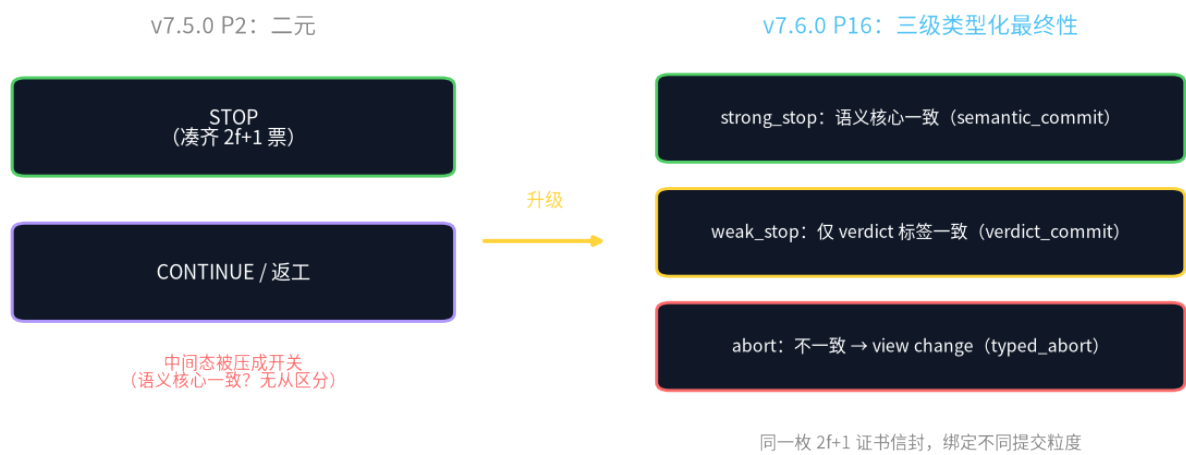
All three share the same $\$2f+1\$$ distinct-signer certificate envelope—that is, the safety skeleton of "how many endorse" is unchanged, and what changes is **what this certificate actually commits**. This design is very friendly to UDOS: it does not need to overturn P2's $\$n\geq 3f+1\$$, $\$2f+1\$$, and equivocation whole-

round invalidation, but only needs to add, "after gathering the quorum," a step of "which level this quorum commits."

2.2 This Paper's Three-Level Mapping

Map the three H-CSC commit objects directly onto P2's stop decision (design proposal):

Three-level finality	Corresponding H-CSC object	Trigger condition (design)	Engineering meaning
strong_stop	semantic_commit	$\geq 2f+1$ members agree on the semantic core (aggregated summary of acceptance points)	high-confidence completion, entering settlement/archiving
weak_stop	verdict_commit	$\geq q$ members agree only on the pass label , but the semantic core does not reach agreement	low-confidence completion, leave a "barely pass" trace, sample-review next round
abort	typed_abort	Neither semantic core nor label reaches agreement, or a semantic common-cause fault sign is detected	turn to view change / add rounds, rather than hard-stopping or hard-reworking



设计提案 · 待实施 [RESULT NEEDED]; 三级收益无新实验

Figure 1 The binary STOP/CONTINUE of v7.5.0 P2 compresses the semantic-consensus gradient into a switch; P16 upgrades it into strong_stop / weak_stop / abort three levels, with the same 2f+1 certificate envelope binding different commit granularities. Conceptual illustration, not measured data.

The key benefit of this mapping is to give each of the three cases in Sec. 1.2 an outlet: case 1 lands in strong_stop, case 2 lands in weak_stop (and triggers next-round review rather than quietly high-confidence archiving), and case 3 lands in abort (turning to a view). **Binary P2 misreads case 2 as a safe "stop," while the three-level design makes it show up as "weak stop + review."**

3 H-CSC v2's Self-Withdrawal: This Paper's Most Important Negative Result

3.1 The Steelman Baseline and "we withdraw it"

Any design borrowing from H-CSC must first digest its v2 self-correction—this is exactly why this paper lists it as a dialogue object rather than a template.

H-CSC v1 once claimed: its "filtered aggregation" mechanism is better than a simple majority at retaining honest content. After re-checking with a **stricter steelman baseline** (i.e. giving the comparison side a stronger, fairer opponent rather than deliberately finding a weak baseline to set itself off), v2 found (paper-reported numbers, not independently rechecked):

- H3: The angle of the core aggregation from the honest center is **5.32° / 5.48°** for H-CSC, while the steelman baseline is **4.27° / 4.32°**—**H-CSC is instead farther from the honest center**;
- H4: Honest retention rate, under the v1 scope H-CSC 0.6475 vs weak baseline 0.4950 (looks dominant); but under the steelman M3 baseline it is **0.6475 vs 0.7325**—**H-CSC is instead lower**. The authors explicitly write "**we withdraw it**" (withdrawing the claim).

This withdrawal is extremely rare in systems papers, and its methodological value is greater than any of its positive conclusions: **a seemingly clever "learned filtered aggregation," under a fair baseline, not only brings no benefit but is worse**. It reminds UDOS: do not introduce a module because it "sounds smart"; every time a processing step is introduced, one must ask "who is its steelman opponent and did it really win." This is fully consistent with the restrained posture of P2 actively disclosing "the full-factorial contrast with single-point QA and naive majority votes has not been done."

3.2 Another Negative Result: The Learned Encoder Is Defeated by the Lexical Predicate

The second hard result of H-CSC v2 is likewise negative but with a clear direction (paper-reported, not independently rechecked): in adversarial scenarios, among the tools deciding "whether two natural-language outputs are semantically consistent," the **deterministic lexical consistency predicate** (mechanical comparison by an agreed set of keywords/phrases) has AUROC **0.865–0.982**, while the **learned CRSE semantic encoder** (using a neural network to encode text into a vector space and then compute similarity) has AUROC only **0.621–0.744**.

That is, on the task of "recognizing whether semantics are consistent," the more advanced-looking neural-network tool is significantly defeated by a simple rule predicate. This directly leads to P32 (rules over learning) and directly constrains this paper's design: **the step in three-level finality of "judging whether the semantic core is consistent" must first be based on a rule predicate, and the learned model can only be a supplement and must pass a steelman contrast before entering a safety-critical path**.

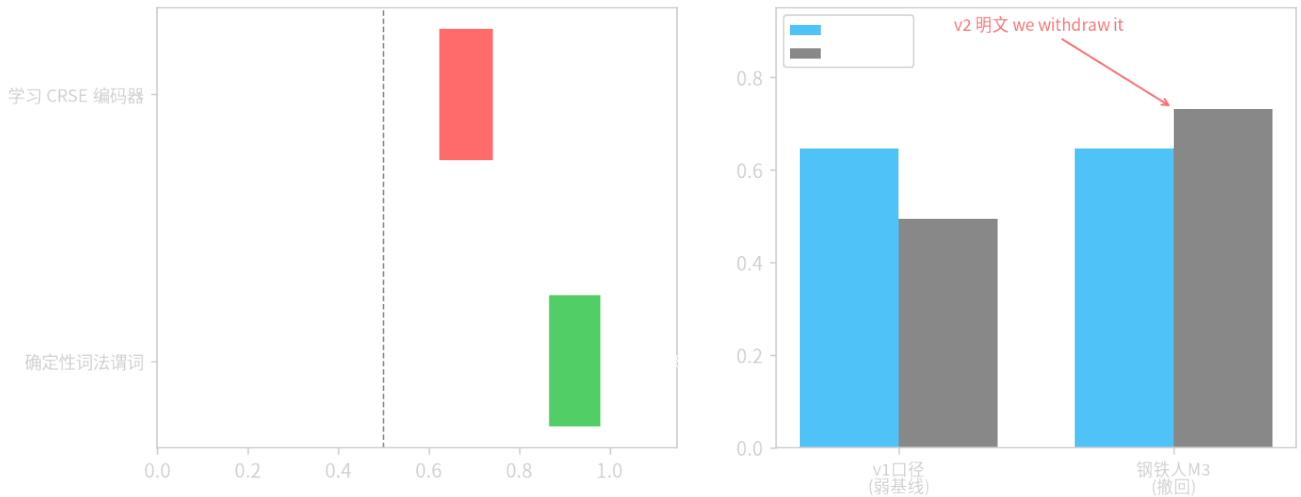


Figure 2 Left: in adversarial scenarios the deterministic lexical predicate AUROC (0.865–0.982) is significantly better than the learned CRSE encoder (0.621–0.744); right: under the steelman baseline the honest retention rate does not rise but falls, explicitly withdrawn in v2. Numbers are paper-reported, not independently rechecked.

4 Formal Design of Three-Level Typed Finality (Proposal)

4.1 Decision Function

Extended on P2's `decide(round_id)`. Suppose each round, besides voting STOP/CONTINUE, members also produce two signable products: L_i (verdict label, discrete) and C_i (comparable fingerprint of the semantic-core aggregated summary). After gathering $q=2f+1$ STOP votes, do one more grading judgment:

$$\text{level} = \begin{cases} \text{strong_stop}, & \text{if the } C_i \text{ of the } q \text{ votes are judged consistent by the rule predicate} \\ \text{weak_stop}, & \text{if the } L_i \text{ of the } q \text{ votes are consistent but } C_i \text{ are not} \\ \text{abort}, & \text{if } L_i \text{ do not reach agreement or a semantic common-cause sign is detected} \end{cases}$$

(This is design pseudocode; the concrete implementation of `cases` is rewritten with safety commands: the `strong_stop` / `weak_stop` / `abort` three branches are given by boolean combinations of rule predicates.) The safety skeleton is unchanged: equivocation invalidates the whole round, more than f silence returns `NO_QUORUM`, and abort triggers view change.

4.2 Relationship with P2's Three Properties

The three-level design **inherits** the three properties P2 has verified (verified):

- **Safety:** `strong_stop` and `abort` cannot be gathered at the same time, because their judgments of C_i are mutually exclusive;
- **Liveness:** `weak_stop` does not hang but gives "barely pass + next-round review"; `abort` turns to a view rather than waiting forever;
- **Illegal configuration rejection:** $n < 3f+1$ is still rejected at the constructor layer, independent of grading.

But it **adds** one property P2 does not have, which has not yet been verified ([RESULT NEEDED]):

- **H4-new (grading monotonicity):** If the degree of semantic-core agreement in a round is higher, the strong_stop share should not anomalously fall; under the steelman baseline, the "wrong-stop rate" of the three-level decision must be **no higher than** P2's binary decision. This comes directly from the H-CSC v2 lesson—any grading module must prove it is not a "worse filter" under a fair baseline.

4.3 Pending Implementation List

The following are all design proposals with no experimental support:

1. Add $\$C_i\$$ semantic-core fingerprints and rule-predicate judgment in `udos7/topology/consensus.py`; [RESULT NEEDED]
2. Extend the 10 property tests into a regression suite covering three-level decisions; [RESULT NEEDED]
3. Design a steelman contrast (naive verdict majority vs three levels), measuring wrong-stop/rework rates; [RESULT NEEDED]
4. Verify that the "next-round review" of weak_stop does not introduce unbounded loops (liveness); [RESULT NEEDED]

5 Dross / Failure Boundaries / Falsifiable Conditions

Dross (what not to do):

1. **Do not copy H-CSC's learned aggregation as safety-critical.** Its steelman contrast proves it is worse (5.32° vs 4.27°; honest retention 0.6475 vs 0.7325), and this paper only takes its "typed commit" idea, not its learned-aggregation implementation.
2. **Do not treat strong_stop as "absolutely safe."** The $\$2f+1\$$ certificate only guarantees "no $\$>f\$$ Byzantine," not that semantic common-cause faults do not break through—P2 has stated this boundary, and the three-level design does not remove it.
3. **Do not misread "three levels" as "all three levels can settle."** weak_stop is "leave a trace + review," not high-confidence archiving; if engineering also settles weak_stop directly, it degenerates into a binary decision with labels.

Failure boundaries:

- When the rule predicate for $\$C_i\$$ is designed too loosely (misjudging wording divergence as semantic agreement), strong_stop will be inflated and the wrong-stop rate rises;
- When a semantic common-cause fault occurs (all QA misled by the same point), the "semantic-core agreement" of strong_stop may itself be **collusive agreement**, and the certificate cannot distinguish "honest agreement" from "induced agreement";
- Three-level judgment introduces an extra round of fingerprint aggregation, costing more than binary decisions, and at very large scale fan-in must be evaluated (coupled with P1 topology complexity).

Falsifiable conditions:

- If under the steelman baseline the wrong-stop rate of the three-level decision is **not lower than** P2's binary decision, H4-new is overturned and this proposal should be rolled back;
- If the rule predicate's AUROC on a new attack family falls below the learned encoder (the current 0.865–0.982 vs 0.621–0.744 advantage reverses), the "rules as base" red line must be re-evaluated;

- If the review of `weak_stop` in actual measurement introduces significant unbounded loops, the liveness hypothesis of the three-level design fails.

6 Conclusion

v7.5.0 P2 makes the stop decision into a falsifiable, CI-able binary committee; H-CSC v2 reveals the semantic-consensus gradient lost by binarization and, with a rare self-withdrawal, warns that "a learned clever module may be worse under a fair baseline." This paper's stance is restrained: **only take the typed-commit idea, upgrading P2's STOP/CONTINUE into `strong_stop` / `weak_stop` / `abort` three levels; the step of judging semantic consistency is based on a rule predicate with the learned model filling in, and a steelman contrast is mandatory.** Three-level finality is a design proposal whose benefits and failure modes are all `[RESULT NEEDED]`—before P2's property-test suite is extended and the steelman contrast runs through, this paper claims no actual improvement.

Replacing "voting on bytes" with "first deciding on what granularity of semantic promise to vote" is a step BFT in the LLM era must cross; but when crossing this step, use the honesty with which H-CSC v2 itself withdrew its conclusion to examine every newly introduced step.

References

Classical literature

1. Castro, M., & Liskov, B. (1999). Practical Byzantine Fault Tolerance. OSDI 1999. (the classical skeleton of $n \geq 3f+1$, $2f+1$ quorum, view change)
2. Yin, M., Malkhi, D., Reiter, M. K., et al. (2019). HotStuff: BFT Consensus in the Lens of Blockchain. PODC 2019. arXiv:1803.05069. (linear-communication-complexity BFT, chained view change)
3. Wu, Q., Bansal, G., Zhang, Y., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. (conversable multi-agent framework, whose "conversations without termination conditions easily dead-loop" is one motivation of this paper)

2026 frontier materials (existence verified; numbers paper-reported, not independently rechecked)

1. H-CSC author team (2026). Certifiable Semantic Agreement Among LLM Agents: What the Admissibility Instrument Decides (v2; v1 original title Hierarchical Certified Semantic Commitment...). arXiv:2606.07316v2. (typed commit objects `semantic_commit` / `verdict_commit` / `typed_abort`, $2f+1$ certificate envelope; actively withdrawing "honest retention advantage" under the steelman baseline—aggregation distance $5.32^\circ/5.48^\circ$ vs $4.27^\circ/4.32^\circ$, honest retention 0.6475 vs 0.7325; lexical predicate AUROC 0.865–0.982 beats learned encoder 0.621–0.744)

UDOS paper volume (evidence grades marked in text)

1. UDOS v7.5.0 P2 "BFT-lite for Multi-Agent Stop Decisions: Safety Verification of the $n \geq 3f+1$ Committee, Equivocation Whole-Round Invalidation, and View Change" (`udos7/topology/consensus.py`, 10 property tests verified; $n=7, f=2$, 24-order end-to-end cpu-proto).
2. UDOS v7.6.0 P26 "Weighted BFT and Reputation: The Weight Direction of Equal-Weight $2f+1$ ", P32 "Rules over Learning: Predicate Robustness Boundaries in Adversarial Scenarios" (sister papers).

Evidence Discipline and Reproduction Notes

- This paper is a design-proposal paper with no new experiment; all H-CSC numbers (aggregation distance, honest retention rate, AUROC intervals) are **paper-reported, not independently rechecked**, and its v2 has actively withdrawn v1's "honest retention advantage" claim. This paper strictly adopts the v2 scope and does not use v1 positive conclusions.
- All UDOS-side numbers go back to v7.5.0 P2: the 10 property tests are verified, and the 24-order end-to-end at $n=7, f=2$ is a cpu-proto single-seed pilot point estimate; the three boundaries of P2—SHA-256 hash placeholder signatures are not cryptographically secure, network partitions and adaptive attackers are not modeled, and semantic common-cause faults break through majority votes—**still hold** for this paper's three-level design.
- All benefits of three-level typed finality, H4-new monotonicity, and the weak_stop unbounded-loop risk are marked [RESULT NEEDED], claimable only after extending the existing consensus.py and running through the steelman contrast; ≥ 30 -seed reproduction is needed before submission.



Weighted BFT and Reputation: From Equal-Weight $2f+1$ to Credibility-Weighted Stop Decisions — A Design Direction

Working Title (EN): Weighted BFT and Reputation: From Equal-Weight $2f+1$ Quorums to Credibility-Weighted Stop Decisions — A Design Direction, Not an Implementation

Consensus governance paper P26 • the weight dimension of v7.5.0 P2 "BFT-lite Stop Decisions" • sister paper to P16 (typed finality) and P31 (fail-safe recovery) • UDOS Reasoning Engine v7.6.0 • Fang Wenxin • 2026-09-21

One sentence first: The committee of v7.5.0 P2 is a **one-person-one-vote, equal-weight $2f+1$** quorum—among 7 QA everyone's vote weighs the same. This is correct when "all members are of comparable ability and wrong votes are independent"; but in a real LLM committee, member quality is uneven and historical performance is traceable. This paper discusses a **design direction**: upgrading the "equal-weight quorum" into a "quorum weighted by historical credibility," and surveys three recent threads—weighted BFT /

coalition governance / proof-of-thought consensus—but explicitly states: **UDOS currently does not implement weighted voting, and all benefits are design hypotheses marked [RESULT NEEDED] .**

Evidence scope (stated only once in the whole paper): All UDOS-side numbers go back to v7.5.0 P2 (cpu-proto / verified). The recent works mentioned in this paper—weighted BFT (WBFT class), CP-WBFT, COALITION-VAST, BlockAgents (Proof-of-Thought)—all come from research threads provided by the user, and **their exact numbers have not been verified one by one in the v7.6.0 evidence ledger (EVIDENCE_LEDGER_v76.md)**; wherever specific percentages/thresholds are cited, they are labeled "thread scope • unverified" or [CITATION NEEDED] and are not treated as facts. The safety of equal-weight $2f+1$ and view change comes from classical PBFT (Castro & Liskov, 1999). Verification date 2026-09-21.

Abstract

v7.5.0 P2 uses equal-weight BFT-lite to guard the "no wrong stop" bottom line of the LLM acceptance committee: $n \geq 3f+1$, quorum $q=2f+1$, equivocation invalidates the whole round, and more than f silence turns to view change. The equal-weight design has a clean safety proof—there are only $n \geq 2f+1=q$ honest votes, so it is impossible for STOP and CONTINUE to each gather q . But this proof implicitly assumes: **the f Byzantine members are a "worst-case uniformly distributed minority," and all honest members have equal voting weight.** A real LLM committee does not satisfy the latter: some QA have high historical accuracy, while others often overstep or are ambiguous; counting the votes of these two classes as equally heavy amounts to wasting the free signal of "who is more credible."

This paper surveys the "equal weight $\rightarrow$ weighted" design space along three directions rather than implementing it. First, **weighting by historical credibility** (WBFT-class idea): bind each member's voting weight to its past acceptance accuracy so that unreliable members' votes contribute less to the quorum; the cost is introducing a new attack surface of "how weights update and how to prevent weight manipulation." Second, **coalition governance and targeted audit** (COALITION-VAST-class idea): when Byzantine behavior is not single-point malice but a few members **colluding** to induce agreement, equal-weight majority votes are precisely the target exploited by collusion; such works use coalition game theory + on-chain rule evolution + trust-driven targeted audit to discover "collusive agreement." Third, **turning the thought process into a consensus object** (BlockAgents' Proof-of-Thought): vote not only on the final label but on the "reasoning process" and stake on it, making poisoning/backdoor attacks harder to hide.

This paper's conclusion is restrained: **weighting and reputation are the natural next dimension of P2's equal-weight design, but it is a design direction rather than an implementation.** The reason the equal-weight $2f+1$ proof is clean is precisely that it does not depend on the correctness of any "weight estimation"; once weights are introduced, safety changes from "pure combinatorial" to the product of two variables, "combinatorial $\times$ weight-estimation quality," and the withdrawal lesson of H-CSC v2 (a learned module being worse under a fair baseline) precisely warns us: do not rush to put the "smarter-looking" weighting logic into a safety-critical path. The benefits of all weighting schemes are [RESULT NEEDED] .

Keywords: weighted BFT; reputation weights; quorum; coalition governance; proof of thought; stop decision; design direction

Structured Abstract (Background problem → Argument → Evidence → Contributions)

- **Background problem:** P2's equal-weight $2f+1$ assumes "all honest members equal-weight, Byzantine uniformly distributed," but real LLM committee member quality is uneven and the signal of "who is more credible" is wasted by the equal-weight design; meanwhile Byzantine behavior may upgrade from single-point malice to **collusion-induced agreement**, with equal-weight majority votes being the target of collusion.
- **Core argument:** Upgrading the equal-weight quorum to historical-credibility weighting is the natural next dimension of P2; but weighting turns "pure combinatorial safety" into "combinatorial $\times$ weight estimation," adding the attack surface of weight updates and weight manipulation, so it should be cautiously explored as a **design direction** rather than immediately replacing the verified equal-weight committee.
- **Evidence threads:** (1) the equal-weight $2f+1$ safety of classical PBFT (Castro & Liskov, 1999); (2) UDOS P2's 10 property tests and 24-order end-to-end at $n=7, f=2$ (cpu-proto / verified); (3) the three recent threads WBFT / CP-WBFT / COALITION-VAST / BlockAgents (**numbers unverified**); (4) the H-CSC v2 self-withdrawal (weighting/aggregation modules must pass a steelman contrast).
- **Contributions:** (i) formalizing the "equal weight $\rightarrow$ weighted" design space and costs; (ii) extending the Byzantine threat from "single-point malice" to "collusion-induced agreement"; (iii) clearly listing the safety-property list of weighting schemes and [RESULT NEEDED] ; (iv) stating UDOS currently does not implement weighting and only registers the direction.

1 Starting Point: The Clean Proof of Equal-Weight $2f+1$ and Its Assumptions

1.1 Why P2 Chose Equal Weight

The reason the safety proof of v7.5.0 P2 can be written as 10 CI-able property tests is that it **deliberately chose the simplest voting model**—equal weight, one person one vote. Its core proof (verified):

$$\$ \$ \text{\text{honest votes}} = n - f \geq 3f+1 - f = 2f+1 = q \$ \$$$

Under $n \geq 3f+1$, there are at most q honest votes, so STOP and CONTINUE **cannot each** gather q honest votes—this is the safety of "at most one decision value." This proof does not depend on any estimate of "who is more accurate"; it is purely combinatorial. P2 actively discloses: its signatures are SHA-256 hash placeholders that do not resist real forgery, and **semantic common-cause faults break through majority votes**.

1.2 What Equal Weight Wastes

Equal weight assumes "all honest members are equally credible." But in a real LLM committee this assumption often does not hold:

- Member A has high historical acceptance accuracy and is never ambiguous;
- Member B often gives "barely pass" and occasionally oversteps to check a work order that should not stop as pass;
- Member C is newly joined with no history.

The equal-weight design counts A's and B's votes as equally heavy. This means: to gather $q=2f+1$, you must "trust enough people," even if some of them are known unreliable. **Weight is a free signal—it tells you "when gathering q , you need not count everyone."** This is precisely the intuitive source of weighted BFT.

But note at the same time why the proof of Sec. 1.1 is clean: it **needs no** estimate of weights. This is a trade-off—exchanging "simplicity of the safety proof" for "the ability to use credibility signals." The rest of this paper computes both sides of this trade-off clearly.

2 Direction One: Weighting by Historical Credibility (WBFT Class)

2.1 Design Intuition

Let member v_i 's historical credibility be $w_i \geq 0$, and change the quorum from "vote count" to "weight sum":

$$\sum_{i \in S} w_i \geq Q, \quad Q = 2f+1 \quad \text{weight units (design proposal)}$$

Intuitively, unreliable members' w_i are small, and gathering Q relies mainly on high-credibility members' votes; this way, **under the same n , fewer high-credibility members need to be "convinced," and Byzantine nodes find it harder to manipulate the stop decision.** The recent weighted-BFT thread (called WBFT class in user research, IEEE TCCN direction, **specific algorithm details and numbers not verified in this ledger, [CITATION NEEDED]**) precisely binds voting weights to each LLM's historical response quality and credibility, so as to incentivize reliable behavior and reduce malicious-node impact. Another thread claims CP-WBFT (AAAI 2026 direction) can tolerate a high proportion of faulty nodes—its claims such as "85.7% fault rate" are **user-thread restatements, unverified**, and this paper does not accept them as facts.



Figure 3 Comparison of P2 one-person-one-vote (left) and historical-credibility weighting (right); weighting introduces three new attack surfaces: weight update / cold start / equivocation interaction. Conceptual illustration, not measured data.

2.2 The Cost of This Trade-off

Weighting is not free. After introducing w_i , safety changes from "pure combinatorial" to:

$$\text{safety} = \text{combinatorial correctness} \times \text{weight-estimation correctness}$$

Specifically, three new attack surfaces are added:

1. **Weight-update manipulation:** A Byzantine member can "behave well early to farm weight and act maliciously at the critical moment"—weight is history, and malice can lag.
2. **Cold start:** A new member's w_i is undetermined, either given 0 (cannot participate) or a default value (equal to equal weight).
3. **Interaction of weight and equivocation:** P2's equivocation whole-round invalidation is purely rule-based; after weighting, "does whole-round invalidation zero out its weight contribution" needs redefinition.

UDOS's stance: Weight updates **must use rule predicates** (such as the agreement rate with the committee's final decision within a rolling window), and weights cannot be dynamically estimated with learned models—this comes directly from P32's red line (H-CSC v2 proving the learned encoder is defeated by the rule predicate in adversarial scenarios). Before P2's property-test suite is extended and the steelman contrast runs through, UDOS **does not implement** weighted voting.

3 Direction Two: Collusion Governance and Targeted Audit (COALITION-VAST Class)

3.1 From "Single-Point Malice" to "Collusion-Induced Agreement"

P2's fault model assumes the Byzantine members are an **independently malicious minority**. But H-CSC v2 and P2's "semantic common-cause fault" warning point to a more dangerous form: **a few members collude to jointly induce the majority to reach a "looks-aligned" label on the same wrong result**. At this point equal-weight majority votes are **precisely the target exploited**—what the colluders must do is not prevent gathering q , but **help gather q** , letting the system complete the wrong result with high confidence.

This is exactly why P2's binary decision is dangerous: it only asks "whether STOP is gathered," not "whether this STOP is honest agreement or induced agreement." P16's three-level typed finality gives an outlet for "whether the semantic core is truly aligned"; but what if the colluders forge even the semantic core into alignment?

3.2 Coalition Game Theory + Targeted Audit

The recent coalition-governance thread (called COALITION-VAST in user research, IEEE 2026-08 direction, **numbers unverified, [CITATION NEEDED]**) targets precisely such "coalition deviation, governance capture, hasty rule change." Its approach: use coalition game theory to identify "which members' votes exhibit collusion patterns," use on-chain rule evolution to prevent governance from being hastily tampered with, and use trust-driven **targeted audit** (auditing only the most suspicious subset rather than full re-audit) to hold audit cost down. The user thread claims that in three scenarios—medical resource allocation, autonomous swarms, and multi-stakeholder finance—alignment is about 0.90 vs a 0.69 baseline and undetected coalition deviation falls by about 98%—**these numbers are thread restatements, not verified in this ledger, and are not treated as facts**.

The lesson for UDOS is not "copying a governance framework" but **a falsifiable design principle**: the audit of stop decisions cannot only look at "whether the quorum is gathered," but must also look at "whether the voter combination of those votes gathering the quorum exhibits collusion features." Implementing this principle needs P16's \$C_i\$ (semantic-core fingerprint) and P3's Trace hash chain as the base, all currently [RESULT NEEDED] .

4 Direction Three: Turning the Thought Process into a Consensus Object (BlockAgents / Proof-of-Thought)

4.1 Vote Not Only on Labels but on the Reasoning Process

The object of equal-weight BFT voting is the **final label** (STOP/CONTINUE). A deeper thread is: turn the **reasoning process itself** into a consensus object. In user research, BlockAgents (Shanghai Jiao Tong University team, ACM Turing Award Celebration China 2024 direction) is said to propose Proof-of-Thought (PoT) consensus: combining stake-based voter designation with multi-round debate-style voting to hold the accuracy interference of poisoning attacks at a low level and the backdoor success rate at a low level (**user thread claims <3% / <5%, unverified, [CITATION NEEDED]**).

Intuitively, voting on the "reasoning process" is harder to cheat than voting only on the "conclusion"—because the malicious actor must not only forge the conclusion but also forge a "plausible reasoning chain," while each step on the chain can be independently checked. This shares a philosophical origin with P16's semantic_commit (committing to the semantic core rather than only the label): **the finer the commit granularity, the higher the cheating cost, but the higher the aggregation cost as well.**

4.2 Division of Labor with P16

P16 already takes "committing to the semantic core" as one level of three-level finality; BlockAgents-class PoT pushes "committing to each step of the reasoning chain" even finer. UDOS's boundary is: **how fine is worth going?** Doing $2f+1$ endorsement for each reasoning step costs $O(\text{number of steps})$ times fan-in, directly coupled with P1's topology complexity. Before actual measurement proves the wrong-stop-rate improvement of "voting on the reasoning chain" over "voting on the label" is **greater than** its cost, UDOS only registers it as a direction and does not implement it.

5 Contrast and Trade-offs of the Three Directions

Table 1 Equal weight vs the three weighted/fine-grained directions

Direction	Voting object	New signal	New attack surface	Safety dependence	UDOS stance
P2 equal weight $2f+1$	final label	none	fewest	pure combinatorial (verified)	currently implemented
Direction one WBFT weighting	label + historical weight w_i	member credibility	weight manipulation, cold start	combinatorial $\times$ weight estimation	design direction, not implemented
Direction two COALITION- VAST	label + collusion detection	voter combination pattern	audit cost	combinatorial $\times$ collusion recognition	design direction, not implemented
Direction three BlockAgents PoT	each step of the reasoning chain	reasoning process	fan-in cost $O(\text{steps})$	combinatorial $\times$ process checking	design direction, not implemented

投票对象 / 新增信号 / 新增攻击面 / 安全性依赖



每加一个信号：作弊成本 $\uparrow$ ，攻击面 $\uparrow$ ，安全性从纯组合退化为组合 $\times$ 新信号估计

Figure 4 An overview of each direction's voting object, new signal, new attack surface, and safety dependence; each added signal increases both cheating cost and attack surface. Conceptual illustration. The core judgment Table 1 conveys is: **each added signal (credibility, collusion pattern, reasoning process) is exchanged for a higher cheating cost while introducing a new attack surface and a new cost; safety degrades from "pure combinatorial" to "combinatorial $\times$ estimation quality of the new signal."** UDOS currently chooses to stop at the first row (verified by 10 tests) and registers the latter three rows as directions—this is not conservatism, but because P2's clean proof is precisely its most valuable asset.

6 Dross / Failure Boundaries / Falsifiable Conditions

Dross (what not to do):

1. **Do not use learned models to dynamically estimate weights on safety-critical paths.** H-CSC v2 has proven the learned encoder is defeated by the rule predicate in adversarial scenarios (AUROC 0.621–0.744 vs 0.865–0.982); weights must be computed with rule predicates (rolling agreement rate).

2. **Do not write "weighting is safer" as implemented.** UDOS has no weighted voting; any benefits are design hypotheses marked [RESULT NEEDED] .
3. **Do not treat the percentages in user threads as facts.** The specific numbers of WBFT/CP-WBFT/COALITION-VAST/BlockAgents are all unverified in the v7.6.0 ledger and must carry "thread scope • unverified" when cited.

Failure boundaries:

- When historical weights w_i are manipulated by "farm first, act maliciously later," the weighted committee loses defense against **lagging** malice—weight is a rear-view mirror that cannot see future betrayal;
- When colluders forge even the semantic core into alignment, "voting on process" and "voting on label" fail equally—PoT can only raise cheating cost, not eliminate collusion;
- The cost of weighted and fine-grained process voting grows with committee size and step count, and at very large scale may break through P1's fan-in budget.

Falsifiable conditions:

- If under the steelman baseline the wrong-stop rate of the rule-weighted committee is **not lower than** P2's equal-weight committee, the benefit hypothesis of direction one is overturned;
- If collusion recognition's recall on injected collusion attacks is not significantly higher than random guessing, the design principle of direction two does not hold;
- If the wrong-stop-rate improvement of voting on the reasoning chain is insufficient to cover its fan-in cost, direction three has no engineering value in the UDOS scenario.

7 Conclusion

P2's equal-weight $2f+1$ is a "minimal and clean" stop committee guarded by 10 property tests. This paper surveys its natural next dimension along three directions (credibility weighting, collusion governance, process consensus) but writes the conclusion as "**design direction, not implemented.**" The reason is a simple but often-overlooked judgment: **how clean the proof of a safety mechanism is depends on how many external estimates it relies on; the reason equal-weight $2f+1$ can be written as a pure combinatorial proof is precisely that it assumes no member is more credible.** Once weights or process are introduced, safety becomes the product of "combinatorial $\times$ new-signal estimation quality"—and the H-CSC v2 withdrawal has just reminded us: that new-signal estimate may well, under a fair baseline, be worse than the simplest opponent.

First use the verified equal-weight committee steadily, then cautiously, step by step, introduce credibility signals with steelman contrasts; before then, weighting is a direction written in the design document, not code running in consensus.py.

References

Classical literature

1. Castro, M., & Liskov, B. (1999). Practical Byzantine Fault Tolerance. OSDI 1999. (the classical skeleton of equal-weight $n/2+1$, $2f+1$ quorum, view change; BFS only about 3% slower than unreplicated NFS)

2. Yin, M., Malkhi, D., Reiter, M. K., et al. (2019). HotStuff: BFT Consensus in the Lens of Blockchain. PODC 2019. arXiv:1803.05069.

2026 frontier threads (neither existence nor exact numbers verified in the v7.6.0 ledger; the following are user-research thread restatements, not facts)

1. Weighted BFT (WBFT class, IEEE TCCN direction): adaptively allocate voting weights by member response quality and credibility. [CITATION NEEDED: exact algorithm and numbers pending verification]
2. CP-WBFT (AAAI 2026 direction): user thread claims it can tolerate a high proportion of faults; claims such as "85.7% fault rate" unverified. [CITATION NEEDED]
3. COALITION-VAST (IEEE 2026-08 direction): coalition game theory + on-chain rule evolution + trust-driven targeted audit; user thread claims alignment 0.90 vs 0.69, coalition deviation -98%. [CITATION NEEDED: numbers pending verification]
4. BlockAgents (Shanghai Jiao Tong University, 2024 direction): Proof-of-Thought consensus + stake-based voter designation; user thread claims poisoning <3%, backdoor <5%. [CITATION NEEDED: numbers pending verification]

UDOS paper volume (evidence grades marked in text)

1. UDOS v7.5.0 P2 "BFT-lite for Multi-Agent Stop Decisions" (equal weight $n \geq 3f+1$, $q=2f+1$, 10 property tests verified; $n=7, f=2$, 24 orders cpu-proto).
2. UDOS v7.6.0 P16 "Typed Finality", P32 "Rules over Learning" (sister papers; weighting/aggregation modules must pass steelman contrasts, weights use rule predicates).

Evidence Discipline and Reproduction Notes

- This paper is a **design-direction registration** paper with no new experiment and no new implementation; UDOS's current `udos7/topology/consensus.py` only implements equal-weight BFT-lite and does not implement any weighting/collusion detection/process voting.
- The three recent threads (WBFT/CP-WBFT, COALITION-VAST, BlockAgents) all come from threads provided by user research, and their exact numbers have not been verified one by one in the v7.6.0 evidence ledger, all labeled "thread scope • unverified / [CITATION NEEDED]," and are not the factual basis of this paper's argument.
- The wrong-stop rate/rework rate/cost improvement of all weighting schemes are design hypotheses marked [RESULT NEEDED] ; if implemented in the future, they must first be extended on P2's 10 property-test suite and run through a steelman contrast (equal weight vs rule weighting), with ≥ 30 seeds before submission.
- P2's existing boundaries (SHA-256 hash placeholder signatures are not cryptographically secure, network partitions and adaptive attackers are not modeled, and semantic common-cause faults break through majority votes) **all hold and are more severe** for this paper's three directions—weighting/process voting does not eliminate common-cause faults, only raises cheating cost.



Fail-Safe & Recovery: Swarm Quorum Abort, Behavioral-Drift Baseline Rollback, and Cascade Containment as Extensions of the P3 Governance Layer

Working Title (EN): Fail-Safe & Recovery: Swarm Quorum Abort, Behavioral-Drift Baseline Rollback, and Cascade Containment as Extensions of the P3 Governance Layer

Consensus governance paper P31 • recovery-layer extension of v7.5.0 P3 "Causal Effect of Fault-Breaker Governance on Completion Rate" • sister paper to P16 (stop decision) and P26 (weighted BFT) • UDOS Reasoning Engine v7.6.0 • Fang Wenxin • 2026-09-21

One sentence first: v7.5.0 P3 already can "recover if curable, isolate and reassign if not, and honestly fail via kill-switch if utterly impossible"; but its "failure stop" is **single-node, rule-triggered**, lacking three things—**quorum abort** when the whole swarm is led astray together (not decided by one circuit breaker), **automatic baseline rollback** when system behavior quietly drifts (not waiting for fault injection to react),

and **cascade containment** when faults spread across nodes. This paper maps these three things from Pillar 3 of the AI SAFE² framework onto P3's governance layer as a **design extension proposal**, with all benefits [RESULT NEEDED] .

Evidence scope (stated only once in the whole paper): All UDOS-side numbers go back to v7.5.0 P3 (reports7/matrix_scale_demo.json, evidence grade cpu-proto; the governance trio and responsibility governance are this-repo designs). The external dialogue object is **AI SAFE² governance framework v3.0 (2026-05-18) Pillar 3 Fail-Safe & Recovery** of the Cyber Strategy Institute; its existence and the F3.3 / F3.4 points have been verified in the v7.6.0 evidence ledger (official-website scope); specific thresholds are that framework's claims, not reproduced in UDOS. The existing numbers of three-level circuit breaking / snapshot rollback / reassignment / kill-switch are cpu-proto single-seed pilots. Verification date 2026-09-21.

Abstract

v7.5.0 P3 factorizes the fault governance of the multi-agent matrix into three independently switchable mechanism groups—three-level circuit breaking (AgentBreaker / sub-matrix group breaker / global kill-switch), snapshot rollback (hash-chain Trace truncation), reassignment (in-flight work orders handed to healthy nodes)—plus a responsibility governance group (unique Owner, hash-chain Trace, explicit Stop Condition), and with the discipline of a pre-registered full-factorial RCT freezes the primary metric (completion rate accepted/total) and guardrails (rework, messages, mis-isolation rate, Trace break count, kill-switch mis-trigger rate). In the single-seed pilot (24 work orders, 9 specialists, 7 QA including 2 Byzantine): no faults 24/24 completion, 0 rework; after injecting crash / drop_context / byzantine three fault types still 24/24 completion, all 3 faulty nodes isolated, 5 reworks; in the extreme "all-domain faults + zero retry budget" scenario it honestly triggers kill-switch and reports non-completion rather than faking success (cpu-proto).

P3's governance is already closed-loop, but its "recovery" still has three structural gaps, precisely corresponding to the three things systematized by the external AI SAFE² framework Pillar 3 (Fail-Safe & Recovery, 2026-05). First, **Swarm Quorum Abort (F3.3)**: P3's kill-switch is a global single-point rule, and when the whole swarm is induced by the same class of error and "all circuit breakers look normal," rule thresholds alone cannot recognize it; Pillar 3's approach is to **coordinate shutdown when over 25% of Agents become abnormal**, and use **cryptographic attestation weighting** to resist Sybil (i.e. prevent a few identity-spoofing nodes from faking "all is well"). Second, **Behavioral Drift Baseline & Rollback (F3.4)**: P3 only reacts to "fault injection" and has no defense against "system behavior quietly drifting without injected faults"; Pillar 3's approach is to build a baseline for normal behavior and automatically roll back to the last known healthy snapshot when drift exceeds a threshold. Third, **cascade containment**: P3's circuit breaking is node-by-node, and a fault may "jump" from an isolated node to its downstream through the handoff chain; cascade containment requires, while isolating the faulty node, suppressing its influence from spreading to the neighborhood along the Trace path.

This paper maps these three things onto UDOS's existing governance layer: Swarm quorum abort shares a source with the committee mechanism of P16/P26 (using the majority judgment of a group of healthy Agents to replace the single-point ignition rule); drift baseline rollback reuses P3's existing hash-chain Trace snapshots; cascade containment reuses P10's TransferBundle handoff chain. This paper explicitly states: **these three are design extensions, UDOS currently does not implement Swarm quorum abort or drift baseline detection, and their impact on completion rate/mis-isolation rate is all [RESULT NEEDED]** . This paper also takes "honest failure" as a first-class result—a system that cannot roll back or cascade-suppress, even if it can complete, is not an honest fail-safe system.

Keywords: fail-safe; recovery; Swarm quorum abort; behavioral drift; automatic rollback; cascade containment; kill-switch

Structured Abstract (Background problem → Argument → Evidence → Contributions)

- **Background problem:** P3 can break circuits node-by-node, roll back snapshots, and honestly fail via kill-switch, but its "failure stop" is single-node rule-triggered, lacking three things: quorum abort when the swarm is led astray together, drift detection and rollback without injected faults, and cascade suppression when faults spread across nodes.
 - **Core argument:** Fail-safe and recovery should not only react to "known fault types" but make first-class defenses against three **failure forms not covered by P3**: "(a) swarm-level joint induction, (b) drift without injection, (c) cross-node fault spread"; these three correspond respectively to Pillar 3's quorum abort, drift baseline rollback, and cascade containment.
 - **Evidence threads:** (1) the single-seed pilot of UDOS P3 three-level governance (cpu-proto, 24/24 completion, 3 nodes isolated, 5 reworks, honest kill-switch in the extreme scenario); (2) the F3.3 / F3.4 framework points of AI SAFE² Pillar 3 (official-website scope, thresholds are its claims); (3) the primary-metric and guardrail design of P3's pre-registered RCT (design first).
 - **Contributions:** (i) extending P3's node-by-node governance into a three-layer recovery structure of swarm-level quorum abort + drift baseline rollback + cascade containment; (ii) connecting the three things respectively to UDOS's existing components (committee mechanism, hash-chain snapshots, TransferBundle chain) rather than starting anew; (iii) listing the pending [RESULT NEEDED] list and falsifiable conditions.
-

1 What P3 Has Closed-Looped, and What It Still Lacks

1.1 P3's Closed Loop

P3's governance goal is defined as three things: **recover when recovery is possible, isolate and reassign when recovery is impossible, and honestly stop and leave a complete audit trail when utterly impossible**. It measures these three things with the discipline of a pre-registered RCT: the primary metric completion rate = accepted/total, and guardrails are rework, messages, mis-isolation rate, Trace break count, and kill-switch mis-trigger rate. In the cpu-proto single-seed pilot, this closed loop behaves stably when "known faults are injected"—24/24 completion, all 3 faulty nodes isolated, 5 reworks, and honest kill-switch in the extreme scenario.

1.2 Three Failure Forms Not Covered

P3's fault model is **human-injected, of known type, node-by-node** (crash / drop_context / byzantine one each). It does not answer three more hidden failure classes:

1. **Swarm-level joint induction:** It is not one node broken but all nodes led astray by the same wrong context—P3 looks node-by-node at "whether each node is normal," and at this point each node's circuit breaker shows normal, but the whole swarm is converging in the wrong direction. P3's kill-switch is a single-point rule, with no mechanism for "a group of healthy Agents jointly judging whether the swarm has drifted."

2. **Drift without injection:** The system has no fault injected, but its behavior distribution quietly departs from the historical healthy baseline over multiple rounds (such as the reassignment ratio quietly rising and the context-loss rate in handoffs slowly climbing). P3 only reacts to "fault events," not to "gradual changes in behavior distribution."
3. **Cascade spread:** A node isolated by circuit breaking may pass wrong context to downstream through the handoff chain, making downstream nodes gradually abnormal as well. P3's node-by-node circuit breaking isolates the source but does not suppress influence spreading along the Trace path.



Figure 6 Three hidden failure forms—swarm joint induction, drift without injection, and fault cascade spread—and their corresponding extensions. Conceptual illustration.

These three failure forms are precisely the objects systematized by the external AI SAFE² framework Pillar 3 (Fail-Safe & Recovery).



Figure 5 Left: the node-by-node circuit breaking/snapshot/reassignment/kill-switch P3 has implemented; right: the Swarm quorum abort, drift baseline rollback, and cascade containment Pillar 3 extends. Conceptual illustration, not measured data.

2 Direction One: Swarm Quorum Abort (F3.3)

2.1 From "Single-Point Rule Ignition" to "Healthy Majority Judgment"

F3.3 (Swarm Quorum Abort, official-website scope) of AI SAFE² Pillar 3 claims: when **over 25% of Agents become abnormal, a coordinated shutdown should be triggered**, and **cryptographic attestation weighting** is used to resist Sybil attacks—i.e. to prevent a few nodes spoofing many identities from faking an "all is well" signal.

Map it onto UDOS: P3's global kill-switch is currently a single-point rule of "trigger when any guardrail threshold is crossed." Swarm quorum abort upgrades it into **the majority judgment of a group of healthy Agents**—this shares a source with the committee mechanism of P16/P26:

$\text{swarm_abort} = \text{true}$, if the abnormal-Agent share $\geq \rho_0$ and the healthy committee's attestation majority supports shutdown

(ρ_0 is a design threshold, the framework claims about 25%, not reproduced in UDOS.) The key difference: the single-point rule only looks at metrics, while Swarm quorum abort looks at **"whether a group of unsuspected Agents independently think the swarm has drifted."** The latter can recognize the case of "all metrics normal but direction wrong"—which is precisely what P3's node-by-node model cannot see.

2.2 Sybil Resistance

F3.3's attestation weighting corresponds to the "credibility signal" discussed in P26: weighted shutdown votes cannot be faked by a few spoofed identities. UDOS's stance is consistent with P26—weights are

computed with rule predicates (historical health), not dynamically estimated with learned models, and it is not implemented before the steelman contrast runs through. Swarm quorum abort is currently a design proposal, [RESULT NEEDED].

3 Direction Two: Behavioral Drift Baseline Automatic Rollback (F3.4)

3.1 From "Reacting to Fault Events" to "Reacting to Behavior Distribution"

F3.4 (Behavioral Drift Baseline & Rollback, official-website scope) of AI SAFE² Pillar 3 claims: build a **baseline** for the system's normal behavior, and when running behavior departs from the baseline beyond a threshold, **automatically roll back** to the last known healthy snapshot.

This precisely fills P3's second gap—gradual change without injection. P3's Trace hash chain is already recording each step's state; what F3.4 does is add a layer of **distribution monitoring** on this chain:

- Baseline: build a distribution with key behavior metrics from historical healthy runs (reassignment ratio, context-loss rate in handoffs, rework rate, average fan-in);
- Drift: the current window's metrics depart from the baseline distribution beyond a threshold;
- Rollback: roll back along the Trace hash chain to the last non-drifted snapshot, rather than waiting for a fault event to trigger.

The benefit of this design is **reusing UDOS's existing assets**: snapshots are already in P3, the rollback path is already in the hash chain, and what is added is only "distribution monitoring + automatic rollback trigger." It is an increment to P3's pre-registered RCT—extending guardrails from "event counts" to "distribution departure."

3.2 Connection with P3's Extreme Scenario

P3 in the "all-domain faults + zero retry budget" extreme scenario **honestly triggers kill-switch and reports non-completion**—this is where it is honest. F3.4's drift rollback is not meant to replace this honest failure, but to intervene at an earlier stage of "not yet extreme but already drifting," letting the system roll back **before** entering the extreme scenario and reducing the number of times it reaches kill-switch. It does not contradict "honest failure": after rollback fails, kill-switch still provides the bottom line.

4 Direction Three: Cascade Containment

4.1 Suppressing Influence Spread Along the Trace Path

P3's node-by-node circuit breaking isolates the fault source, but faults may jump downstream through the handoff chain. Cascade containment requires: while isolating the faulty node, **mark the affected downstream nodes along its Trace / TransferBundle path**, giving special handling to these nodes' in-flight work orders (down-weighting, re-checking, or temporary isolation), rather than letting them keep working with polluted context.

This direction reuses P10's TransferBundle (P10 has implemented the six-field handoff contract, 26 contract tests verified): the handoff chain itself is a "who passed to whom" graph, and cascade containment only needs to do **bounded-radius influence-spread marking** from the fault source on this graph, with nodes outside the radius unaffected—avoiding the overreaction of "one broken, all stop."

5 The Three-Layer Recovery Structure and Its Relationship with P3

Table 1 Extension mapping from P3 node-by-node governance to Pillar 3 three-layer recovery

Failure form	P3 current response	Pillar 3 extension	Reused UDOS component	Status
Single-node crash/drop	AgentBreaker three-level circuit breaking	—	<code>circuit_breaker.py</code>	implemented (cpu-proto)
Byzantine	QA committee + quality-failure accumulation	—	<code>consensus.py</code>	implemented (cpu-proto)
Swarm joint induction	single-point kill-switch rule	F3.3 Swarm quorum abort + attestation weighting	P16/P26 committee mechanism	design proposal, not implemented
Drift without injection	none (only reacts to events)	F3.4 drift baseline automatic rollback	P3 hash-chain Trace snapshots	design proposal, not implemented
Fault cascade spread	node-by-node isolation	bounded-radius containment along the TransferBundle path	P10 handoff chain graph	design proposal, not implemented
Utterly unrecoverable	kill-switch honest failure	not replaced, as bottom line	<code>governance.py</code>	implemented (cpu-proto)

The judgment of Table 1 is clear: **P3 makes the fault governance of "single-node, known-type" into a measurable closed loop; what Pillar 3 fills are the three more hidden failure forms of "swarm-level, gradual, cascade."** All three can connect to UDOS's existing components without starting anew—this is why they can be registered as "design extensions" rather than "brand-new systems."

6 Dross / Failure Boundaries / Falsifiable Conditions

Dross (what not to do):

1. **Do not write Swarm quorum abort as "a more sensitive kill-switch."** Its value is not igniting more frequently but recognizing "metrics normal but direction wrong"—if after implementation it only lowers the threshold, it degenerates into a more aggressive kill-switch, losing the meaning of healthy majority judgment.
2. **Do not let drift rollback replace honest failure.** Rollback intervenes in the early drift stage; when the system has already entered P3's extreme scenario, the correct behavior is still kill-switch honestly reporting non-completion, not infinite rollback.
3. **Do not make cascade containment into "one broken, all stop."** Influence spread must have a bounded radius, with no reaction outside the radius; unbounded containment amounts to magnifying a single fault into a full-matrix shutdown.

Failure boundaries:

- Swarm quorum abort's attestation weighting still faces the "weight-estimation manipulation" problem discussed in P26—healthy Agents themselves may be induced, and quorum abort cannot distinguish "the healthy majority really drifting" from "the healthy majority being colluded";

- The drift baseline has no defense against "fault types that have never appeared"—the baseline only knows "departure from the known healthy distribution," not "whether the departure is a brand-new attack";
- Cascade containment's bounded radius is a prior parameter: too small a radius misses real downstream pollution; too large a radius over-isolates.

Falsifiable conditions:

- If Swarm quorum abort in a "swarm joint induction" injection experiment neither lowers the mis-trigger rate nor the miss rate compared to P3's single-point kill-switch, the benefit hypothesis of the F3.3 extension is overturned;
- If drift baseline rollback's mis-rollback rate on fault-free normal runs is significantly higher than the number of drift events it captures, F3.4's side effects outweigh its benefits;
- If cascade containment's downstream mis-isolation rate on injected cross-node pollution is not significantly lower than node-by-node isolation, F3 direction three has no engineering value in the UDOS scenario.

7 Conclusion

v7.5.0 P3 makes fault governance into a measurable closed loop that "can recover, can isolate, can honestly fail"; AI SAFE² Pillar 3 reminds us this closed loop still lacks three layers—swarm-level quorum abort, drift baseline automatic rollback, cascade containment. This paper's stance is in the same line as P3: **first register these three things as design extensions, connecting to existing components, rather than immediately implementing.** The reason is the same as P26—the reason P3's pre-registered RCT is credible is that its governance mechanisms are independently switchable and causally measurable; any new mechanism (quorum abort, drift detection, cascade containment) must first become an "independently switchable, marginally measurable" factor before entering the primary-metric experiment.

A system that cannot recognize the swarm jointly drifting, cannot automatically roll back when quietly drifting, and cannot suppress fault spread along the handoff chain—even if it can complete node-by-node—is not an honest fail-safe system. But before putting these mechanisms into safety-critical paths, it must first prove they are not "more sensitively mis-triggering" under a steelman contrast—consistent with P3's usual restraint.

References

Classical literature

1. Castro, M., & Liskov, B. (1999). Practical Byzantine Fault Tolerance. OSDI 1999. (the classical skeleton of fault tolerance and committee majority judgment)
2. Microservice circuit breaker pattern (circuit breaker: closed/open/half-open, error-rate threshold, cooldown probe). [CITATION NEEDED: the original source of the microservice circuit breaker pattern pending verification]

2026 frontier materials (existence verified; thresholds are framework claims, not reproduced in UDOS)

1. Cyber Strategy Institute (2026). AI SAFE² governance framework v3.0 (2026-05-18), **Pillar 3 Fail-Safe & Recovery**: F3.3 Swarm Quorum Abort (coordinated shutdown at 25%+ abnormal, cryptographic attestation weighting against Sybil), F3.4 Behavioral Drift Baseline & Rollback (automatic rollback on behavioral drift baseline), circuit breaker/circuit breaking/cascade containment.

UDOS paper volume (evidence grades marked in text)

1. UDOS v7.5.0 P3 "Causal Effect of Fault-Breaker Governance on Agent Matrix Completion Rate" (three-level circuit breaking / snapshot rollback / reassignment + unique Owner / hash-chain Trace / explicit Stop Condition; primary metric completion rate accepted/total; 24 work orders, 9 specialists, 7 QA including 2 Byzantine, cpu-proto single-seed pilot).
2. UDOS v7.5.0 P10 "AgentAsTool and the Measurability of TransferBundle Handoff Information Loss" (six-field handoff contract, 26 contract tests verified).
3. UDOS v7.6.0 P16 "Typed Finality", P26 "Weighted BFT and Reputation" (the mechanism source of Swarm quorum abort and attestation weighting).

Evidence Discipline and Reproduction Notes

- This paper is a **design extension** paper with no new experiment; UDOS's current `udos7/topology/circuit_breaker.py`, `governance.py`, and `consensus.py` only implement P3's node-by-node governance and **do not implement** Swarm quorum abort, drift baseline detection, or cascade containment.
- P3's existing numbers are cpu-proto single-seed pilot point estimates (24/24 completion, 3 nodes isolated, 5 reworks); "all-domain faults + zero retry budget triggering kill-switch" is an extreme scenario constructed by design, whose exact accepted count and governance-item details await re-running (P3 text marks [RESULT NEEDED]) and are not within this clean/stressed two runs.
- The 25% threshold, attestation weighting, and drift baseline of AI SAFE² Pillar 3 are **framework claims, official-website scope**, not reproduced in UDOS; this paper only does mechanism mapping and does not claim these thresholds are optimal in the UDOS scenario.
- The impact of the three extensions on completion rate/mis-isolation rate/mis-trigger rate is all [RESULT NEEDED] ; if implemented, they must, per P3's pre-registered RCT discipline, become independently switchable factors, with ≥ 30 seeds before submission.



Rules Before Learning: The Robustness Boundary Between Deterministic Predicates and Learned Encoders in Adversarial Safety-Critical Paths

Working Title (EN): Rules Before Learning: The Robustness Boundary Between Deterministic Predicates and Learned Encoders in Adversarial Safety-Critical Paths

Consensus governance paper P32 • critical flagship of the whole volume • generalizing the H-CSC v2 self-withdrawal lesson into a design principle for UDOS's checkpoints • sister paper to P16 (typed finality), P26 (weighted BFT), and P31 (fail-safe recovery) • UDOS Reasoning Engine v7.6.0 • Fang Wenxin • 2026-09-21

One sentence first: On the safety-critical path of "deciding whether to trust an Agent product," **do not default to "using a neural network for semantic judgment" being more advanced than "using rules for mechanical checking."** The adversarial experiments of H-CSC v2 prove: the AUROC of the deterministic lexical consistency predicate (0.865–0.982) is significantly better than the learned semantic encoder (0.621–

0.744). This paper generalizes this negative result into a design principle—**rules as base, learning as supplement**: all safety-critical checks (P2 stop, P10 handoff, P12 evidence grading) are first based on explainable, reproducible, CI-able rule predicates, and learned models can only "supplement" after the rules have already held the bottom line, and must pass a steelman contrast before entering the safety path.

Evidence scope (stated only once in the whole paper): The core external evidence is H-CSC (arXiv: 2606.07316, v2 retitled Certifiable Semantic Agreement Among LLM Agents: What the Admissibility Instrument Decides); its "lexical predicate AUROC 0.865–0.982 beats learned CRSE encoder 0.621–0.744" and "actively withdrawing honest retention advantage (aggregation distance 5.32°/5.48° vs 4.27°/4.32°, honest retention 0.6475 vs 0.7325)" are all **paper-reported, not independently rechecked**. UDOS-side numbers go back to v7.5.0 P2/P10/P12 (cpu-proto / verified). This paper newly creates no measurements; "rules first" is a design principle, not a contrast experiment already run at all checkpoints. Verification date 2026-09-21.

Abstract

Multi-Agent engineering in 2026 has an intuitive preference: facing questions like "whether two natural-language outputs are semantically consistent" or "whether this handoff is complete," the first reaction is "train/call a semantic encoder to judge"—because neural networks "understand semantics better." The adversarial experiments of H-CSC v2 give a **counterintuitive but key** negative result: in that paper's adversarial scenarios, when deciding whether two texts are semantically consistent, the **simple deterministic lexical consistency predicate** (mechanical comparison by an agreed set of keywords/phrases) has AUROC **0.865–0.982**, while the **learned CRSE semantic encoder** (encoding text into a vector space and then computing similarity) has AUROC only **0.621–0.744**. The tool that "understands semantics better" is instead worse under adversarial conditions.

H-CSC v2 also does one more thing worth recording: after re-checking with a stricter steelman baseline, it **actively withdrew** its own v1 claim that "filtered aggregation retains more honest content"—its core aggregation is 5.32°/5.48° from the honest center, while the steelman baseline is only 4.27°/4.32°; honest retention against steelman M3 is 0.6475 vs 0.7325, and the authors explicitly write "we withdraw it." These two negative results together point to a judgment UDOS must write into its design discipline: **on safety-critical paths, a "learned module" is not the default better solution; it must first prove itself, under a fair baseline and under adversarial conditions, to really win over the simplest rule opponent.**

This paper generalizes this judgment into the "**rules as base, learning as supplement**" principle and lands it on three existing UDOS safety checkpoints: (1) P2 stop decision—equivocation whole-round invalidation and the $\$2f+1\$$ quorum are pure rules, correct; if a "learned semantic-consistency judgment" is introduced to decide the stop level, it must first be based on a rule lexical predicate; (2) P10 TransferBundle—the six-field mechanical check itself is a rule predicate, which is precisely why its 26 contract tests are verified, and do not use a learned model "approximation" to replace six-field non-empty checking; (3) P12 evidence grading—the grading criteria of verified / cpu-proto / cpu-proxy / unverified are rules, and learned models can only give "supplementary hints" after grading is complete, not decide the grading. This paper also draws the boundary of this principle: **rules first is not "rules are omnipotent"**—rule predicates have limited coverage, and for open semantic judgments rules cannot understand, learned models are a necessary supplement; the principle is "rules hold the safety bottom line, learning handles flexibility beyond rules," rather than "use rules for everything."

Keywords: rule predicate; learned encoder; adversarial robustness; steelman baseline; negative result; safety-critical path; evidence grading

Structured Abstract (Background problem → Argument → Evidence → Contributions)

- **Background problem:** Multi-Agent engineering defaults to "neural networks for semantic judgment, rules only for structured fields"; but H-CSC v2's adversarial experiments show that under adversarial conditions the learned semantic encoder is significantly defeated by the simple lexical predicate (AUROC 0.621–0.744 vs 0.865–0.982), and its learned aggregation module actively withdraws its positive claim under the steelman baseline.
- **Core argument:** Safety-critical paths should be "**rules as base, learning as supplement**"—first use explainable, reproducible, CI-able deterministic rule predicates to hold the bottom line, and learned models only supplement after the rules hold the bottom line, and must pass a steelman contrast before entering the safety path.
- **Evidence threads:** (1) the lexical predicate vs learned encoder AUROC contrast and active withdrawal of H-CSC v2 (paper-reported); (2) the existing designs of UDOS P2 (equivocation whole-round invalidation as pure rules, verified), P10 (TransferBundle six-field mechanical check, 26 contract tests verified), and P12 (evidence grading as rule criteria).
- **Contributions:** (i) generalizing H-CSC's single-point negative result into a cross-checkpoint design principle; (ii) landing on the three concrete checkpoints P2/P10/P12 to give "where pure rules are required and where learning can supplement"; (iii) drawing the boundary and falsifiable conditions of "rules first $\neq$ rules omnipotent."

1 A Default Intuition: Semantic Judgment Should Use Neural Networks

1.1 Where the Intuition Comes From

In LLM engineering there is an almost default judgment: "judging whether two texts are semantically consistent" is a **semantic understanding problem**, and semantic understanding is a neural network's strength and rules' weakness. So facing such questions, the engineering first reaction is almost always "put on a semantic encoder"—encode the two texts into vectors, compute cosine similarity, and treat them as consistent above a threshold.

This intuition often holds in **non-adversarial** scenarios: within the training distribution, without an attacker deliberately constructing samples that are "literally different but semantically induced consistent" or "literally the same but semantically polluted," the learned encoder indeed understands semantics better than keyword matching.

1.2 But Safety-Critical Paths Are Adversarial Scenarios

The problem is: **the path deciding "whether to trust an Agent product" is precisely the path an attacker most wants to manipulate.** The attacker will not play by the training distribution—he will deliberately construct text where "literal keywords are all correct but semantics are quietly shifted," making the learned encoder (which has seen too many "keyword correct = semantic correct" samples) misjudge as consistent. At this point, the learned model that "understands semantics better," precisely

because it has learned too many statistical associations, is less sensitive to samples carefully designed by the attacker.

H-CSC v2's experiments are done precisely in this adversarial scenario. Its conclusion is a reversal of the above intuition.

2 H-CSC v2's Two Negative Results: This Paper's Evidence Base

2.1 The Lexical Predicate Defeats the Learned Encoder

H-CSC v2 (arXiv:2606.07316v2, paper-reported, not independently rechecked) compares under adversarial scenarios two tools for "judging whether two natural-language outputs are semantically consistent":

- **Deterministic lexical consistency predicate:** mechanical, deterministic comparison by an agreed set of keywords/phrases;
- **Learned CRSE semantic encoder:** using a neural network to encode text into a vector space and then compute similarity.

The result (paper-reported numbers): lexical predicate AUROC **0.865–0.982**, learned encoder AUROC **0.621–0.744**. The simple rule tool, under adversarial conditions, is significantly better than the learned model that "understands semantics better."

Why? A reasonable explanation is: the learned encoder's judgment basis is **statistical association**, and the attacker's goal is precisely to break these associations—to decouple "keyword correct" from "semantic correct"; the lexical predicate's judgment basis is **explicitly agreed predicates**, and the attacker either satisfies the predicate or does not, with no "statistically close" fuzzy space. In adversarial games, **rules that are clear, predictable, and precisely computable by the attacker are instead harder to exploit than learned models that are "fuzzy but look smart."**

2.2 Active Withdrawal: Worse Under the Steelman Baseline

H-CSC v2's second negative result is more important than the first, because it is not "someone else's tool failing" but **the authors' self-denial of their own tool**. v1 once claimed its "filtered aggregation" is better than a simple majority at retaining honest content; after re-checking with a **steelman baseline** (giving the comparison side a stronger, fairer opponent), v2 found (paper-reported):

- Core aggregation distance from the honest center: H-CSC **5.32°/5.48°**, steelman baseline **4.27°/4.32°**—worse;
- Honest retention rate: against steelman M3 **0.6475 vs 0.7325**—worse;
- The authors explicitly: **"we withdraw it."**

The methodological value of this withdrawal is: it demonstrates **a learned/aggregative module, under a fair baseline, may not only bring no benefit but be worse**. If H-CSC v2 had not done the steelman contrast and directly used v1's "we are better" conclusion, this negative result would never have been recorded. What UDOS learns here is not "don't use learned models" but—**before introducing any learned module, one must first ask: who is its steelman opponent? Did it really win?**

3 Generalization: Rules as Base, Learning as Supplement

3.1 Statement of the Principle

Generalize H-CSC v2's two negative results into one design principle:

Rules as base, Learning as supplement

1. The **bottom line** of safety-critical paths (paths deciding "trust or not, stop or not, accept or not") must be borne by deterministic, explainable, reproducible, CI-able rule predicates;
2. Learned models only, after the rule predicates have already held the bottom line, **supplement** the open semantic judgments rules cannot understand;
3. Any learned module that wants to enter a safety-critical path must first, under a **steelman contrast** and under **adversarial conditions**, prove it is better than the simplest rule opponent; if it cannot prove it, it stays outside the rules.

The reason for this principle is not "learned models are bad" but the **engineering trade-off of auditability and adversarial robustness**: each judgment of a rule predicate can be reviewed by human eyes, re-run by CI, and have its boundary precisely computed by an attacker; learned models' judgments are black boxes whose failure modes under adversarial conditions are hard to predict. The first requirement of a safety-critical path is not "smartest" but "**when something goes wrong it can be found, reproduced, and regressed.**"



Figure 7 The bottom layer is deterministic rule predicates (safety bottom line), the upper layer is learned models filling in, and entering the safety path requires passing the steelman contrast threshold. Conceptual illustration.

3.2 Why Not "Use Rules for Everything"

This principle has an often-misread opposite that must be blocked first: **rules first is not rules omnipotent**. Rule predicates have limited coverage—they can only judge "whether the pre-agreed

predicates hold"; for open semantics rules cannot understand (such as "whether this proposal's argument is self-consistent"), learned models are a necessary supplement. The principle is not "ban learned models" but "**put learned models behind the rule defense line, not in front of it.**"

4 Landing on UDOS's Three Checkpoints

4.1 P2 Stop Decision: The Pure-Rule Part Is Correct

The core safety mechanisms of v7.5.0 P2—equivocation whole-round invalidation, $\$n \geq 3f+1$, $\$2f+1$ quorum, and more than $\$f$ silence turning to view change—are all **deterministic rule predicates**. This is precisely why it can be written as 10 CI-able property tests (verified): each is "given a vote combination, the rules mechanically decide."

When the three-level typed finality proposed by P16 (`strong_stop` / `weak_stop` / `abort`) introduces the judgment of "whether the semantic core is consistent," it **must be based on a rule lexical predicate** and cannot directly use a learned semantic encoder—H-CSC v2 has proven the latter has lower AUROC under adversarial conditions. That is: P16's `strong_stop` judgment should preferentially use "rule comparison of semantic-core fingerprints," and the learned model only acts as a "supplementary hint when rules are uncertain," and must not alone decide the stop level.

4.2 P10 TransferBundle: The Six-Field Mechanical Check Is a Rule Predicate

The TransferBundle handoff contract of v7.5.0 P10 requires six fields non-empty and mechanically checkable, with 26 contract tests verified. This itself is a "rules as base" example—whether a handoff is complete does not rely on "semantically looking clearly passed" but on "all six fields present and formats correct."

P10's discipline should be explicitly written as: **Do not use a learned model "approximation" to replace six-field non-empty checking.** A learned model saying "this handoff is semantically complete" cannot replace the CI-able hard assertion of "all six fields present." Learned models can be used to hint "this handoff, although fields are complete, may semantically have lost context," but they cannot decide "whether the handoff is compliant"—compliance is a rule.

4.3 P12 Evidence Grading: The Grading Criteria Are Rules

v7.5.0 P12 divides evidence into four levels `verified` / `cpu-proto` / `cpu-proxy` / `unverified`, and the grading criteria are **rules** (whether CI ran, whether single seed, whether `cpu-proto`). This is the third landing point of rules as base:

- Learned models can be used to "supplement hints"—"this evidence's wording looks like `unverified`, want to recheck";
- But the grading itself must be rules—verified is precisely "CI ran and ≥ 30 seeds," not accepting "semantically feels reliable."

The reason P12's version-number incident (a silent failure caused by a missing `v` prefix) is a counterexample is precisely that it **bypassed rule assertions**—if grading were pure rules, "version-number format wrong" should be mechanically intercepted rather than found by human eyes.

5 The Division-of-Labor Table Between Rules and Learning

Table 1 UDOS safety checkpoints: where pure rules and where learning supplements

Checkpoint	Safety bottom line (rule predicate, must be pure rules)	Learned model can fill in (after rules)	Basis
P2 stop	equivocation whole-round invalidation, $2f+1$ quorum, $\geq 3f+1$	hint stop level when rules uncertain	H-CSC lexical predicate AUROC 0.865–0.982 beats learning 0.621–0.744
P16 three-level finality	rule comparison of semantic-core fingerprints	hint strong/weak when rules fuzzy	same; learned aggregation must pass steelman
P10 handoff	TransferBundle six fields non-empty + format	hint "fields complete but semantics may have lost context"	26 contract tests verified
P12 evidence grading	verified/cpu-proto/cpu-proxy/unverified rule criteria	hint "wording suspected unverified, suggest recheck"	version-number incident counterexample
P3 fault governance	three-level circuit-breaking thresholds, kill-switch rules	hint "metrics normal but direction may be drifting"	rules CI-able, reproducible

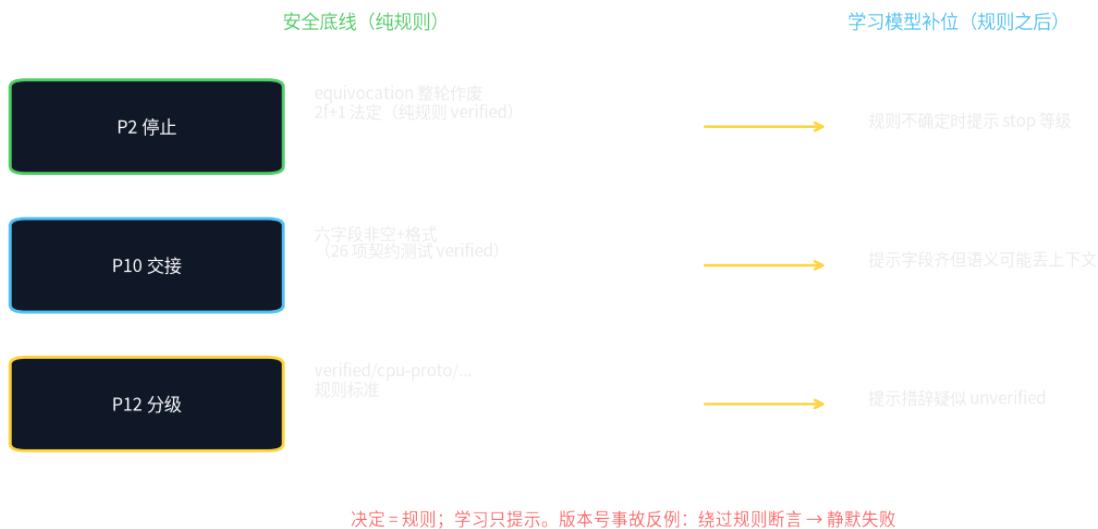


Figure 8 The P2/P10/P12 three checkpoints: the safety bottom line is borne by pure rules, and learned models only give fill-in hints. Conceptual illustration.

Table 1's judgment: **All hard judgments "deciding trust or not, stop or not, accept or not" are borne by rule predicates; the learned model's position is "hint," not "decision."**

6 Dross / Failure Boundaries / Falsifiable Conditions

Dross (what not to do):

1. **Do not understand "rules first" as "reject all learned models."** Rules have coverage, and open semantics rules cannot understand must be filled by learned models; this principle is "learning after rules," not "no learning."

2. **Do not put a learned module into the safety path without doing a steelman contrast.** H-CSC v2's withdrawal proves: learned modules may be worse under a fair baseline; going on without contrast amounts to gambling.
3. **Do not confuse "rules explainable" with "rules correct."** Rule predicates themselves may be designed too loosely (misjudging attack samples as pass); rules first requires rules to be covered by adversarial tests, not that writing rules counts.

Failure boundaries:

- When an attacker learns to **precisely mimic the satisfaction conditions of rule predicates** (literally all correct, semantics polluted), rule predicates will likewise be exploited—rules first lowers the risk of "statistical manipulation" but does not eliminate "precisely satisfying rules" attacks;
- Learned encoders in **non-adversarial, within-training-distribution** scenarios may indeed be better than rules; this principle only claims rules first on **safety-critical, adversarial** paths and is not generalized to all semantic tasks;
- H-CSC's AUROC contrast is the result of **that paper's adversarial experiment**, and changing the attack family or predicate definition may give a different conclusion—this principle is "design discipline," not a theorem that "learned models are inferior to rules in all adversarial scenarios."

Falsifiable conditions:

- If in UDOS's adversarial injection experiments the learned semantic encoder's AUROC on stop-level judgment is **significantly and stably higher than** the rule lexical predicate (under a steelman contrast), the hypothesis of "P16 based on lexical predicates" is weakened and needs re-evaluation;
- If TransferBundle's learned semantic-completeness judgment, beyond the 26 contract tests, can stably detect handoff losses rules miss, the boundary of "six-field pure rules sufficient" needs extension;
- If evidence grading's learned automatic grading in ≥ 30 -seed reproduction has a lower error rate than rule-based manual grading, "grading must be pure rules" needs loosening.

7 Conclusion

The most precious legacy H-CSC v2 gives UDOS is not its typed commit design (that is P16's material) but its two negative results: **under adversarial scenarios, the simple lexical predicate defeats the learned semantic encoder; under the steelman baseline, the authors' own learned aggregation module is withdrawn.** These two negative results together are more worth writing into UDOS's design discipline than any positive conclusion—because it corrects an intuition that is defaulted but costs on safety-critical paths: "semantic judgment should use neural networks."

This paper's conclusion is therefore restrained and clear: **rules as base, learning as supplement.** The safety-critical bottom line is borne by rule predicates that are explainable, reproducible, CI-able, and whose boundaries are precisely computable by attackers; learned models fill in after the rules hold the bottom line; any learned module wanting to enter the safety path must first, under a steelman contrast and under adversarial conditions, prove it really wins over the simplest rule opponent. P2's equivocation whole-round invalidation, P10's six-field mechanical check, and P12's evidence grading criteria are already concrete implementations of this principle—any new mechanism introduced by P16/P26/P31 must first pass this gate.

Where deciding "whether to trust an Agent product," the smartest tool is not necessarily the safest tool; a simple rule that can be re-run, reviewed, and CI-regressed is more fit to stand on a safety-critical path than a neural network that "understands semantics better" inside a black box.

References

Classical literature

1. Castro, M., & Liskov, B. (1999). Practical Byzantine Fault Tolerance. OSDI 1999. (the classical skeleton of rule-based majority judgment on safety-critical paths)
2. Vaswani, A., et al. (2017). Attention Is All You Need. arXiv:1706.03762. (the source of learned semantic representation; this paper's argument is not to deny its capability but to limit its position on adversarial safety paths)

2026 frontier materials (existence verified; numbers paper-reported, not independently rechecked)

1. H-CSC author team (2026). Certifiable Semantic Agreement Among LLM Agents: What the Admissibility Instrument Decides (v2). arXiv:2606.07316v2. (under adversarial scenarios the deterministic lexical predicate AUROC 0.865–0.982 beats the learned CRSE encoder 0.621–0.744; actively withdrawing honest retention advantage under the steelman baseline—aggregation distance 5.32°/5.48° vs 4.27°/4.32°, honest retention 0.6475 vs 0.7325)

UDOS paper volume (evidence grades marked in text)

1. UDOS v7.5.0 P2 "BFT-lite for Multi-Agent Stop Decisions" (equivocation whole-round invalidation, $\$2f+1\$$ quorum as pure rules, 10 property tests verified).
2. UDOS v7.5.0 P10 "AgentAsTool and the Measurability of TransferBundle Handoff Information Loss" (six-field mechanical check, 26 contract tests verified).
3. UDOS v7.5.0 P12 "Evidence-Grading-Driven Reproducible Agent Systems Engineering" (verified / cpu-proto / cpu-proxy / unverified rule grading; version-number incident counterexample).
4. UDOS v7.6.0 P16 "Typed Finality", P26 "Weighted BFT and Reputation", P31 "Fail-Safe & Recovery" (before introducing new mechanisms, rule-predicate base and steelman contrast are required).

Evidence Discipline and Reproduction Notes

- This paper is a **design principle** paper with no new experiment; all H-CSC numbers (AUROC intervals, aggregation distance, honest retention rate) are **paper-reported, not independently rechecked**, and are the v2 actively-withdrawn scope, and this paper strictly adopts v2.
- "Rules as base, learning as supplement" is a **design discipline**, not a theorem that "learned models are inferior to rules in all adversarial scenarios"; its failure boundaries (attacker precisely mimicking rules, non-adversarial scenarios, changing attack family) are made clear in Sec. 6.
- The existing rule designs of UDOS P2/P10/P12 are cpu-proto / verified; the "P16 based on lexical predicates, P10 not replaced by learned approximation, P12 grading pure rules" proposed in this paper are all design constraints, whose pros and cons relative to learned approaches need actual measurement under a steelman contrast, marked [RESULT NEEDED] ; ≥ 30 seeds are needed before submission.